ASTO-BOT

AI Streamer Twitch Orchestrator

User Manual

Version 1.1.15 | Developed by CaptainApolloo

Table of Contents

(Please insert in Word: References → Table of Contents → Automatic Table 1)

1. What is ASTO-BOT?

ASTO-BOT — the *AI Streamer Twitch Orchestrator* — is a complete Twitch streaming tool for Windows. It connects directly to **Twitch**, **OBS Studio** and various **AI services** and lets you react to events in your stream automatically — no programming knowledge required.

With ASTO-BOT you can, among other things:

- react automatically to **follows, subs, bits, raids, channel point redemptions** and much more,
- send **AI-generated chat replies** with a character of their own,
- control **OBS scenes and sources** from an event or a script,
- create your own **overlay animations** and put them straight into OBS,
- record, store and automatically play **clips**,
- run a **points and giveaway system** for your community,
- write your own automations with the built-in scripting language **ASTOSCRIPT**.

> 💡 ASTO-BOT runs **entirely locally** on your PC. All sensitive data such as tokens and passwords is stored **encrypted** and never leaves your computer.

1.1 Where to get ASTO-BOT

- **Download:** https://github.com/CaptainApolloo/ASTO-BOT
- **Discord** — support, help and news: https://discord.gg/n2nstVwspk
- **Ko-fi** — if you would like to support the development: https://ko-fi.com/captainapolloo

1.2 How this manual is organised

The chapters follow the tabs in the program. You do not have to read them in order — but there is one chapter you should not skip: **Events**. Almost everything is controlled there, and the other chapters keep pointing back to it.

You will find two kinds of boxes in this manual. The **cyan** ones hold tips and tricks from practice. The **orange** ones are notes you really should read — they tell you what can go wrong.

2. Installation & First Start

2.1 System requirements

- **Windows 10** or **Windows 11** (64-bit)
- an **internet connection** — for the Twitch login and the AI functions
- **OBS Studio 28** or newer — optional, but required for overlays and scene control

For the AI replies you also need the API key of an AI provider. Where it is entered and which providers are supported is described in the Settings chapter.

2.2 Installation

ASTO-BOT is set up with an installer.

- Run the file **ASTO-BOT_Installer.exe**.
- The first time, choose an installation folder, for example `C:\Program Files\ASTO-BOT\`.
- Follow the instructions of the installer.
- After the installation ASTO-BOT **starts automatically**.
- On the first start ASTO-BOT creates all the subfolders it needs by itself.

> **Uninstall ASTO-BOT only** via **Control Panel → Programs → Uninstall ASTO-BOT** or with the included **uninstall-asto.exe** in the installation folder. **Never delete the folder by hand** — otherwise entries are left behind in the Windows registry.

2.3 The first start — in this order

On the first start ASTO-BOT is not connected to anything yet. You will be done fastest if you keep to this order. All steps take place in the **Settings**

(Chapter 4).

- **Connect Twitch.** Log in with **both** accounts: your **streamer account** and your **bot account**. Without this step almost nothing works.
- **Set up the AI.** Choose a provider, enter the API key, define the bot name and its character.
- **Connect OBS.** Enable the websocket in OBS, enter the connection details in ASTO-BOT, then create the sources for overlay, clips and photo mode.
- **Sounds and SpeakerBot** — if you want to use them.
- Run the **System Check**. It tests configuration, AI, Twitch and OBS and tells you what is still missing.

> 💡 If you add permissions later, Twitch requires a **complete re-login of both accounts**. That is normal and takes two minutes.

After that, take a look at the **Events** chapter: the Standard group that ships with ASTO-BOT contains ready-made example events — Follow, Resub, Raid and others. Take one of them, press **Simulate** and watch what happens. That is the shortest way to understand ASTO-BOT.

2.4 Updates

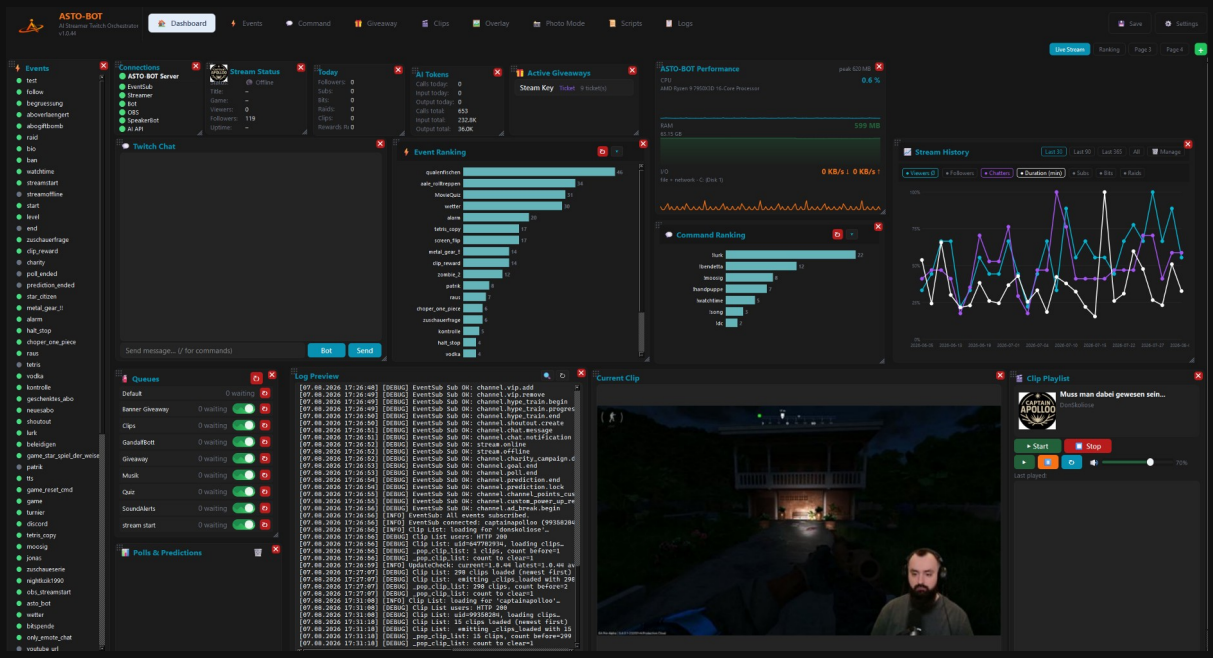
When a new version is available, ASTO-BOT tells you by itself — at the top in the header bar — The installer only overwrites **the program files**. Your configuration, sounds, overlays and prompts are kept completely.

Before the update, ASTO-BOT also puts aside a **backup of the running version**. Should anything be wrong with the new one, that gets you back at any time — section 4.11 explains how.

> While the update is running, **nothing may be interrupted**. Wait until ASTO-BOT **restarts by itself**.

3. The Dashboard

The Dashboard is the home page of ASTO-BOT and your command centre during the stream. It consists of individual **cards**, each covering one area — connections, stream status, chat, queues, statistics, clips and more. You decide which cards you see and where they sit.



The ASTO-BOT Dashboard — all cards at a glance

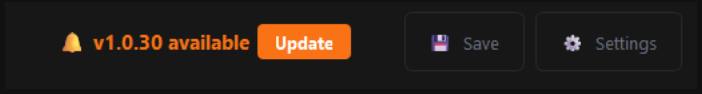
3.1 The header bar

At the very top you find the ASTO-BOT logo with the installed **version number**, and next to it the tabs you use to switch between the areas:

- **Dashboard** — the overview page described in this chapter
- **Events** — automatic reactions to Twitch events
- **Command** — chat commands
- **Giveaway** — giveaways and the points system
- **Clips** — clip management and playback
- **Overlay** — the overlay editor
- **Photo Mode** — photo mode
- **Scripts** — the ASTOSCRIPT editor
- **Logs** — the complete log

In the top right sit the buttons **Save** and **Settings**. **Save** saves everything, **Settings** takes you to the settings, where all connections and basic options are configured (see Chapter 4).

As soon as a new version of ASTO-BOT is available, a notice with an **Update** button appears to the left of the Save button.



When an update is available, the notice appears right in the header bar

3.2 Working with cards

All cards on the Dashboard can be arranged freely. So you can set up your Dashboard exactly the way you need it while streaming:

- **Move:** grab the card with the mouse and drag it where you want it.
- **Resize:** grab the card at an edge or corner and drag it larger or smaller.
- **Close:** click the red **X** in the top right corner of the card. The card disappears from the Dashboard.
- **Bring back:** the green **+** button in the top right brings closed cards back onto the Dashboard.

> 💡 Tip: cards you do not need can be closed without worry — the green + button brings them back at any time. That leaves room for the cards you really want to keep an eye on during the stream.

Four pages instead of one

The Dashboard has **four pages**. The tabs in the top right — right next to the green **+** — take you straight to the one you want. That way you can spread your cards out instead of squeezing them onto a single surface.

- **Move a card:** right-click the card, then **Move to Page X**. On cards that bring their own right-click menu — the Twitch chat card, for instance — aim for the title strip.
- **Rename a page:** right-click a tab. *Page 2* becomes *Translator* or *Debug*.
- **Switch from outside:** through the websocket, for example from a Stream Deck. The commands are in chapter 4.6.

> Cards on other pages carry on exactly as before. They only stop refreshing their display while you cannot see them — the translator keeps translating, overlays keep running, events keep firing.

> 💡 The red **X** still means "card closed", regardless of the page. A card sitting on another page is not closed — it is simply somewhere else.

3.3 Status & connection

Connections

This card shows what is currently connected to ASTO-BOT. A green dot means: the connection is up.

- **ASTO-BOT Server** — the program's internal server
- **EventSub** — the connection to Twitch through which all events arrive
- **Streamer** — your main Twitch account
- **OBS** — the connection to OBS Studio
- **Bot** — the separate bot account for chat messages
- **SpeakerBot** — the link for text-to-speech
- **AI API** — the connection to the AI

> Important: **ASTO-BOT Server**, **EventSub**, **Streamer** and **OBS** are the four connections that matter for normal operation. **Bot**, **AI API** and **SpeakerBot** are optional — ASTO-BOT also runs without them.

Stream Status

Shows at a glance whether you are live:

- **Status** — live or offline
- **Title** — the current stream title
- **Game** — the category or game currently set
- **Viewers** — the current viewer count

- **Followers** — your total number of followers
- **Uptime** — how long the stream has been running

Today

This card counts what has come in **during the current stream**: followers, subs, bits, raids, clips and **Rewards Redeemed** (how many channel point rewards have been redeemed).

> 💡 The card resets automatically with every new stream — so you always see the numbers of the current stream, not the grand total.

AI Tokens

An overview of how much the AI is consuming:

- **Calls today / Input today / Output today** — the consumption of the **current session**
- **Calls total / Input total / Output total** — the total consumption across all sessions

> Careful: the three "today" values only last as long as ASTO-BOT is open. If you close the program and start it again, they are back at 0. The "total" values, on the other hand, are kept.

3.4 Live operation

Events

The Events card lists all the events you have created. In front of each entry sits a little lamp:

- **Green** — the event is active and will fire
- **Grey** — the event is inactive

Clicking the lamp switches the event on or off. That way you can quickly mute individual events during the stream without deleting them.

Queues

The queues are the waiting lines in which actions are processed one after another. For every queue the card shows **how many events are currently waiting**.

- **On/Off:** every queue can be paused individually — with the **exception of the Default queue**, which must always stay active.
- **Reset:** every queue has its own reset button, including the Default queue. On top of that there is a button at the top of the card that resets **all queues at once**.

If a queue is switched off, it is **paused**: the events are not lost, they wait. They are processed as soon as the queue is switched back on — or they disappear if you reset the queue.

> 💡 Tip: handy when you do not want sound alerts during a quiet scene, for example: switch the queue off for a moment, back on later — everything that was waiting then runs through.

Twitch Chat

Here you write straight into your Twitch chat without leaving the program.

- The **Bot** button next to the input field switches between the **streamer account** and the **bot account** — so you decide in whose name the message appears.
- Twitch chat commands work as well, including **polls** and **predictions**.
- **Emotes and GIFs** are shown as images and animate when Twitch serves them animated, instead of sitting there as a bare name.

Polls & Predictions

Shows the polls and predictions that are currently running.

Log Preview

A compact live view of the log. Handy for keeping an eye on what ASTO-BOT is doing. The complete log is in the **Logs** tab.

3.5 Statistics

Event Ranking

A ranking of the events — it shows how often each event has fired.

> Important: **only events triggered by channel point rewards or power-ups are counted**. Events with other triggers do not appear in this ranking.

The **arrow pointing down** in the card header shows you which events are currently being counted. Clicking an entry there resets its statistics — so you can put every event back to zero individually.

Command Ranking

The same principle for the chat commands: a ranking of how often each command has been used. Here too the **arrow pointing down** shows the counted commands, and clicking an entry resets its counter.

Stream History

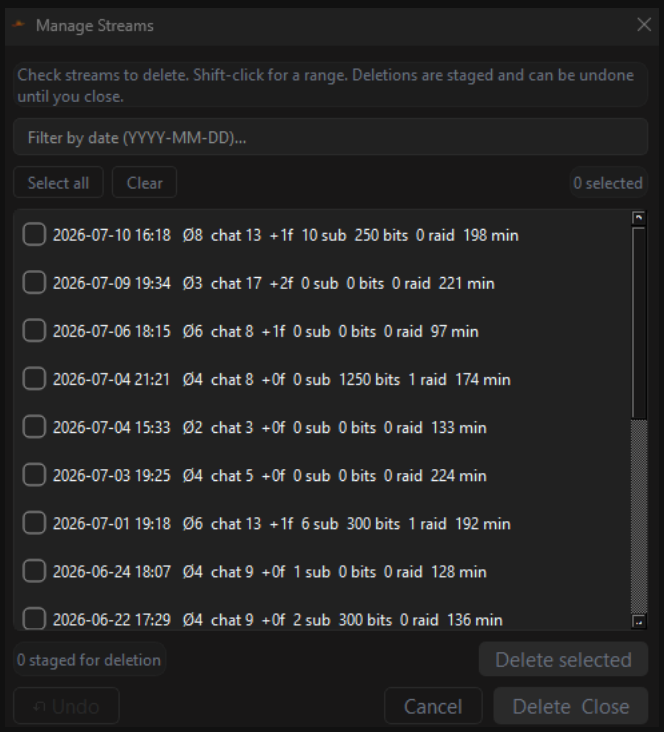
The Stream History collects the data of your past streams. It shows:

- **Viewers Ø** — average viewer count
- **Followers** — followers gained
- **Chatters** — active chatters
- **Duration (min)** — stream length in minutes
- **Subs**, **Bits**, **Raids**

With the buttons in the top right you change the period: **Last 30**, **Last 90**, **Last 365** or **All**. From 365 days on, the display is grouped into **weeks** so the curve stays readable.

Manage Streams — deleting entries

The **Manage** button in the Stream History card opens a window in which you can remove individual streams from the statistics.



Manage Streams — mark, delete and undo until the window is closed

- **Single:** tick the box next to the date you want gone.
- **Range:** click date A, then **Shift + left mouse button** on date B — everything in between is marked, no matter how far apart the two are.
- **Filter:** the field at the top lets you search for a date (format YYYY-MM-DD).

> Careful: as long as the window is open, a deletion can still be **undone**. Once you close the window, the entries are gone for good — there is no way back.

3.6 Giveaway, clips & media

Active Giveaways

Shows which giveaways are currently running.

Current Clip

Shows the thumbnail of the currently selected clip.

Clip Playlist

This card controls the clip playlist player:

- **Start / Stop** — start and stop the playlist
- **Play / Pause** — a running clip can be paused and resumed at any time
- **Repeat** — plays the clip that ran last one more time
- **Volume** — slider for the playback volume

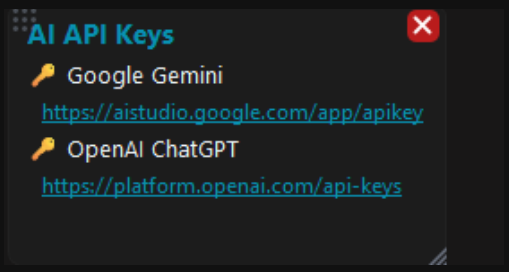
Below that, a list of the most recently played clips is built up under **Last played**.

3.7 More cards

The next two cards may be hidden by default — you bring them back onto the Dashboard with the green **+** button in the top right.

AI API Keys

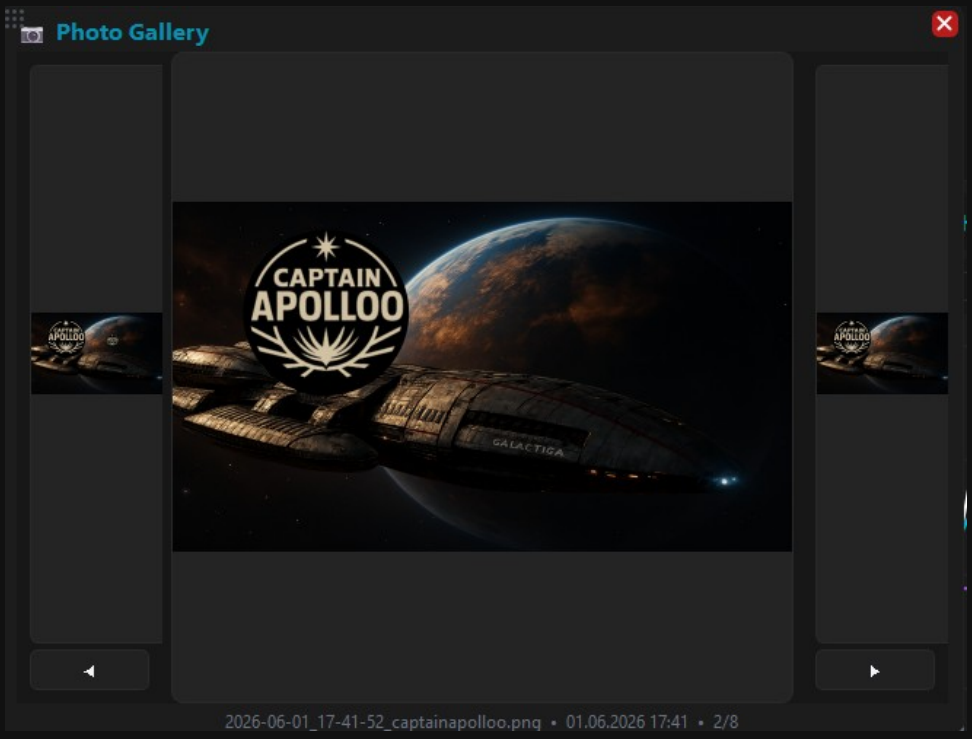
Purely a memory aid: the card holds the links to the websites where you get your API keys — Google Gemini and OpenAI ChatGPT. The keys themselves are entered in the **Settings**.



The AI API Keys card with the links to the providers

Photo Gallery

Shows the pictures taken in Photo Mode. Use the arrow keys on the left and right to page back and forth; below you find the file name, the time it was taken and the position in the gallery.

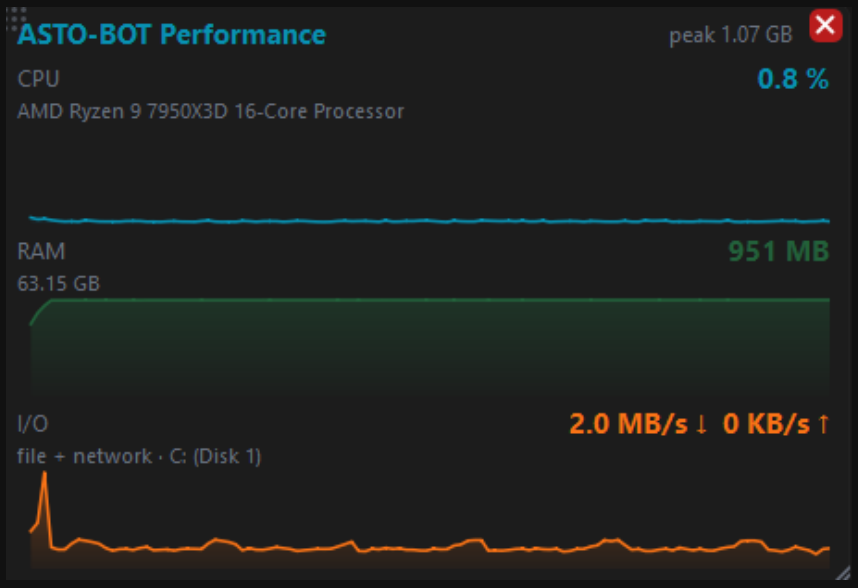


The Photo Gallery with the back and forward buttons

3.8 ASTO-BOT Performance

This card shows how much processing power and memory ASTO-BOT itself is using right now. It is hidden by default — bring it onto the Dashboard with the green **+** button in the top right.

While it is hidden nothing is measured at all. It costs you nothing if you do not need it.



The Performance card with CPU, RAM and I/O

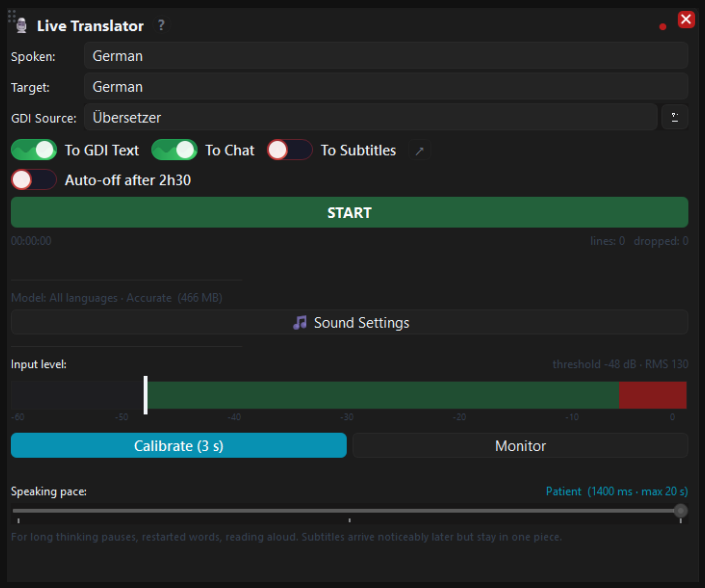
Each of the three values shows a number on the right and a two-minute history below it. That way you see not just the current moment, but also whether something is creeping up over time.

- **CPU** — the share of your processor's total capacity, the same number the Task Manager shows per program. Your CPU model is named underneath. When idle ASTO-BOT usually stays below 1 %; while an overlay plays it rises briefly.
- **RAM** — memory in use in MB, with the total installed underneath. The highest value since startup (**peak**) is shown in the top right. If the number keeps climbing over hours without ever coming down, it is worth a closer look — usually there are very large GIFs in an overlay.
- **I/O** — reading (↓) and writing (↑). The label deliberately says **file + network**: Windows reports file and network activity of a program as one combined figure, there is no split. The drive ASTO-BOT works on is named after it.

> Windows does not offer per-program GPU and network figures in a way that is both reliable and cheap enough to read continuously. That is why they are not here — three values you can trust beat five with two placeholders.

3.9 Live Translator

This card turns what you say into subtitles for your stream. Speech recognition runs **on your own PC** — no audio goes to a service, and there is no per-minute cost.



The Live Translator card

Before you can start you need a speech model and an input device. Both live in **Settings → Sounds**; the **Sound Settings** button on the card takes you straight there. The little **?** beside the card title walks you through the whole setup.

- **Spoken** — the language you speak. Only selectable with a multilingual model.
- **Target** — the language to translate into. Set it to the same as **Spoken** and you get plain captions with no translation; the AI is not asked at all in that case.
- **GDI Source** — the OBS text source to write into.

Where the text goes

Three outputs, switchable independently — even while the translator is running:

- **To GDI Text** — writes into a text source in OBS. Font, size and colour are set there.
- **To Chat** — posts the lines in your Twitch chat.
- **To Subtitles** — uses ASTO-BOT's own subtitle page. It needs **neither OBS nor a Twitch account**, which makes it the only route that works without either. The arrow beside it jumps straight to its setup.

Levels and speaking pace

The **Input level** bar shows your level. The white line is the threshold above which ASTO-BOT treats sound as speech. **Calibrate (3 s)** measures your background and sets the threshold to match — do it in silence, not while music is playing. **Monitor** turns the bar on without translating.

Speaking pace decides how long a pause may be before a sentence counts as finished. Three steps:

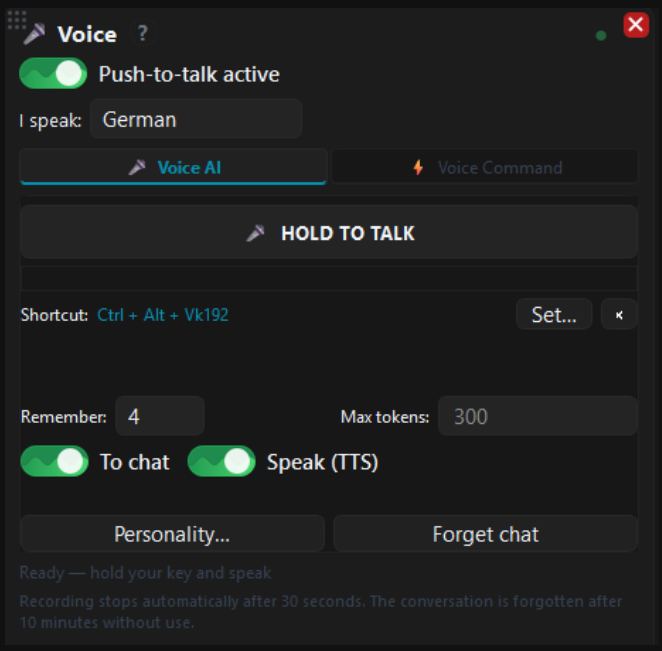
- **Quick** — short pauses, subtitles appear fastest. For fluent, uninterrupted speech.
- **Normal** — tolerates a breath or a moment of thinking mid sentence. Fits most people.
- **Patient** — for long thinking pauses, restarted words and reading aloud. Subtitles arrive later but stay in one piece.

> 💡 If subtitles are cut off mid-sentence, move one step towards **Patient**. Recognition gets noticeably worse when sentences are chopped into fragments — context is what the program works the words out from.

> Everything on the selected device is recorded. If your game audio sits on the same output, speech recognition hears it too — it cannot separate your voice from it. So send only your microphone to the bus you capture.

3.10 Voice

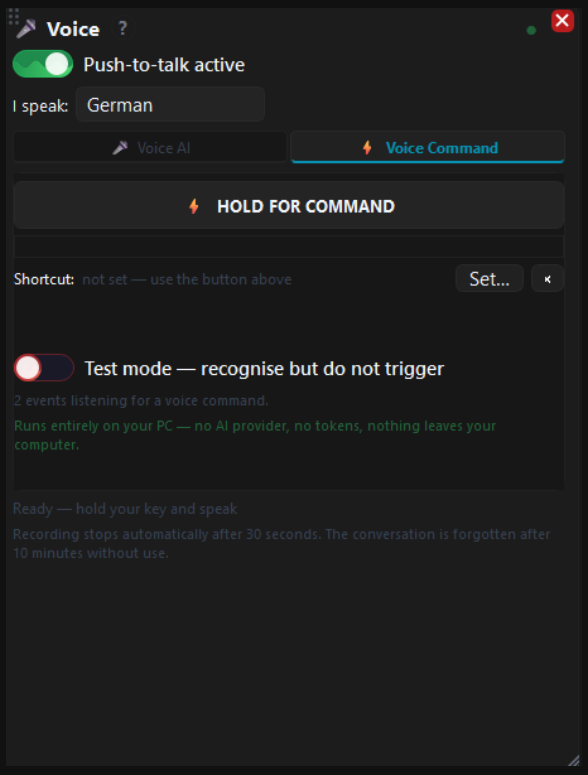
This card lets you talk to ASTO-BOT instead of typing. Two modes: **Voice AI** sends what you said to the AI, **Voice Command** uses it to trigger an event.



The Voice card with push-to-talk

- **Push-to-talk active** — must be on, or nothing is recorded.
- **HOLD TO TALK** — hold, speak, let go. The assigned key works the same way.
- **Shortcut** — the key you record with. **Set…** changes it. A key is not required, though: the button on the card, a Stream Deck and the websocket all do the same job.
- **To chat** and **Speak (TTS)** — whether the answer lands in chat and whether it is read aloud.

Voice Command is the second mode. It triggers an event by saying a sentence — no AI, no internet, everything stays on your own machine.



The Voice card on the Voice Command tab

- **HOLD FOR COMMAND** — hold, say the command, release. This tab has its **own** key, so a command never ends up at the AI by mistake.
- **Test mode** — recognises the command and shows what it *would* trigger, without doing it. That way a new phrase can be rehearsed during a live stream at no risk.
- Below the switch you see how many events are currently listening for a voice command. **0** means none has one set yet.

> The phrase is set on the event itself: **Events** → **Trigger** → 🎤 **Voice command**. Use two or three words rather than a single one — short words sound too much alike and get confused. ASTO-BOT warns you when two events have phrases that are too similar, and when in doubt it triggers **nothing** rather than the wrong thing.
>
> Voice uses **the same speech model as the Live Translator**. With none selected there, Voice cannot run either — the card tells you so directly. The **?** beside the title walks you through the setup.

Controlling a game by voice

A voice command can do more than post in chat — it can **press keys in your game**. That lets you control things without taking your hands off the mouse: in Star Citizen, for instance, raising and lowering the landing gear or toggling shields.

It takes three pieces you already know:

- An **event** with a voice command under **Events** → **Trigger** → 🎤 **Voice command**, say "landing gear up".
- **Run Script** as its action.
- The key press inside the script: `SENDKEY PRESS "N"` — or a combination such as `SENDKEY PRESS "CTRL+N"`.

> **Important:** the key has to be exactly the one the game uses for that function. ASTO-BOT only presses it; what it does is decided entirely by the game's key bindings. Check the game settings first and use the same key.

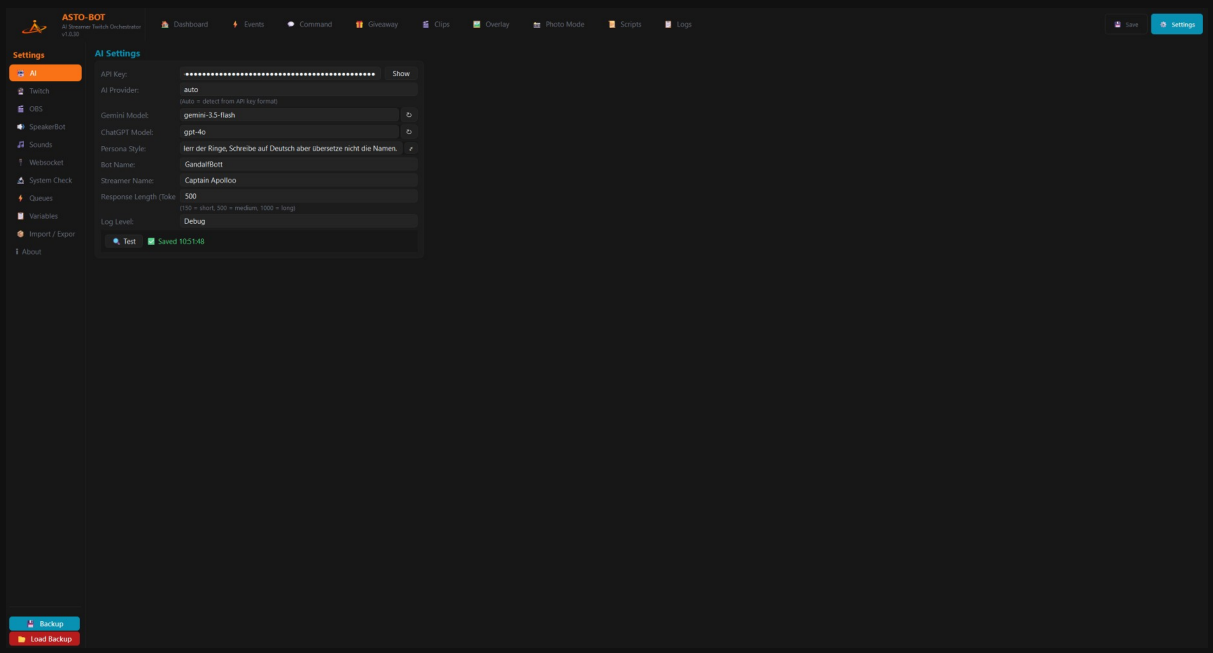
> 💡 If the key press is meant for one particular window, put `SENDKEY WINDOW "Star Citizen"` in front of it. Without that line the key goes to whatever window is in the foreground — quite possibly ASTO-BOT itself.

> Games with anti-cheat do not always accept simulated key presses. Try the command out in a quiet moment before relying on it during a stream.

4. Settings

The **Settings** are where you configure everything ASTO-BOT needs in order to work — the AI, the Twitch login, the connection to OBS, sounds and more. You reach them with the **Settings** button in the top right of the header bar.

On the left you see the areas you switch between. At the very bottom sit the two buttons **Backup** and **Load Backup**.

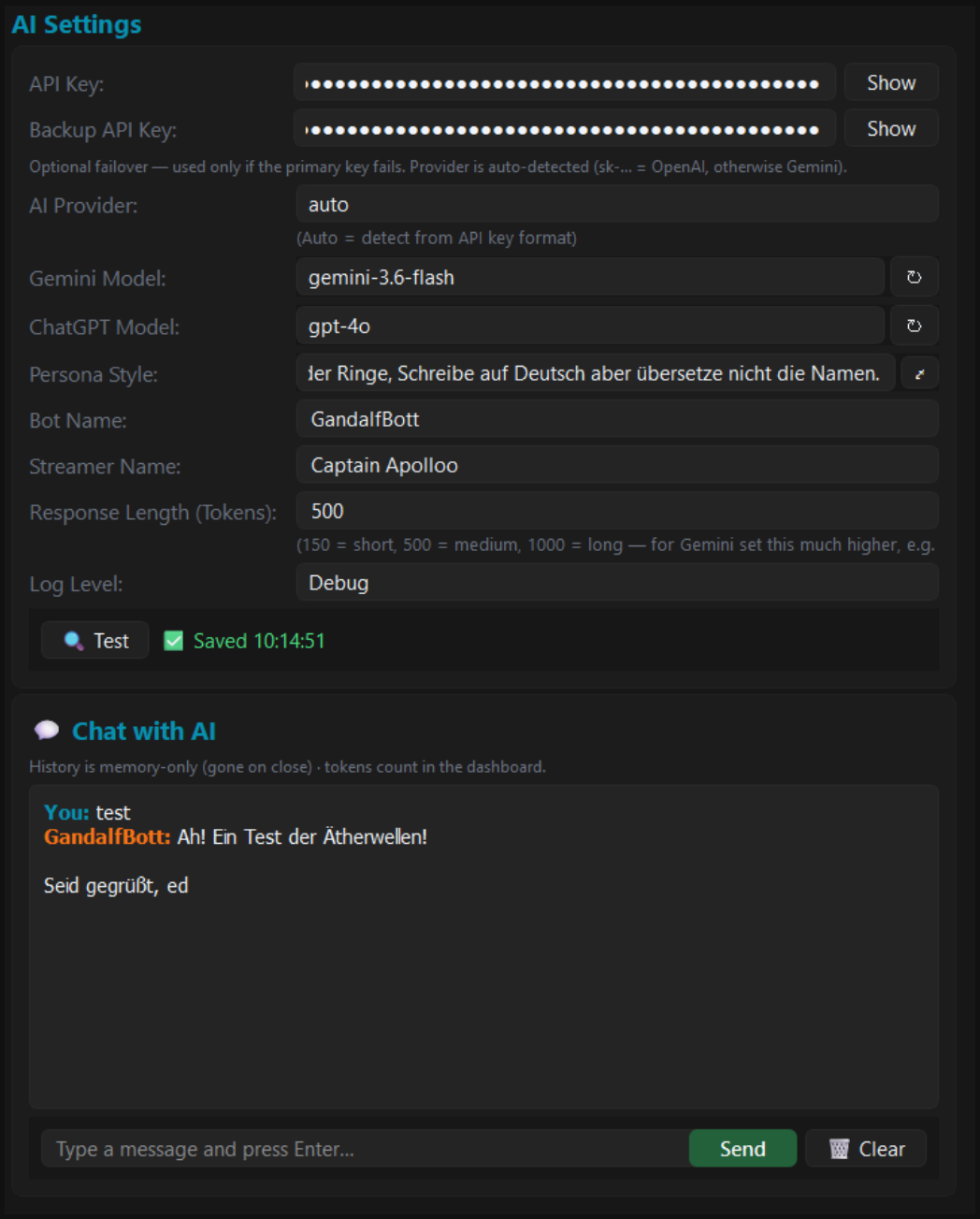


The Settings with the list of areas on the left

> 💡 To get started you need very little: **AI** (only if you want to use the AI), **Twitch** (login) and **OBS** (connection). Everything else you can look at later in peace.

4.1 AI

This is where you set up the artificial intelligence that ASTO-BOT uses to generate chat replies.



The AI settings

- **API Key** — your key from the AI provider. ASTO-BOT recognises by itself whether it is a Google Gemini or an OpenAI key. **Show** makes the key visible.
- **Backup API Key** — optional. A second key that **only steps in if the first one reports an error**. Its provider is detected automatically from the key format (`sk-…` = OpenAI, otherwise Gemini). Leave the field empty and everything behaves just like with a single key.
- **AI Provider** — normally **auto**. You can also fix the provider to **Gemini** or **OpenAI**, but that only applies to the **first** key; the backup key always detects its provider by itself.
- **Gemini Model** / **ChatGPT Model** — this is where you pick the model. The small button next to it fetches the models currently available from the provider, automatically using the matching key from either of the two fields.
- **Persona Style** — the character the bot should take on. This is the most important dial for your bot's personality.

- **Bot Name** / **Streamer Name** — these are the names under which the AI knows the two of you.
- **Response Length (Tokens)** — how long the replies may be. 150 = short, 500 = medium, 1000 = long. **For Gemini, set this much higher** (e.g. 5000): Gemini models "think" internally, and those thinking tokens count toward this limit — set it too low and the reply is cut off mid-sentence.
- **Log Level** — how detailed the log is.
- **Test** — checks whether the connection to the AI is up.

To fetch the model list, ASTO-BOT looks for the matching key in **both** fields: **Gemini Model** needs a Gemini key, **ChatGPT Model** an OpenAI key. Only when **neither** field holds a key of the right provider does the button say so — that is not a bug, just a hint that the matching key is missing.

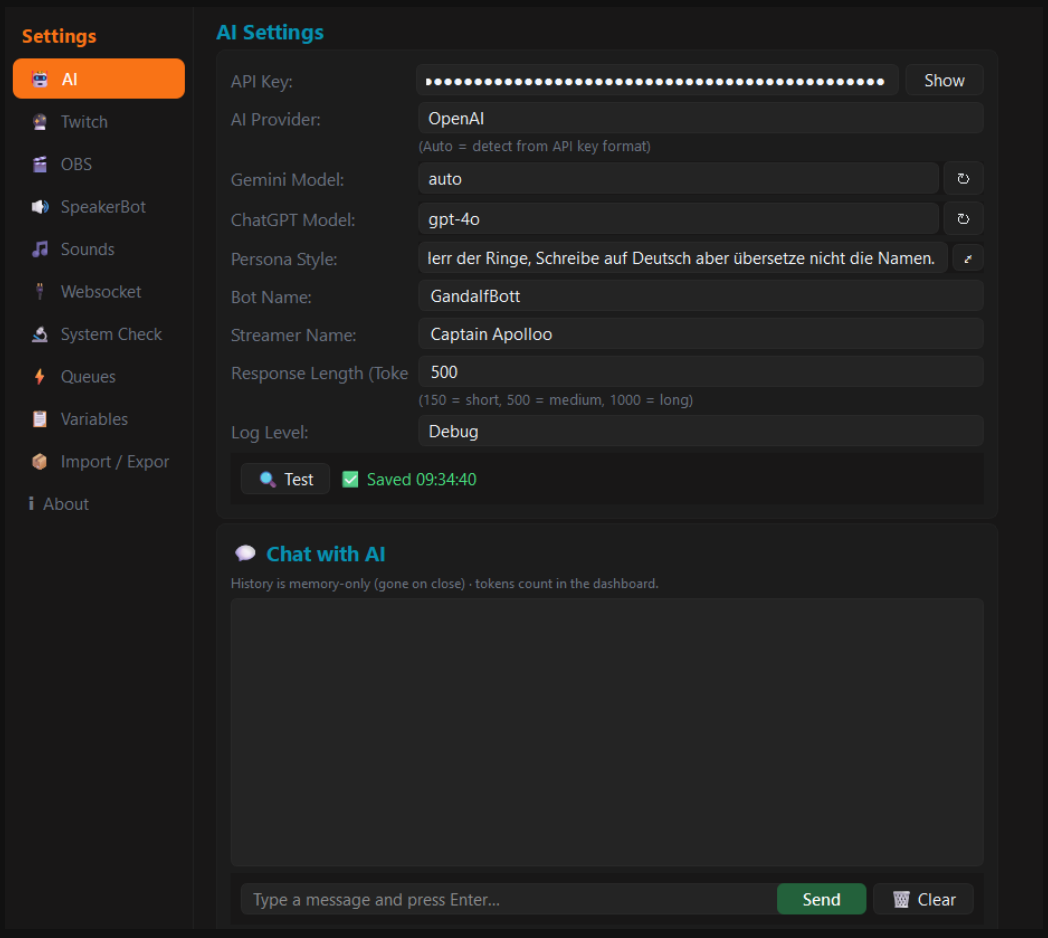
Failover: if the first key reports an error, the backup key steps in and **its** reply goes to chat; the error from the first key then only shows up in the **log**. If **every** configured key fails (or none is entered), a short notice appears in chat saying the AI is currently unavailable.

💡 About the **Log Level**: for normal operation the setting hardly matters. If you want to help the developer track down a bug, set it to **Debug** — then everything ends up in the log.

Every change in the AI settings is **saved automatically** — for text fields (keys, persona, names, token count) shortly after you stop typing, for the dropdowns immediately. A brief "Saved" confirms it. There is no separate save button here.

Chat with AI

Below the AI settings there is a small chat window in which you can **write to the AI directly** — handy for quickly trying out your persona style or the chosen model without a viewer seeing any of it.



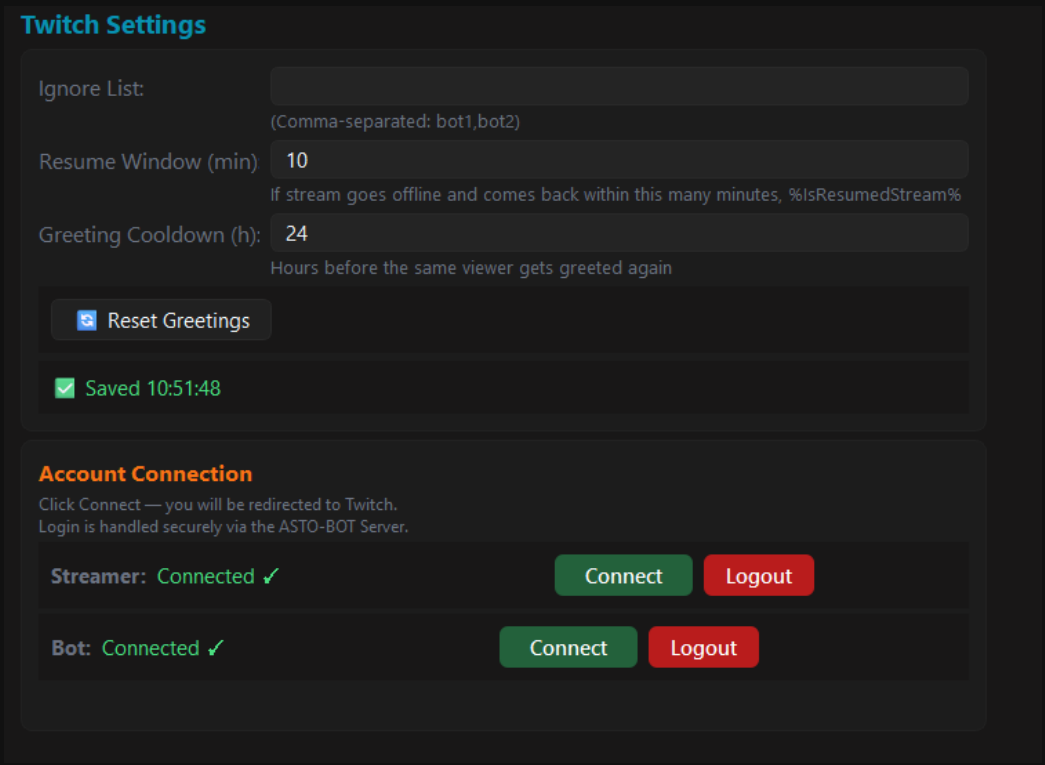
The AI settings with the chat window below

Type your message at the bottom and press **Enter** or **Send**. **Clear** empties the history.

> The chat history lives **in memory only** and is stored nowhere: as soon as you close ASTO-BOT, it is gone. The tokens it uses do show up in the dashboard, though.

4.2 Twitch

This is where you log in to Twitch and set how ASTO-BOT behaves towards your viewers.



The Twitch settings with the account login

Account Connection

A click on **Connect** takes you to Twitch, where you log in. The login runs securely through the ASTO-BOT server — ASTO-BOT never gets to see your password.

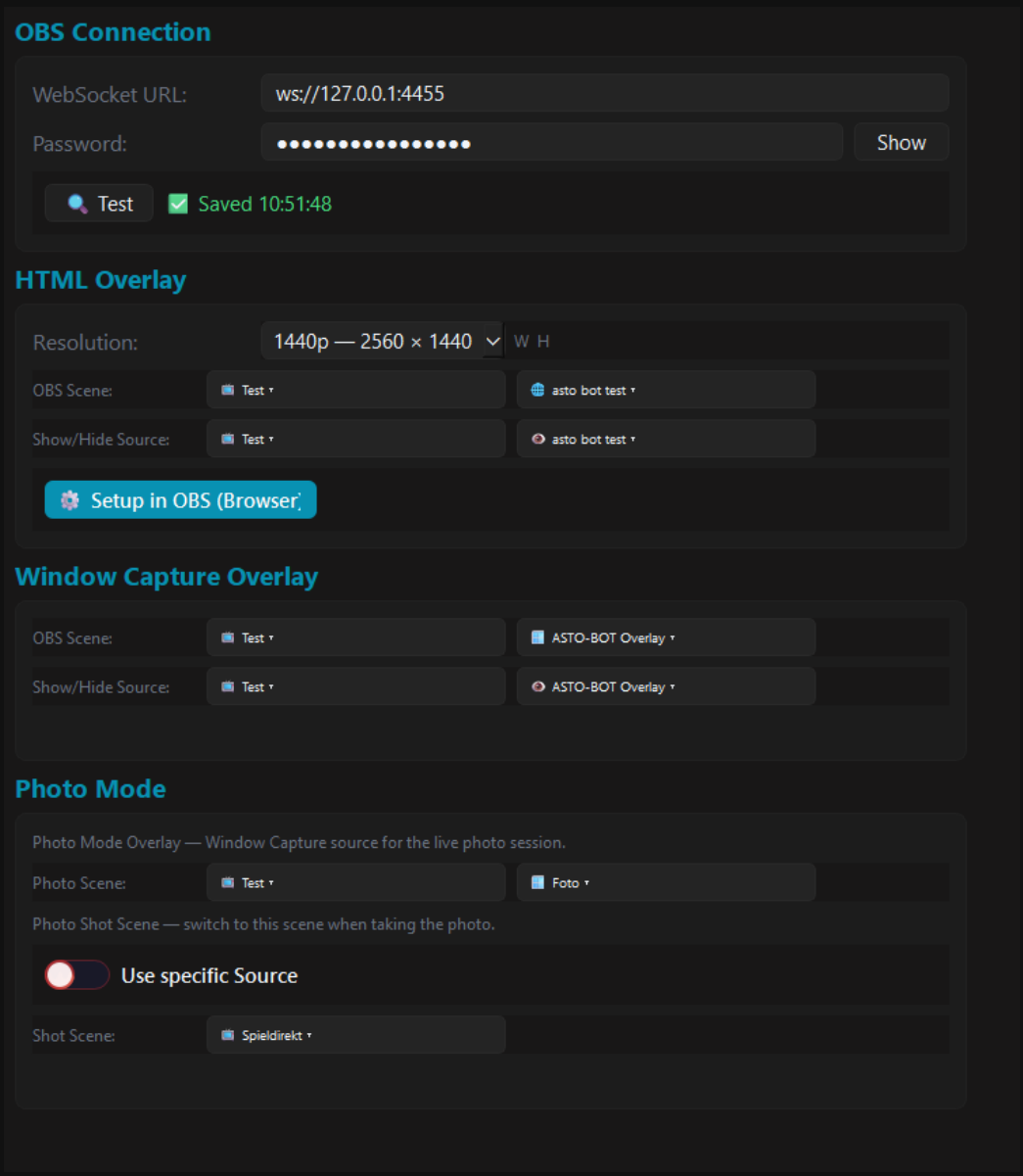
- **Streamer** — your main account. This is the login ASTO-BOT absolutely needs.
- **Bot** — a separate account under whose name the bot writes in chat. Optional, but recommended.
- **Logout** disconnects the respective account again.

Settings

- **Ignore List** — names ASTO-BOT should ignore, separated by commas. Handy for other bots in your chat so they do not trigger events.
- **Resume Window (min)** — in minutes. If your stream drops, for example because the internet goes down, and comes back within this time, ASTO-BOT counts both as **one single stream** instead of two.
- **Greeting Cooldown (h)** — in hours. That is how long it takes until the same viewer is greeted again. It belongs to the **Greeting** trigger in the Events area.
- **Reset Greetings** — resets the counter. From that click on, everyone is greeted again as if they were here for the first time.

4.3 OBS

This area is the most important one — and the one most people get stuck on. Take a few minutes here and everything else will run smoothly later.



The OBS settings: connection, HTML overlay, window capture overlay and photo mode

4.3.1 OBS Connection

- **WebSocket URL** — the default is `ws://127.0.0.1:4455`. Only change this if you have set a different port in OBS.
- **Password** — the password from the OBS websocket settings.
- **Test** — checks whether the connection to OBS is up.

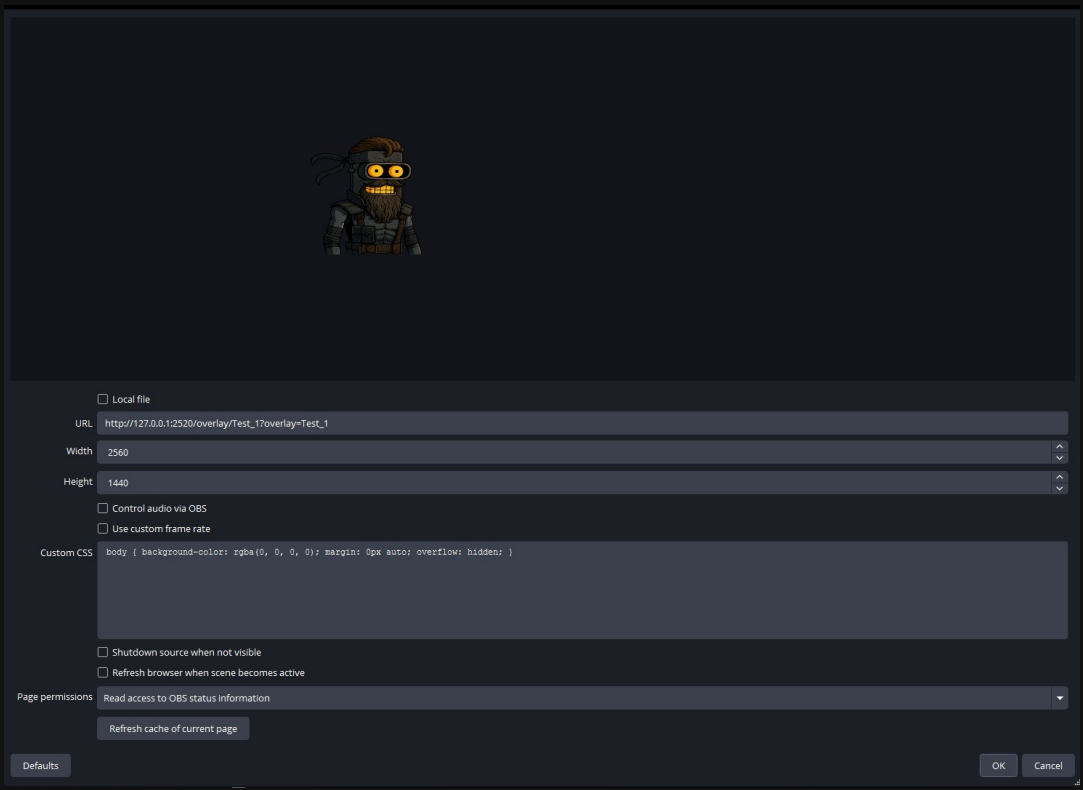
4.3.2 HTML Overlay

These settings belong to the overlay area. ASTO-BOT needs to know **where** in OBS the HTML source sits.

- **Resolution** — should match the resolution set as the default canvas in OBS (usually the one of your monitor).
- **OBS Scene** — on the left you choose the **scene**, next to it the **source** inside it. That is the browser source with the overlay.

- **Show/Hide Source** — which scene and source are shown and hidden when the overlay starts. This is deliberately separate: sources can be nested, and under some circumstances the HTML source is supposed to appear in a completely different scene.
- **Setup in OBS (Browser)** — updates the chosen HTML source in OBS.

This is what the browser source looks like in OBS:



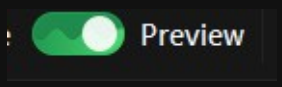
The browser source in OBS — ASTO-BOT enters the URL by itself

> Important: the tick next to **Local file** must **not** be set. The URL is entered by ASTO-BOT automatically (`http://127.0.0.1:2520/overlay/...`).

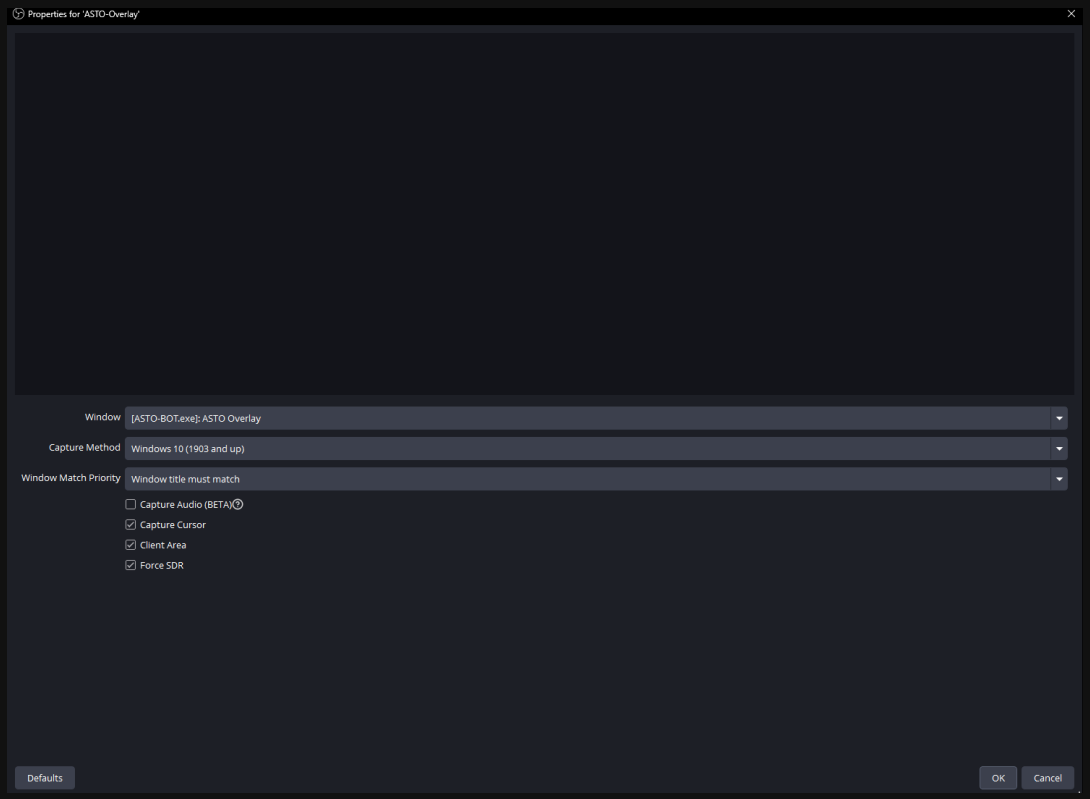
4.3.3 Window Capture Overlay

The second method: instead of a browser source, OBS captures a window of ASTO-BOT. For that a **window capture** must be created in OBS.

For the window to show up in the OBS list at all, it has to be open: switch the **Preview** toggle on in the **Overlay** area while you set up the source in OBS.



The Preview toggle in the Overlay area



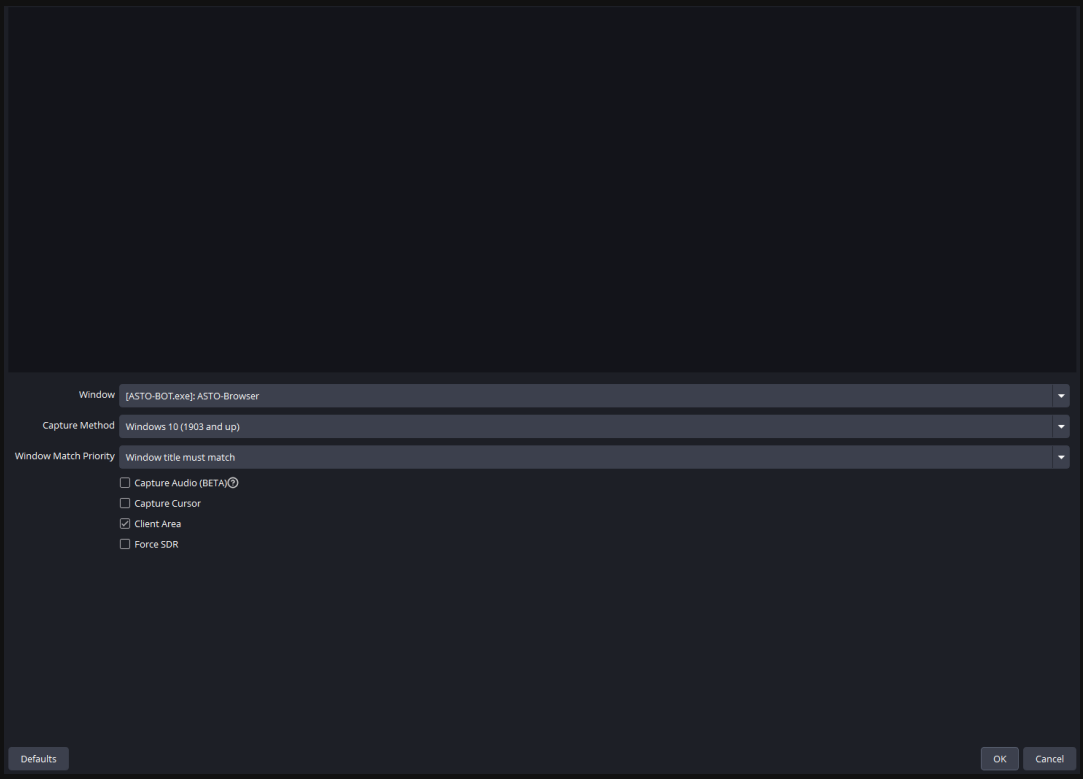
The window capture for the overlay in OBS

- **Window** — `[ASTO-BOT.exe]: ASTO Overlay` must be selected.
- **Capture Method** — must be set to **Windows 10 (1903 and up)**.
- **Window Match Priority** — set it to **Window title must match** so OBS does not accidentally capture a different window.
- Everything else is optional.

In the settings you then enter **OBS Scene** (scene + source) and **Show/Hide Source** again — exactly as with the HTML overlay.

4.3.4 Browser Overlay

The **Browser Overlay** (Chapter 9.8) is a separate window of ASTO-BOT and therefore needs its **own** OBS source. It is set up — exactly like the Window Capture Overlay — as a **Window Capture**, just with a different window.



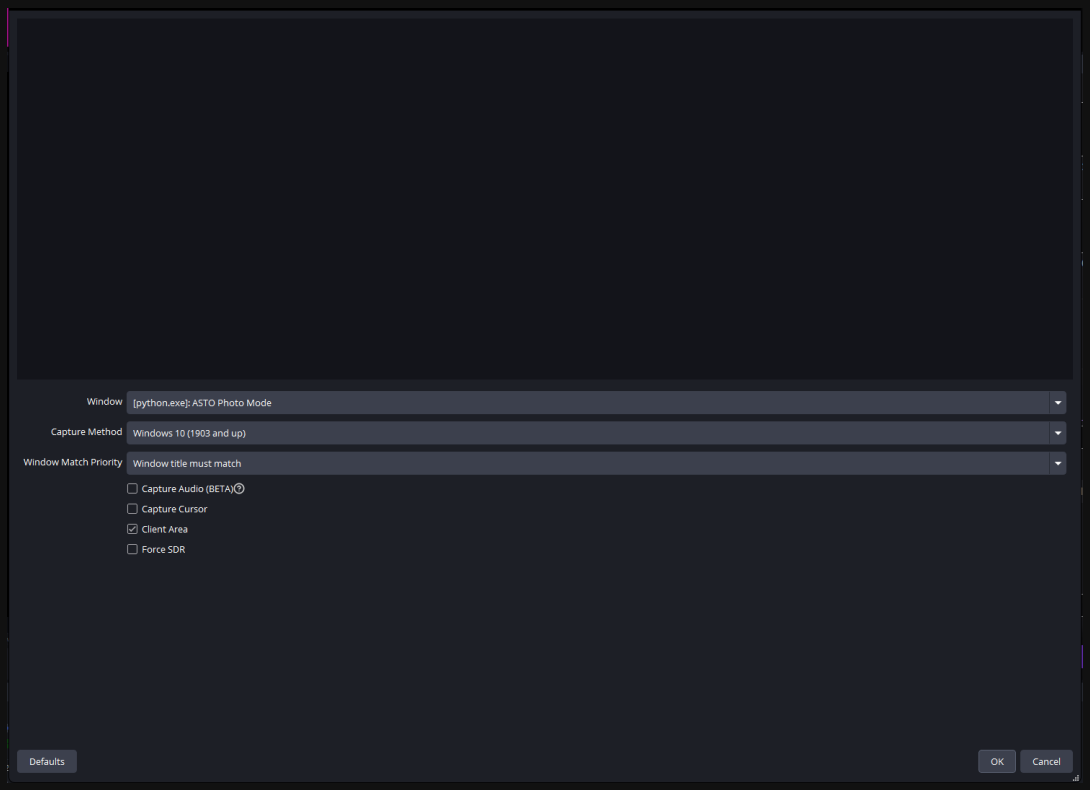
The window capture for the Browser Overlay in OBS

- **Window** — here you must pick **`[ASTO-BOT.exe]: ASTO-Browser`** (not ASTO Overlay).
- **Capture Method** — must be set to **Windows 10 (1903 and up)**.
- **Window Match Priority** — set it to **Window title must match**.
- **Capture Cursor** — off, so the mouse pointer does not end up in the picture.

> For **`[ASTO-BOT.exe]: ASTO-Browser`** to appear in the window list, the Browser Overlay has to be running: switch it on in the **Overlay** area with the **ON** toggle while you set up the source in OBS.

4.3.5 Photo Mode

Photo Mode is captured in exactly the same way as the window capture overlay — here too it is a window of ASTO-BOT.



The window capture for Photo Mode

> The decisive difference: as **Window** you must `ASTO Photo Mode` select. This window has to be open as well — so start a photo in the Photo Mode area while you set up the source in OBS.

Photo Mode is started with a chat command, for example:

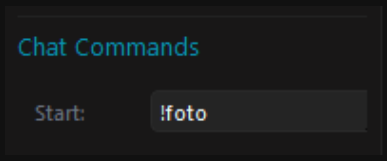


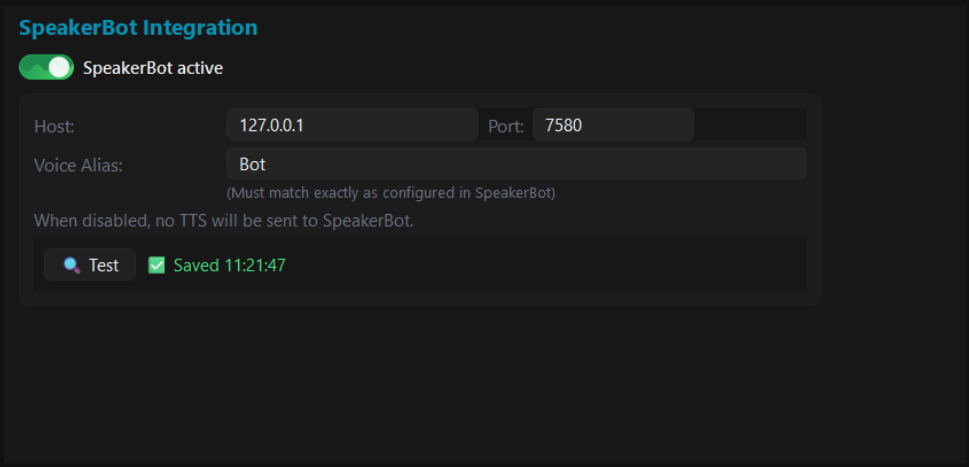
Photo Mode, started with the chat command !foto

In the settings you then configure:

- **Photo Scene** — scene and source of the window capture for the running photo session.
- **Shot Scene** — the scene that is switched to when the photo is taken.
- **Use specific Source** — if the toggle is **off**, the **scene** is switched to. If it is **on**, a single **source** is shown instead.

4.4 SpeakerBot

SpeakerBot takes care of the text-to-speech (TTS). The integration is optional — ASTO-BOT runs without it too.

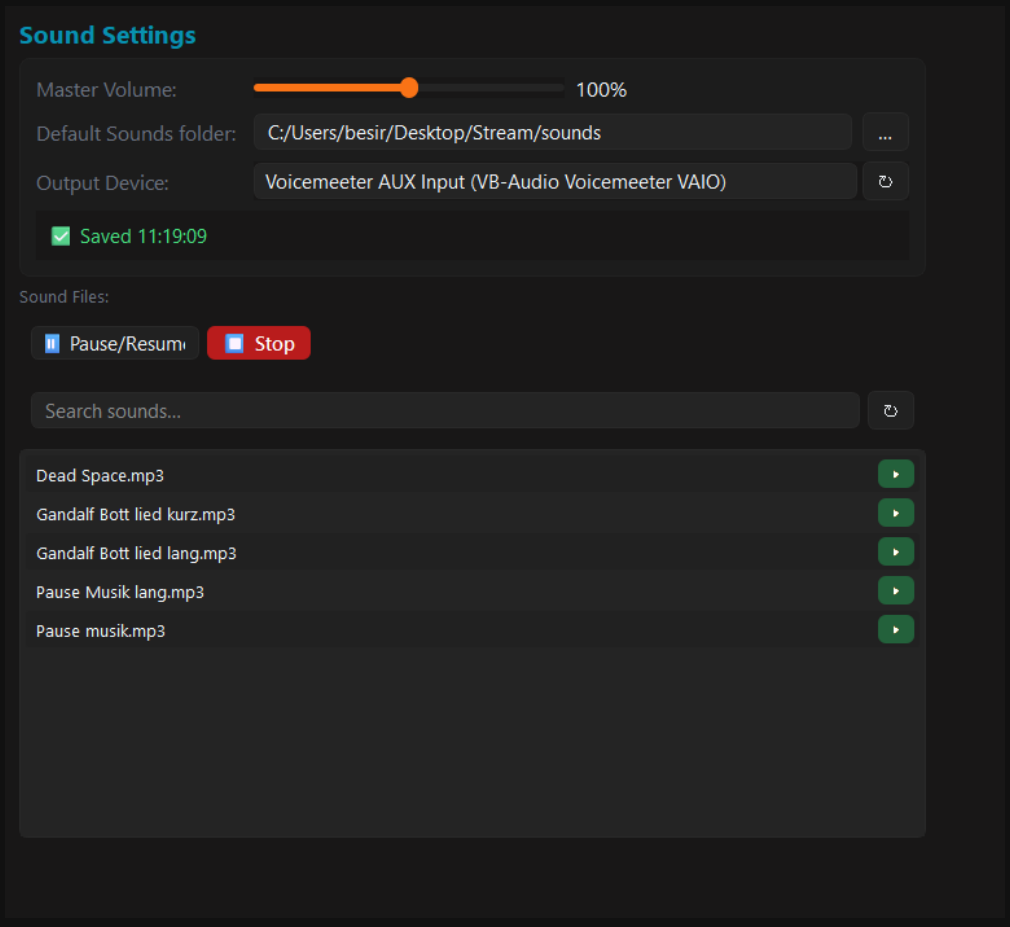


The SpeakerBot integration

- **SpeakerBot active** — switches the integration on and off. If it is off, no TTS is sent to SpeakerBot.
- **Host** and **Port** — must match your SpeakerBot installation.
- **Voice Alias** — must be named **exactly** as configured in SpeakerBot itself.
- **Test** — checks the connection.

4.5 Sounds

Here you set how ASTO-BOT plays sounds — the basis for all sound actions in events.



The sound settings with folder, output device and sound list

- **Master Volume** — the overall volume.

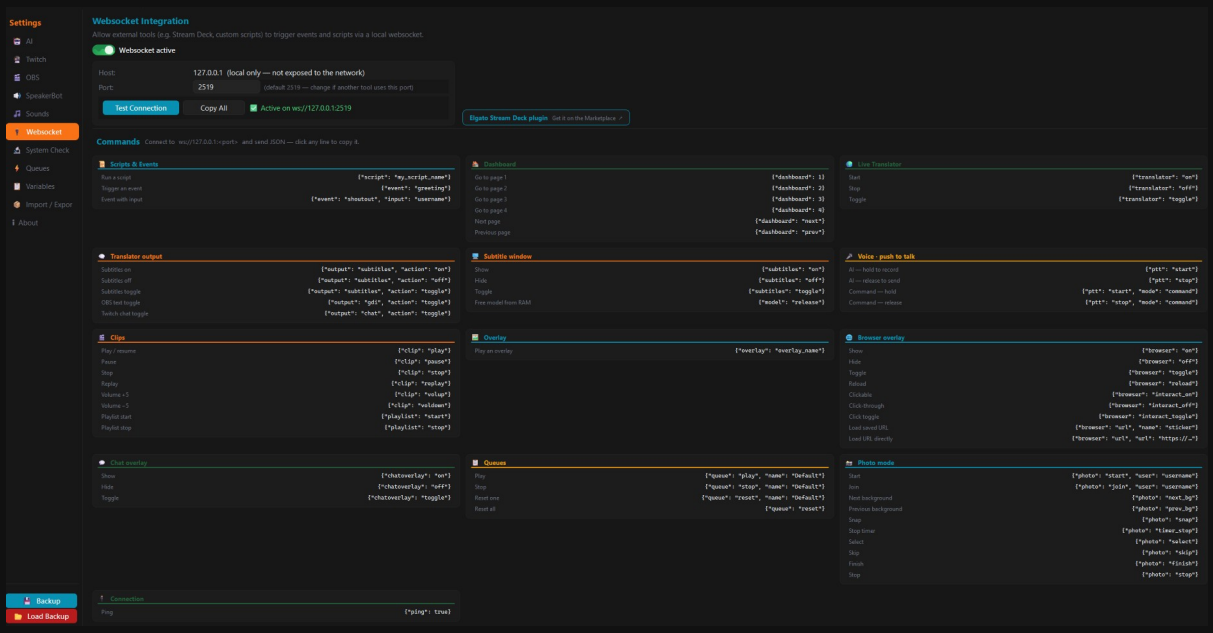
- **Default Sounds folder** — the folder your sound files live in.
- **Output Device** — which device the sounds are played through.

Below that, under **Sound Files**, you see every file in the chosen folder. The green button on the right plays a file straight away — handy for previewing. The search field helps you find the right file quickly in large collections.

> **Pause/Resume** and **Stop** act on the sound that is **currently playing** — not on all of them. If no sound is running, nothing happens.

4.6 Websocket

Through the websocket, ASTO-BOT can be controlled from the outside — by an Elgato Stream Deck or your own scripts, for example. At first glance this looks like a lot, but it is not.



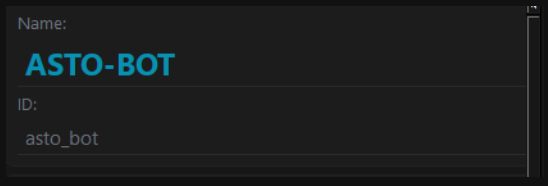
The websocket integration with the list of commands

- **Websocket active** — must be switched on if you want to use the remote control.
- **Host** — fixed at `127.0.0.1`. The websocket can only be reached **locally** and is not visible on the network.
- **Port** — the default is 2519. Only change it if another program already uses this port.
- **Test Connection** — checks whether the websocket is running.

Below that is the list of all commands. **Clicking a command copies it straight to the clipboard** — so you do not have to type anything. Afterwards you only replace the placeholder.

> Very important: for events you enter the **event ID**, **not the name**. For all other commands — such as `{"script": "..."}` — the **name** is used instead.

You find the event ID in the **Events** area, it always sits right below the event name. As a rule it is the name in lower case, with an underscore _ instead of a space:



Name and ID of an event — the websocket needs the ID

Controlling the Browser Overlay

The **Browser Overlay** (Chapter 9.8) has its own commands. With them you can, for example, switch it on and off from a Stream Deck, reload it or show a different URL:

```
# Browser Overlay via websocket
{"browser": "on"}                        # switch on
{"browser": "off"}                       # switch off
{"browser": "toggle"}                    # toggle
{"browser": "reload"}                    # reload
{"browser": "interact_on"}               # make clickable
{"browser": "interact_off"}              # click-through (default)
{"browser": "interact_toggle"}           # toggle clickability
{"browser": "url", "name": "sticker"}    # load a URL from the list by name
{"browser": "url", "url": "https://..."}   # load a URL directly
```

URL by name loads an entry from your URL list — `name` is the name you gave it there. **URL directly** shows any address you like.

Controlling the chat overlay

The **chat overlay** (chapter 9.10) has a command of its own as well. **Off** here really means off: no new messages are sent, and the lines still on screen are cleared.

```
# Chat overlay over the websocket
{"chatoverlay": "on"}                    # switch on
{"chatoverlay": "off"}                   # switch off and clear
{"chatoverlay": "toggle"}                # toggle
```

Setting up the Stream Deck

There is a **dedicated ASTO-BOT plugin** for the Elgato Stream Deck. It is the most convenient route: you drag an action onto a key and pick from a list — no address and no JSON command to type by hand.

You find it on the Elgato Marketplace:

https://marketplace.elgato.com/product/asto-bot-2db69caf-ee1e-4520-8375-ae8a6a8dbd67

Alternatively, search for *ASTO-BOT* under **More Actions** in the Stream Deck software. And inside ASTO-BOT itself, a click on **Elgato Stream Deck plugin** in the websocket area takes you straight to the Marketplace page.

The plugin comes with six actions:

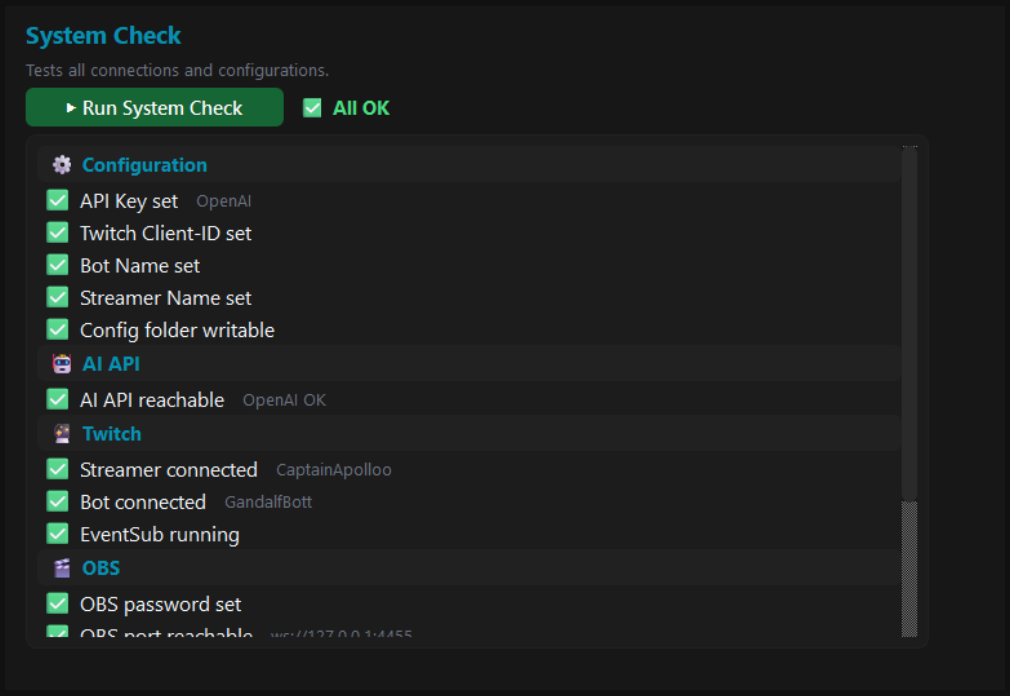
- **Command** — sends any websocket command. This covers everything in the command list further up.
- **Run Script** — starts a script. You pick the name from a list that the plugin fetches live from ASTO-BOT.
- **Trigger Event** — fires an event, again from a drop-down list. No need to worry about name versus ID here.
- **Translator** — starts and stops the Live Translator (chapter 3.9).
- **Push to Talk** — records for as long as the key is held and sends on release (chapter 3.10).
- **Reconnect** — rebuilds the connection to ASTO-BOT if it ever drops.

All keys share **one** connection to ASTO-BOT. If it drops — because you restarted ASTO-BOT, for instance — the plugin reconnects on its own.

> For the plugin to work, **Websocket active** must be switched on and the **port** has to match. If you set a port other than 2519 in ASTO-BOT, enter it in the key settings as well.

4.7 System Check

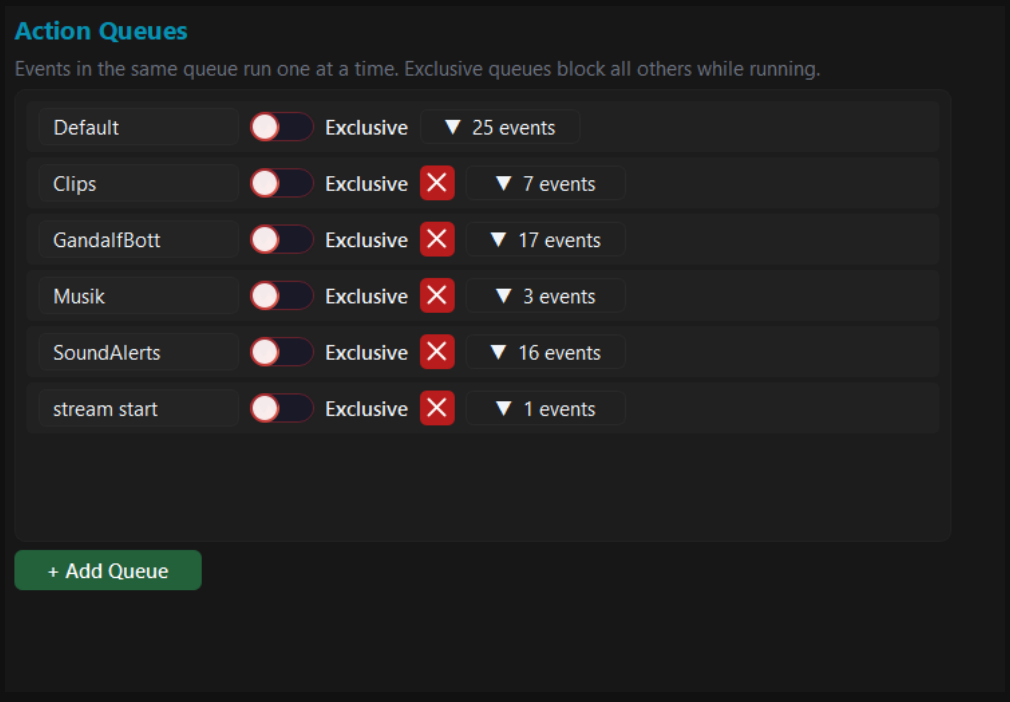
At the push of a button the System Check goes through all connections and settings and shows you what is up and what is not. When something does not work, this is the first place you look.



The System Check tests configuration, AI, Twitch and OBS

4.8 Queues

This is where you create the queues you later assign events to. Events in the same queue are processed **one after another** — never at the same time.



The action queues with the Exclusive toggle and the event counter

- **+ Add Queue** — creates a new queue. There is no limit.
- **Exclusive** — an exclusive queue **blocks all other queues**while it is running. Only once it is finished does everything else continue.
- The **arrow** with the number (e.g. `▼ 7 events`) shows how many events are assigned to this queue. If you click an event in the list, ASTO-BOT jumps straight to that event.

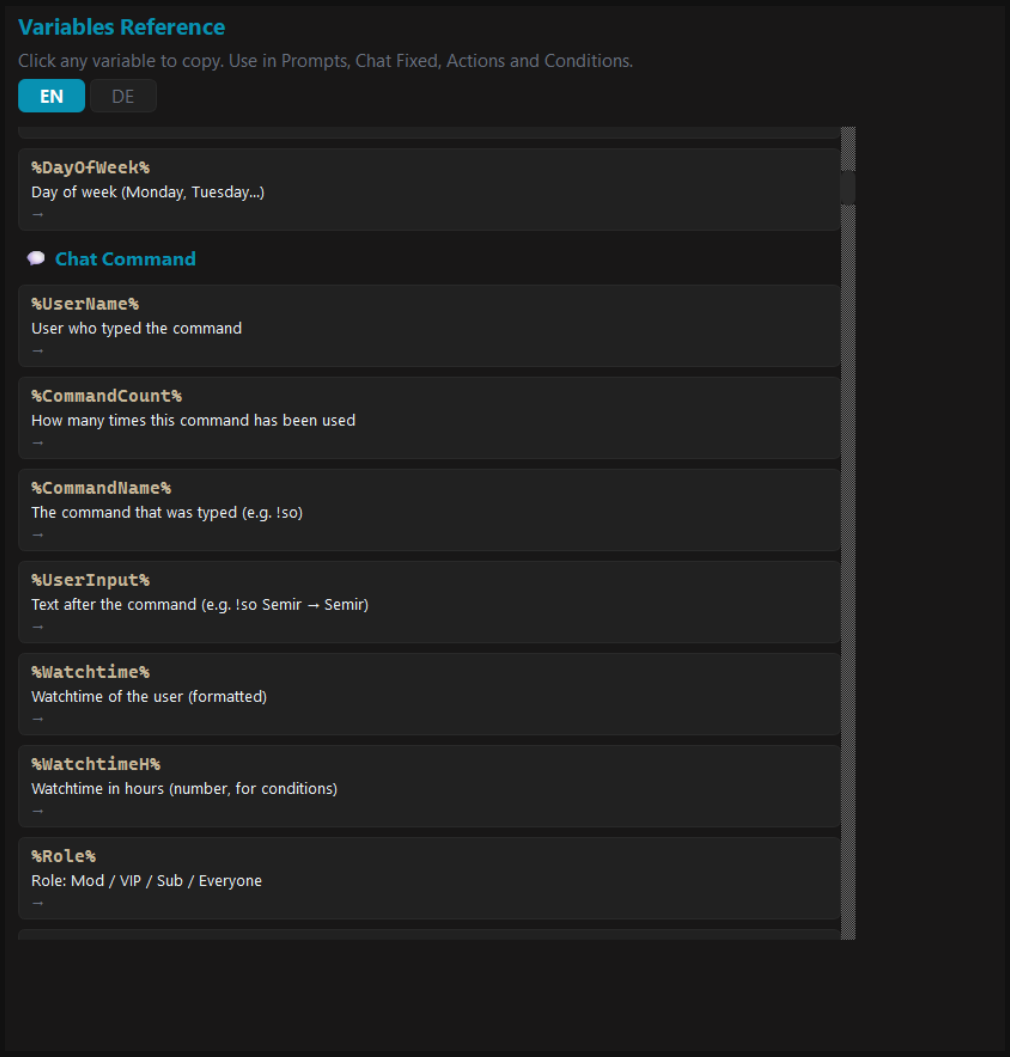
- The red **X** deletes a queue.

> The **Default queue** cannot and must not be **deleted** — which is why it is the only one without a red X.

> 💡 Queues of your own are worth it to keep things apart: sounds do not get in the way of clips, music not in the way of alerts. Use **Exclusive** for something that needs the whole stage — a clip, for example, during which nothing else should butt in.

4.9 Variables

A reference list of all variables you can use in prompts, chat messages, actions and conditions. **Clicking a variable copies it** to the clipboard.



The variable reference, sorted by area

When a variable has no value

Not every variable is filled in every situation. Fire a channel-point event by hand — from the Stream Deck or the websocket — and there was no redemption at all, so there is no cost either. **Variables like that are removed** before the text reaches chat, an overlay or a GDI text source. Your viewers never read a raw percent token.

If something should stand there instead, write it into the variable with a colon:

```
%RewardCost:0%       →  the cost, otherwise 0
%UserName:someone%   →  the name, otherwise "someone"
%UserInput:–%        →  the input, otherwise a dash
%RewardCost%         →  without a fallback: silently dropped
```

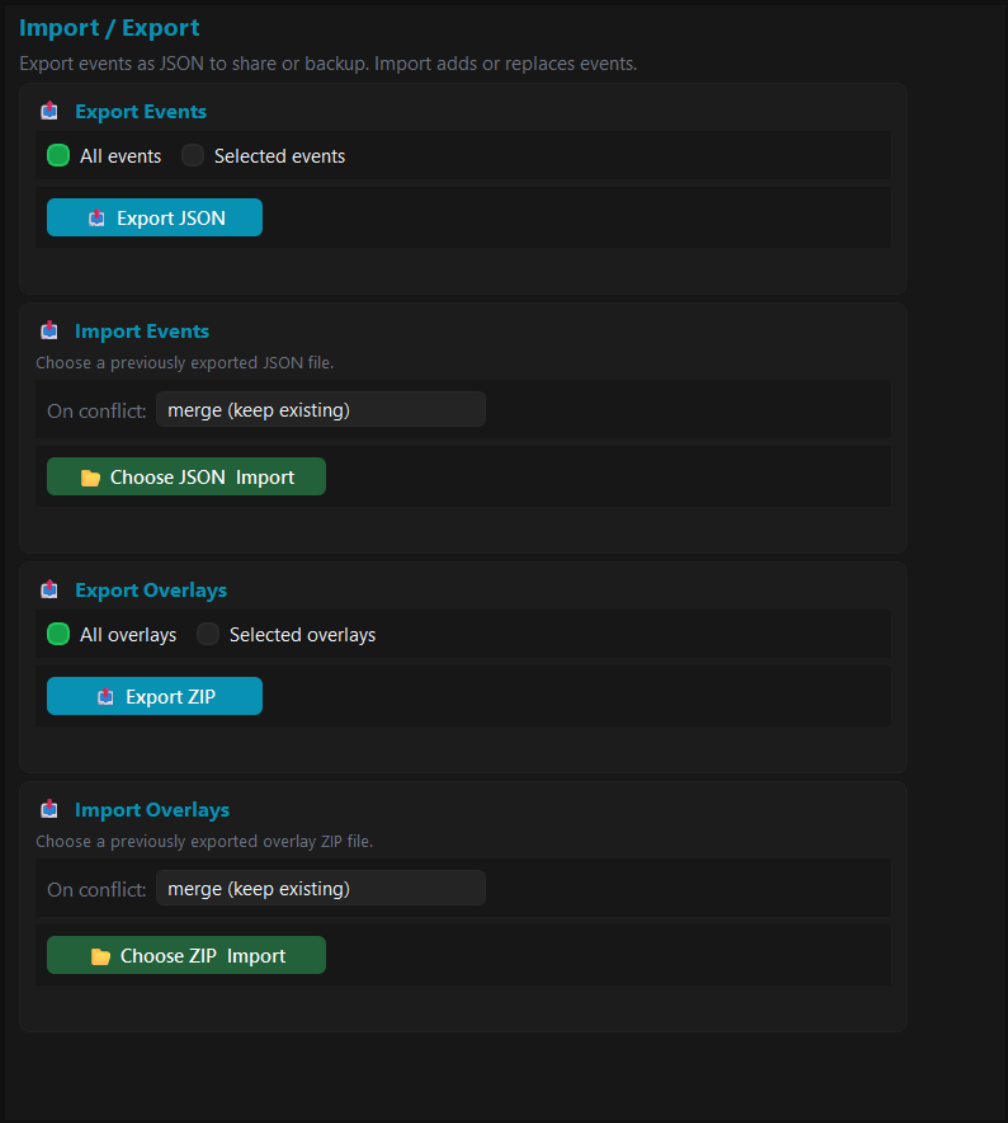
- The fallback starts after the **first** colon, so it may contain colons itself — **%StartTime:20:15%** falls back to *20:15*.
- An **empty** value counts as missing. The fallback also applies when the variable exists but holds nothing.
- A real value always wins. The fallback only shows up when there is genuinely nothing there.

> Misspell the name and the variable stays **visible**. ASTO-BOT does not know **%RewardCoast%** — and visible is a better hint here than silently gone. You will find a line about it in the log.

> 💡 This works everywhere variables appear: in chat messages, in AI prompts, in overlay text elements, on GDI text sources and in ASTOSCRIPT.

4.10 Import / Export

Here you pass on events and overlays or fetch them — handy for sharing things with other streamers or for having a backup of individual parts.



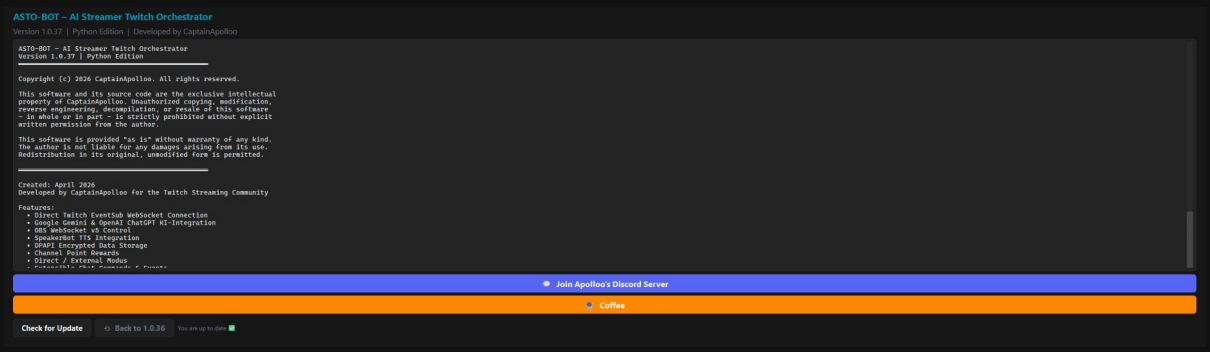
Import and export of events and overlays

- **Export Events** — as a JSON file, either **all** events or only **selected** ones.
- **Export Overlays** — as a ZIP file, again all or selected ones.
- **Import** — pick the matching JSON or ZIP file and read it in.

> Under **On conflict** you decide what happens if an entry already exists: **merge (keep existing)** keeps what is there, **overwrite (replace existing)** overwrites it.

4.11 About

Shows the licence and the program information. You also find here:



The About area with licence, Discord, coffee donation and update check

- **Join Apolloo's Discord Server** — support and news.
- **Coffee** — if you would like to support the development.
- **Check for Update** — looks for a new version.
- **Back to …** — restores the previous version. The button names the target version directly and only appears when a backup exists (see below).

If an update is available, ASTO-BOT also shows it at the top of the header bar, to the left of the Save button — so you do not have to check yourself.

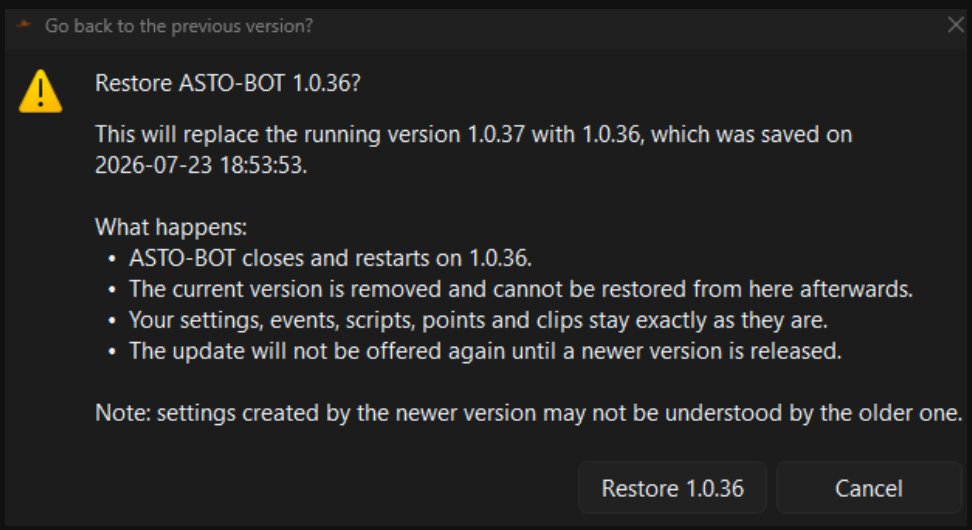
> While the update is running, **nothing may be interrupted**. Wait until ASTO-BOT restarts by itself.

> 💡 If you have moved the **runtime** folder and the EXE somewhere else, ASTO-BOT will ask on the next update whether this new place should be kept as the storage directory.

Back to the previous version

Before every update through **Check for Update**, ASTO-BOT puts the running program aside. If it turns out afterwards that something is wrong with the new version, you do not have to download anything by hand — the way back is built in.

The **Back to …** button sits right next to *Check for Update* and names the version it goes back to. It is only visible when a backup really exists — after a fresh installation and before the very first update there is none yet.



The confirmation names both versions and lists what will happen

A click opens this confirmation first. Once you confirm, ASTO-BOT closes, swaps the program files and restarts on the older version. That only takes a few seconds.

- Your **settings, events, scripts, points and clips** stay unchanged — only the program files are swapped.
- The newer version is **removed** in the process. A second step back is therefore not possible.
- This update is **not offered again** until a higher version number is released.

> The backup is only created for updates **through the button inside ASTO-BOT**. Anybody who downloads the installer from GitHub by hand and installs over the top bypasses this step — after that there is no way back, or still the one from the last update through the button. The rollback then goes back further accordingly; which version it will be is always written on the button and in the confirmation.
>
> Settings created by a **newer** version may not be known to the older one. As a rule it simply ignores them. If you want to be completely safe, put a copy of your configuration aside beforehand with **Backup** (section 4.12).

> 💡 The backup is stored as a ZIP inside the program folder under **Backups\rollback** and takes up space there permanently. Every update replaces it with the current one — so there is always exactly one.

4.12 Backup & Load Backup

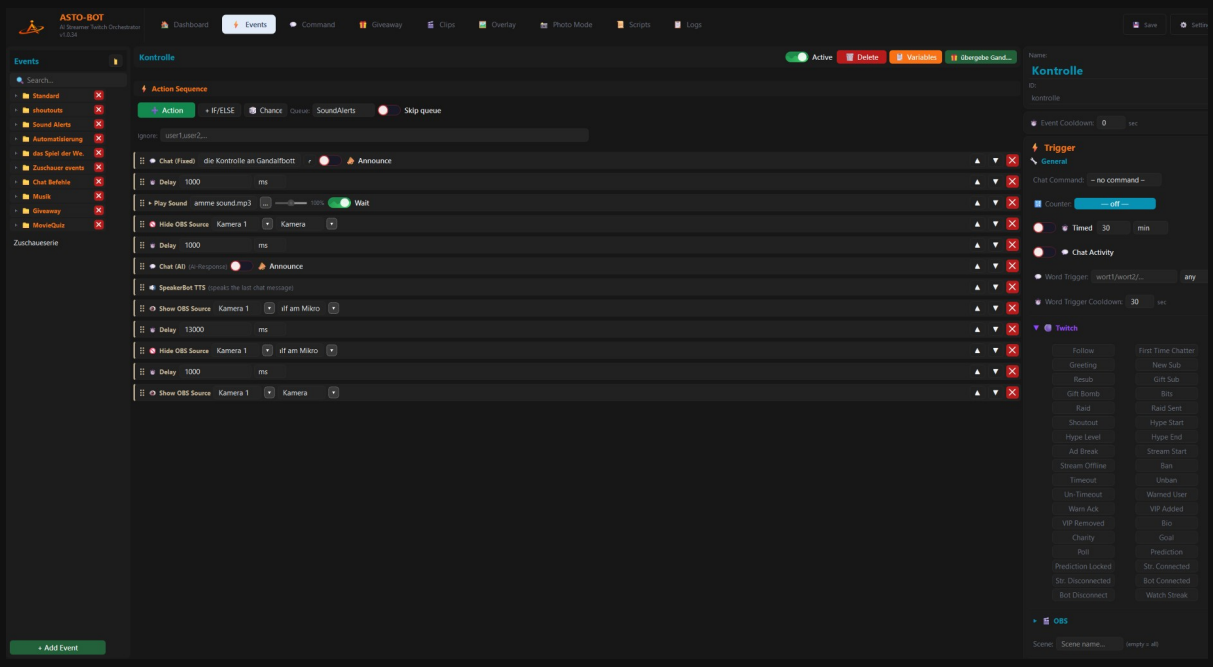
At the very bottom of the list of areas sit two buttons:

- **Backup** — immediately creates a backup of all configuration files.
- **Load Backup** — restores an earlier state.

> 💡 Make a backup before you make bigger changes. It takes one click and, in case of doubt, saves you an evening of work.

5. Events

Events are the heart of ASTO-BOT. Almost everything the bot does during your stream is controlled from here. An event always consists of two parts: a **trigger** — the thing that decides **when** something happens — and an **Action Sequence** — the list of actions that are then worked through one after another. A follow comes in, ASTO-BOT plays a sound, has the AI write a thank-you, sends it to chat and reads it out loud: that is an event.

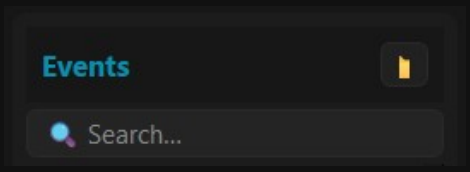


The Events page: the event list on the left, the Action Sequence in the middle, the triggers on the right

> 💡 Take your time with this chapter. Once you have understood how an event is built, you have understood ASTO-BOT.

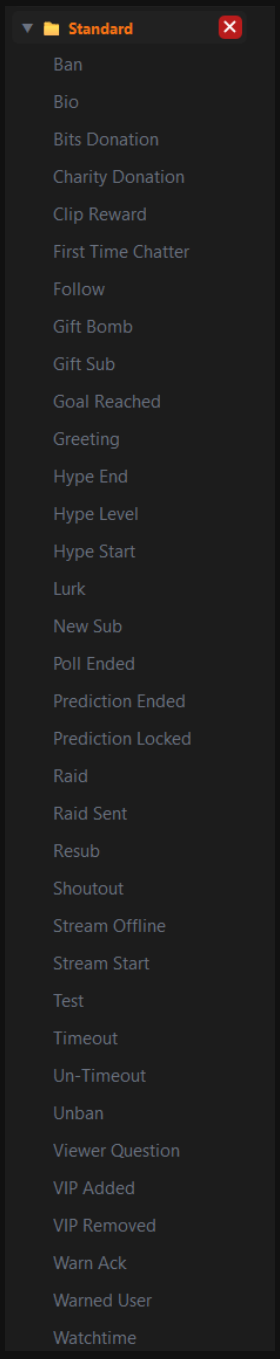
5.1 How the Events page is built

On the left sits the **event list**. At the very top you find the **folder button**, which creates a new group, and below it a **search field**. At the very bottom sits **+ Add Event**.



The folder button and the search field above the event list

ASTO-BOT ships with a **Standard group** of ready-made example events — Follow, Resub, Raid, Bits and so on. These are **suggestions**, not obligations. You can use them, rebuild them or delete them.



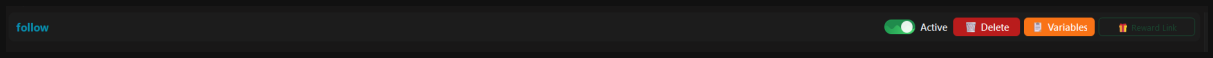
The Standard group with the example events that ship with ASTO-BOT

- With the **folder button** you create your own **groups** — *Sound Alerts*, *Shoutouts* or *Music*, for example.
- You move an event into another group with a **right-click** — or simply with **drag & drop**.
- Drag & drop also pulls an event **completely out of a group** again.
- **+ Add Event** creates a new, empty event.

> 💡 Groups are purely about order — they change nothing about the behaviour. With 50 events you will still want them.

5.2 The header of an event

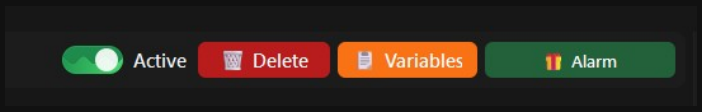
When you select an event, a narrow bar with the name and four switches appears at the top.



The header of an event — no reward is linked here

- **Active** — switches the event on and off. If it is off, nothing happens, no matter what the trigger reports.
- **Delete** — deletes the event. With a confirmation prompt.
- **Variables** — opens the variable overview (see below).
- **Reward Link** — links the event to a channel point reward.

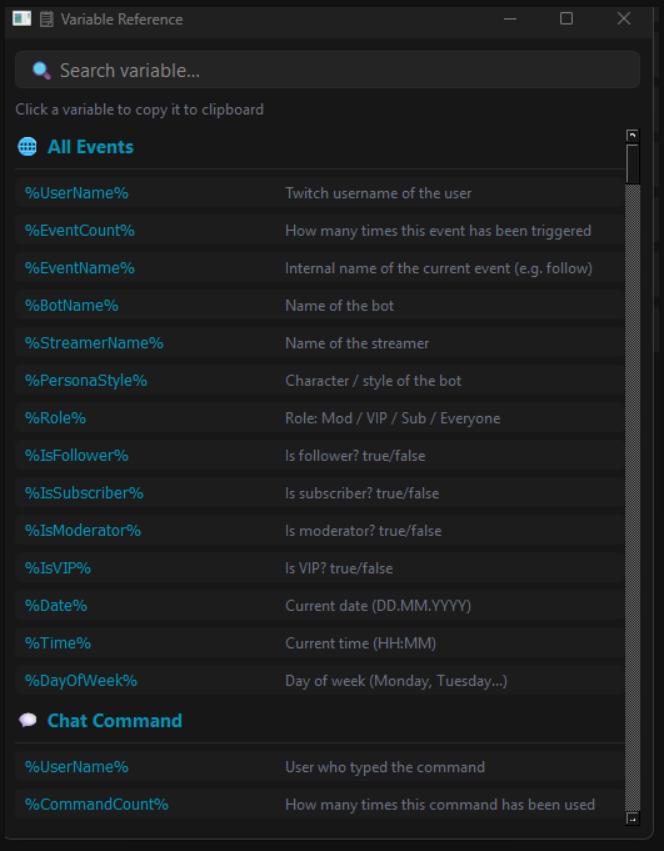
The **Reward Link** button tells you at once where you stand: if it says **Reward Link** in dark green, **no** reward is linked. If one is linked, the button shows the **name of the reward** instead.



Here the reward "Alarm" is attached to the event — the button shows the name

The Variables window

Variables opens a list of **all** the variables ASTO-BOT knows — sorted by category, with a search field. Clicking a variable copies it to the clipboard, and you can paste it straight into a field or a prompt.

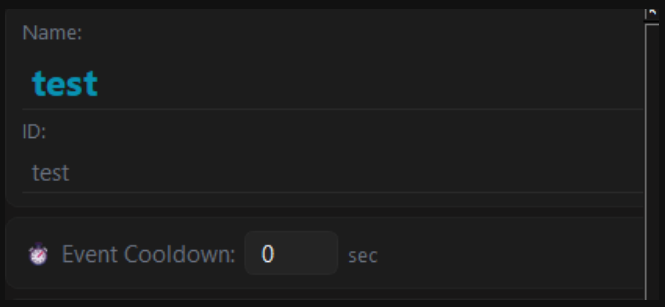


The variable overview — a click copies the variable

> 💡 Some variables appear more than once, for example `%UserName%` or `%UserInput%`. That is not a bug: they exist in several categories and mean the same thing in each of them.

Name, ID and event cooldown

In the top right of the panel sit the **name** and the **ID** of the event. As a rule the ID is the name in lower case, with spaces turned into _. You need it when you want to fire the event from outside via **ASTOSCRIPT** or **websocket** (see chapter Settings → Websocket).



Name, ID and event cooldown — top right in the trigger area

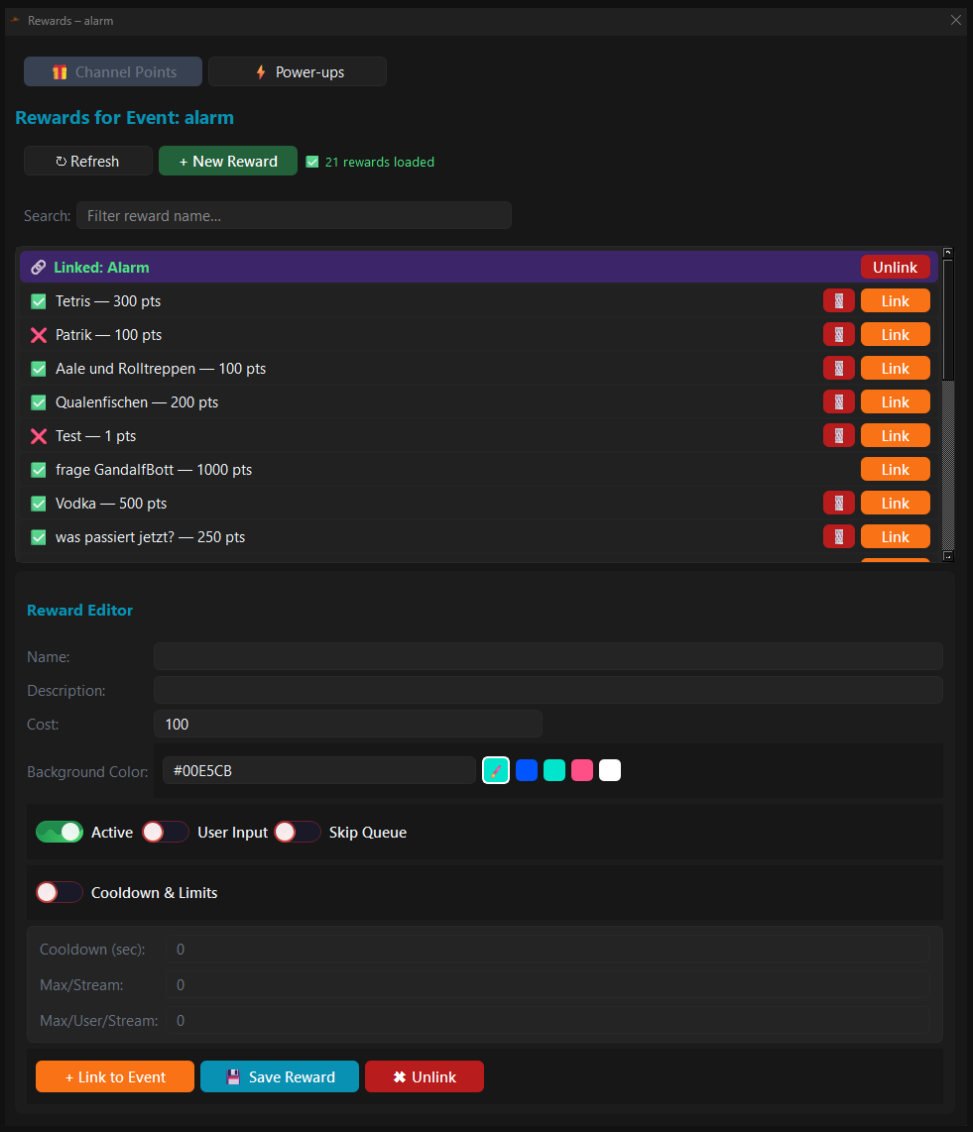
Below that sits the **Event Cooldown** in seconds. It makes sure that this **particular** event does not fire again for a while after it has been triggered. In most cases you do not need it, `0` is perfectly fine.

> Every event has its **own** cooldown. There is **no** overall cooldown for all events — if you want to slow several events down, you have to enter it for each one separately.

5.3 Rewards: channel points and power-ups

The **Reward Link** button opens a window of its own with two tabs: **Channel Points** and **Power-ups**.

Channel Points



The reward window with all the channel point rewards of the channel

- **Refresh** fetches the reward list fresh from Twitch.
- **+ New Reward** creates a new reward.
- The **search field** filters the list by name.
- The **green tick** in front of the name means: the reward is active. A **red X** means: the reward is disabled.
- **Link** (the orange button) links the reward to the event, **Unlink** removes the link again.

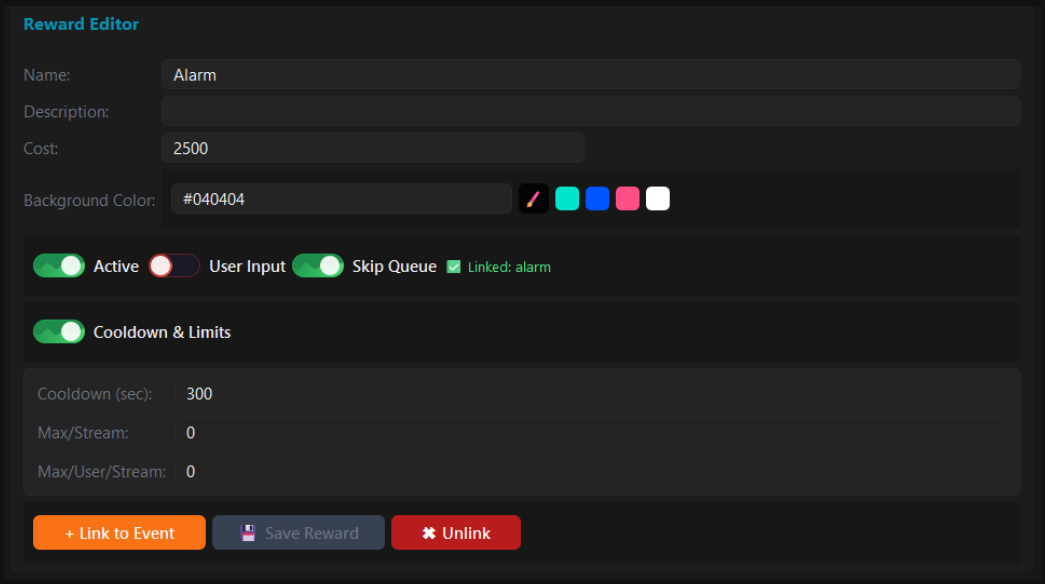
The **trash can** only appears on rewards that **ASTO-BOT created itself**. Rewards that come from Twitch or from another tool can only be **linked** — editing or deleting them is not possible. That is a restriction from Twitch, not from ASTO-BOT.

💡 **The most important rule:** an event can have **one** reward — but a reward can hang on **any number** of events. So one reward can trigger several things at once.

The Reward Editor

The lower part of the window holds the **Reward Editor**. This is where you put a reward together or change an existing one — provided ASTO-BOT created it.

If a reward is already linked to the event, it is **loaded into the editor** the moment the window opens — no need to find it in the list first. Whoever opens the Reward Editor almost always wants to adjust that one reward.



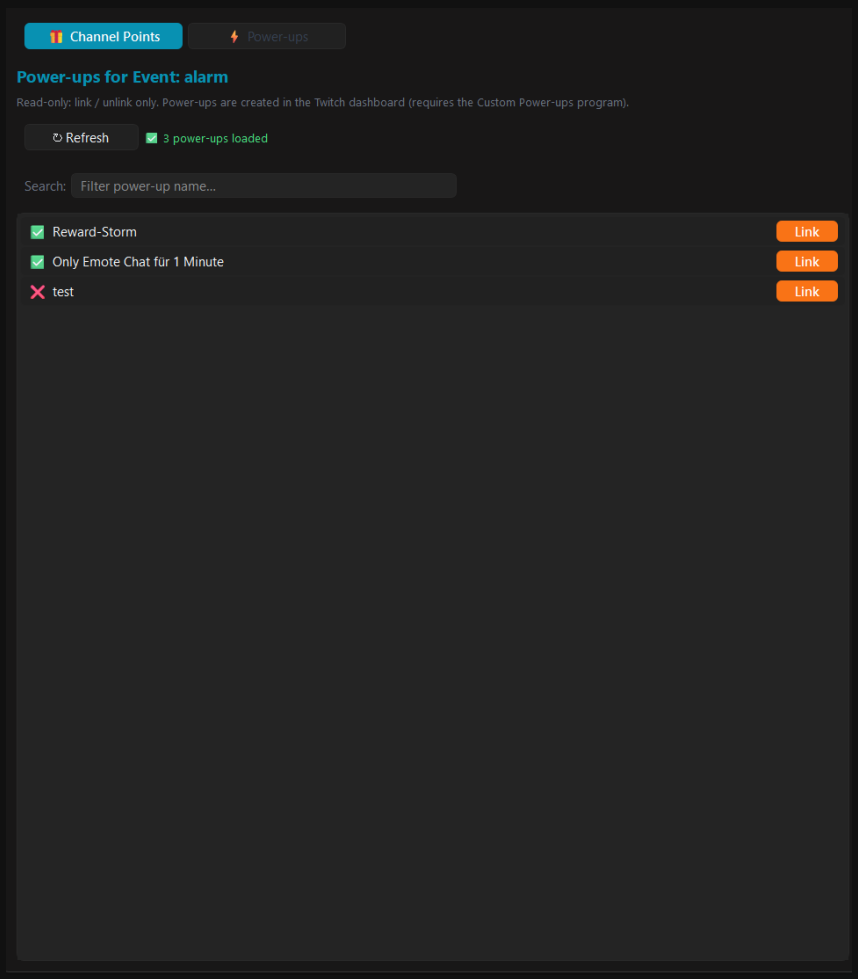
The Reward Editor with cost, colour, cooldown and limits

- **name**, **Description** and **Cost** (the channel points) — what the viewer sees on Twitch.
- **Background Color** — as a hex value or with the colour swatches next to it.
- **Active** — switches the reward on or off on Twitch.
- **User Input** — the viewer can send a text along when redeeming. It ends up in `%UserInput%`.
- **Skip Queue** — the redemption skips the reward queue in the Twitch dashboard and is confirmed immediately.
- **Cooldown & Limits** — a toggle, below it: **Cooldown (sec)**, **Max/Stream** (how often in total per stream) and **Max/User/Stream** (how often per viewer per stream). `0` means **unlimited**.
- At the bottom: **+ Link to Event**, **Save Reward** and **Unlink**.

> Press **Save Reward before** you close the window. Otherwise your changes are gone.

Power-ups

The second tab shows the **power-ups** of your channel. Here there is only **Link** and **Unlink** — nothing else.

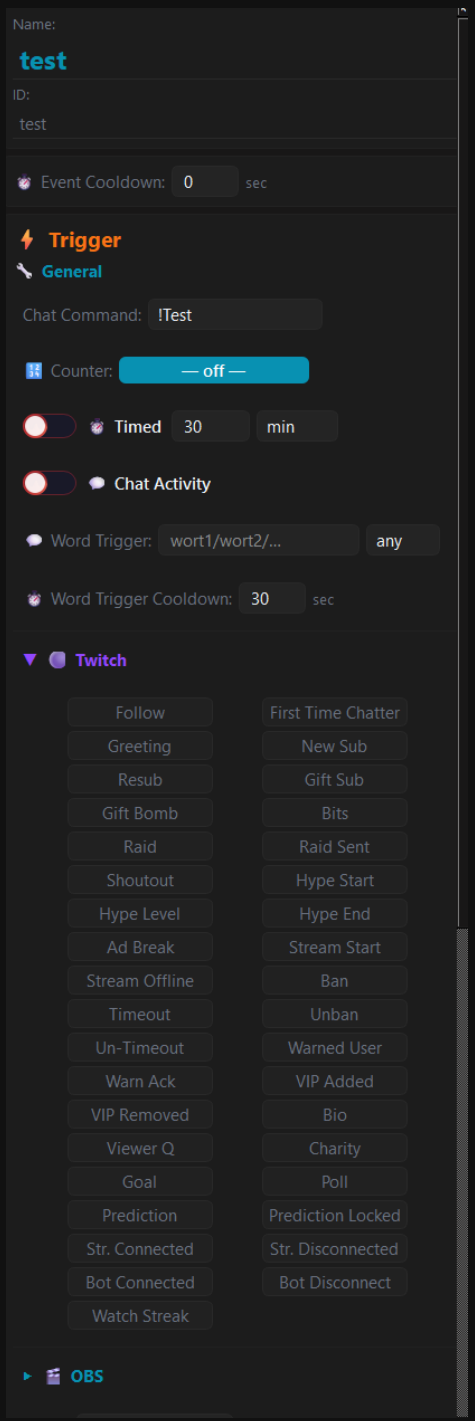


Power-ups can only be linked, not edited

> Power-ups are created in the **Twitch dashboard**, ASTO-BOT cannot create or change them — only react to them.

5.4 Triggers — when does the event fire?

The trigger area sits on the right of the panel, below name, ID and cooldown. It is split into groups: **General**, **Twitch**, **OBS**, **Giveaway** and **Music**. You can set several triggers at once — the event then fires as soon as **one** of them applies.

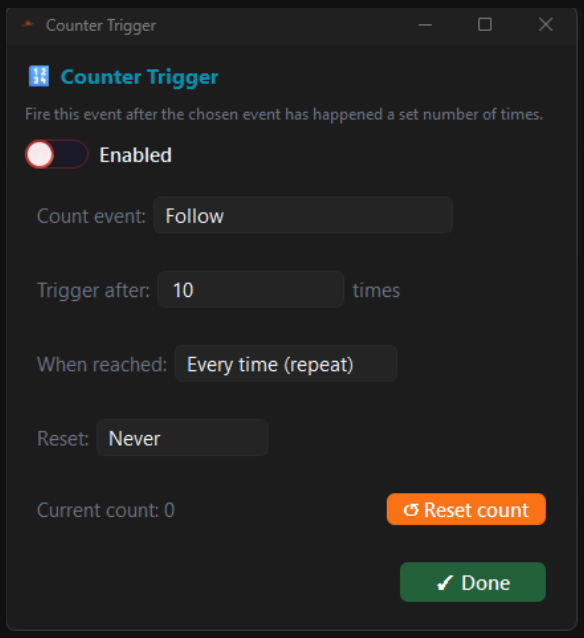


The trigger area of an event

5.4.1 General

- **Chat Command** — the event hangs on a chat command. The commands themselves are created in the **Command** area (see the Command chapter), here you only pick one.
- **Timed** — the event fires at a fixed interval. The unit can be switched: seconds, minutes, hours or days. Ideal for recurring announcements.

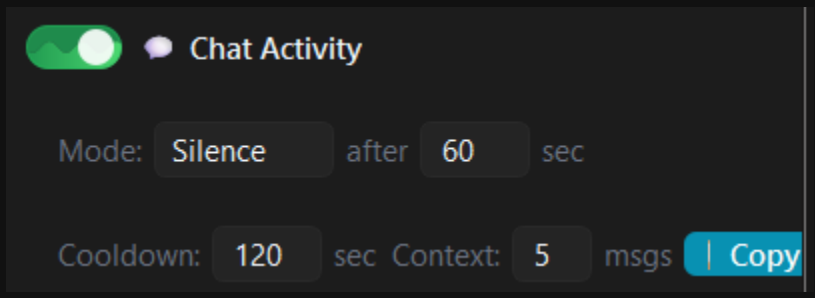
Counter — counts how often **another** event has fired and then fires itself. A click opens a window of its own:



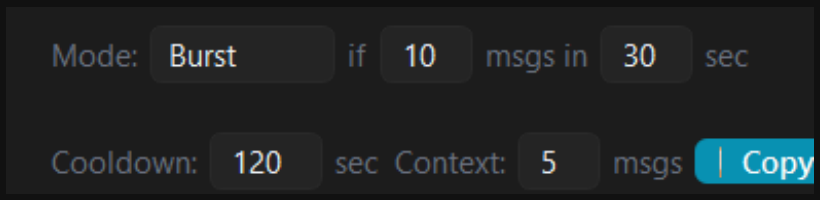
The Counter trigger — here the event fires on every 10th follow

- **Enabled** — counter on/off.
- **Count event** — which event is counted, *Follow* for example.
- **Trigger after: N times** — after how many times it goes off.
- **When reached:** `Every time (repeat)` fires again and again — `Only once` fires exactly **one single time** and never again.
- **Reset:** `Never` keeps the counter forever — `per stream` resets it after every stream.
- **Current count** shows where it stands, **Reset count** puts it back to 0 immediately.

Chat Activity — reacts to the activity in chat. There are two modes:



Silence — fires when nobody has written for 60 seconds



Burst — fires when 10 messages come in within 30 seconds

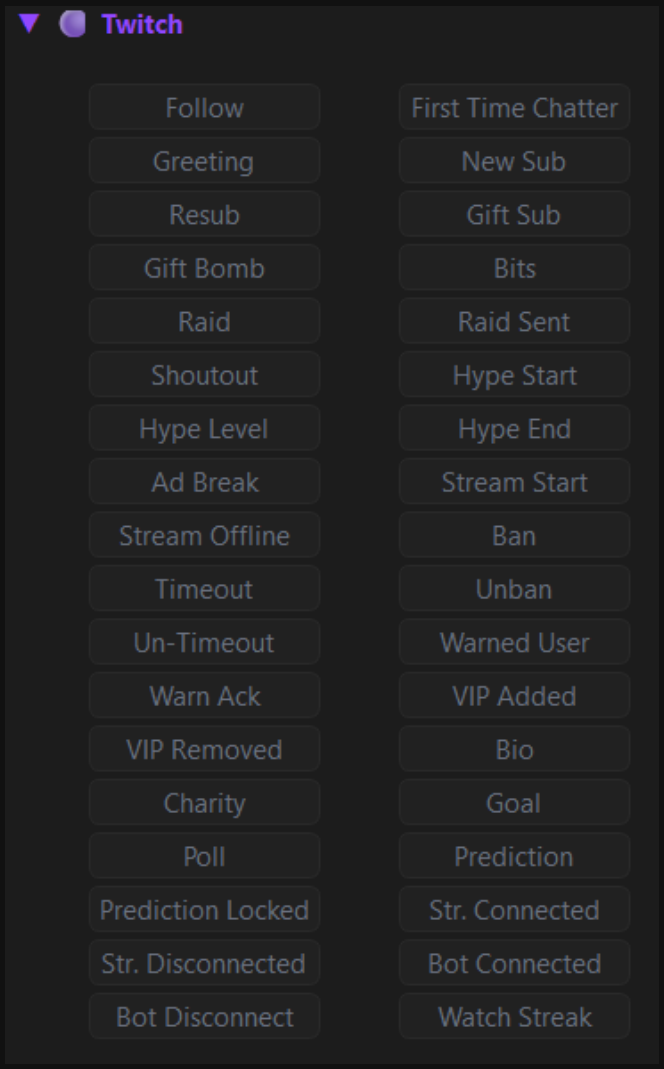
- **Silence** — fires when **nobody** has written for X seconds. Good for waking up a chat that has fallen asleep.
- **Burst** — fires when many messages come in within a short time, 10 messages in 30 seconds for example.
- **Cooldown** stops the trigger from firing over and over. **Context** decides how many of the last messages ASTO-BOT hands to the AI.

Word Trigger — reacts to certain words in chat. Several words are separated with `/`. The mode next to it decides: **`any`** — one of the words is enough. **`all`** — all of the words must appear in the message. The **Word Trigger Cooldown** below only applies to this trigger.

> If you enter just a **`*`**, **all** words are accepted — the event then fires on **every** chat message. That is powerful, but dangerous: combined with an AI reply or a sound it means constant fire in a lively chat. Use it carefully and always with a cooldown.

5.4.2 Twitch

The biggest group: **36 triggers** that listen directly to Twitch events. A click on a button activates it.



The 36 Twitch triggers

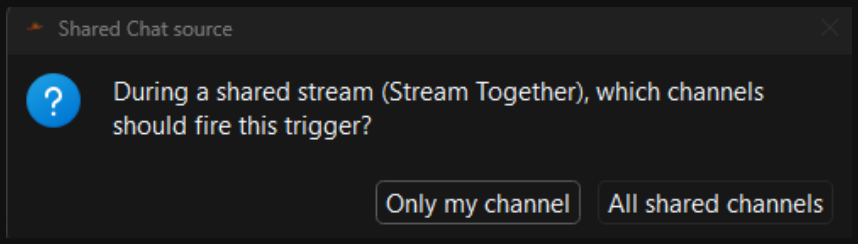
Follow · First Time Chatter · Greeting · New Sub · Resub · Gift Sub · Gift Bomb · Bits · Raid · Raid Sent · Shoutout · Hype Start · Hype Level · Hype End · Ad Break · Stream Start · Stream Offline · Ban · Timeout · Unban · Un-Timeout · Warned User · Warn Ack · VIP Added · VIP Removed · Bio · Charity · Goal · Poll · Prediction · Prediction Locked · Str. Connected · Str. Disconnected · Bot Connected · Bot Disconnect · Watch Streak

Most of them explain themselves. These are worth a second look:

- **Shoutout** — somebody gives **you** a shoutout.
- **Raid** — you are being raided. **Raid Sent** — you raid somebody.
- **Bio** — fetches information from the Twitch biography and the games the target has played. The AI can write a summary from it.

- **Goal** — a Twitch goal has been reached.
- **Watch Streak** — the watch streak a viewer has reached.
- **Warn Ack** — a warning has been acknowledged by the viewer.
- **Str. Connected / Str. Disconnected** and **Bot Connected / Bot Disconnect** — the connection of the streamer account and the bot account.

Shared Chat (Stream Together): for **Greeting**, **First Time Chatter** and the three hype train triggers (**Hype Start**, **Hype Level**, **Hype End**) ASTO-BOT asks, when you click them, how it should behave in a shared chat:

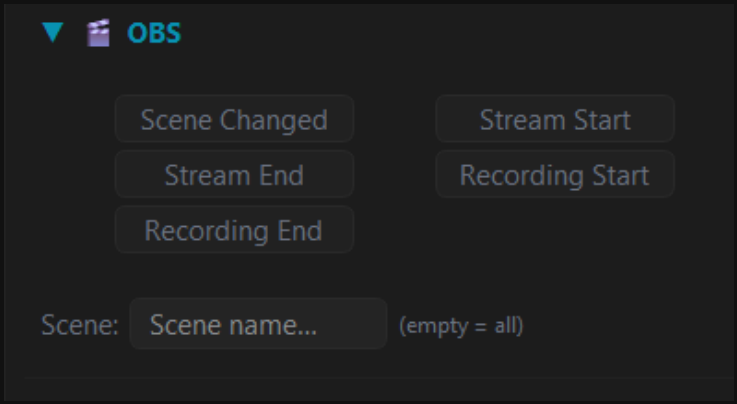


In a shared chat ASTO-BOT asks which channels should count

- **Only my channel** — only viewers from your own channel fire the event.
- **All shared channels** — the viewers from the other channels of the shared chat count as well.

5.4.3 OBS

Five triggers that listen to OBS: **Scene Changed**, **Stream Start**, **Stream End**, **Recording Start** and **Recording End**.



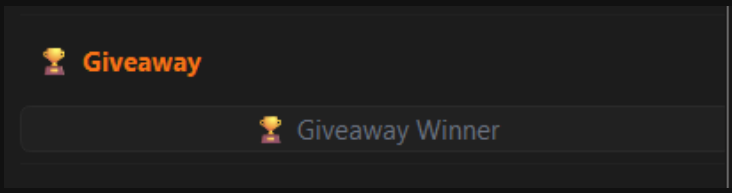
The OBS triggers with the Scene field

In the **Scene:** field below you enter which scene the trigger should react to. If you leave it **empty**, it applies to **every** scene.

The **Scene Changed** trigger has an **IN/OUT** switch next to it. **IN** — the default — fires when you switch **into** the scene; **OUT** fires when you **leave** it again. That lets you start something on entering a scene and end it on leaving, without building a second event the long way round.

5.4.4 Giveaway

Giveaway Winner — fires as soon as a giveaway has been finished and a winner has been drawn. The details are in the Giveaway chapter.



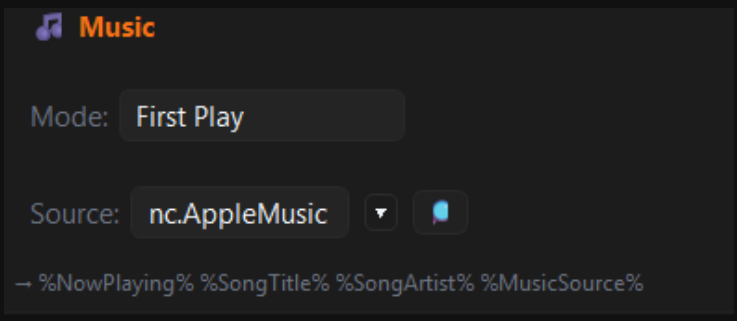
The Giveaway trigger

This trigger is fired by the **Draw** button on the Giveaway page, by the action **Draw Giveaway** (chapter 5.6) and by the ASTOSCRIPT command `GIVEAWAY DRAW … ANNOUNCE`. Which giveaway is meant is set on the trigger itself.

> ⚠ Never put the action **Draw Giveaway** with the **same** giveaway and **Announce** switched on into the **same** event — it would trigger itself endlessly. See chapter 5.6, group *Giveaway*.

5.4.5 Music

The Music trigger reads what music is currently playing on your PC — from Spotify, Apple Music or even from the browser.



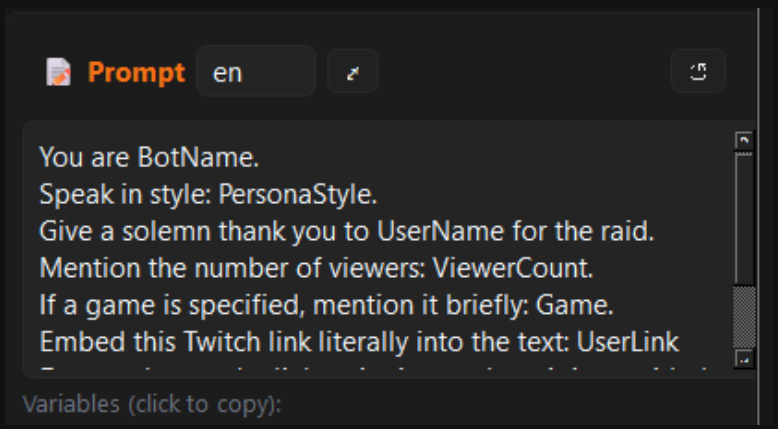
The Music trigger with mode and source

- **Mode** — `First Play` fires on the first playback, `On Song Change` on every change of song.
- **Source** — the program that plays the music. With the **magnifier** ASTO-BOT scans your PC for programs that are currently playing music.
- It delivers: `%NowPlaying%`, `%SongTitle%`, `%SongArtist%`, `%MusicSource%`.

> The program must actually be **playing sound**, otherwise Windows reports nothing and ASTO-BOT does not find it. This is by far the most common source of trouble: if the player is merely open but paused, you get no data.

5.4.6 AI Prompt

At the very bottom of the trigger area sits the **Prompt** field. That is **not a trigger**, it is the instruction to the AI: what should it say when this event fires?

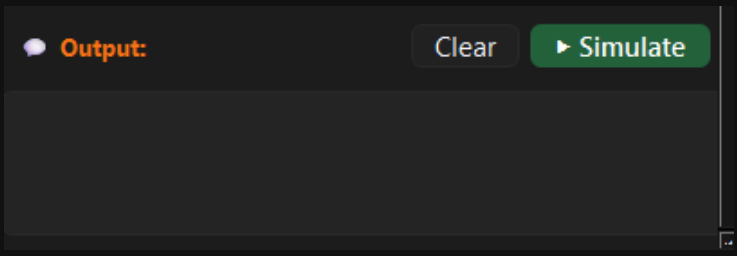


The AI prompt of an event, in English here

- The standard events come with **German and English** prompts. The small button (en) switches the language.
- The **small arrow** opens a larger text field in which longer prompts are more comfortable to write.
- **Variables are allowed in the prompt as well** — `%UserName%`, `%CheerAmount%`, `%SongTitle%` and all the others.

5.4.7 Simulate

At the very bottom sits the **Output** field with **Clear** and **Simulate**. **Simulate** fires the event as a test, without a viewer having to do anything — the AI's answer appears right in the output field.

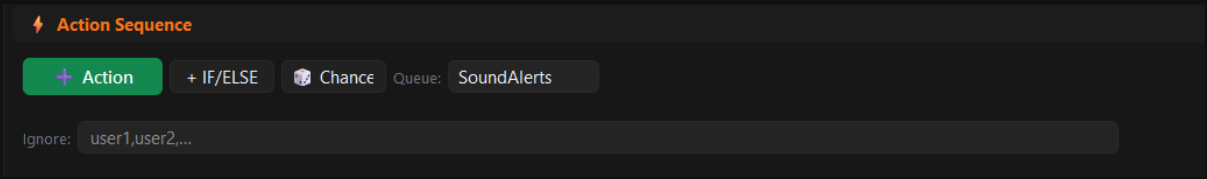


With Simulate you test an event without waiting for a viewer

> 💡 Build an event, press **Simulate**, look at the result, change the prompt, press Simulate again. That way you get a tone that fits your channel within five minutes.

5.5 The Action Sequence

The Action Sequence is the list of actions that are worked through when the event fires — from top to bottom.



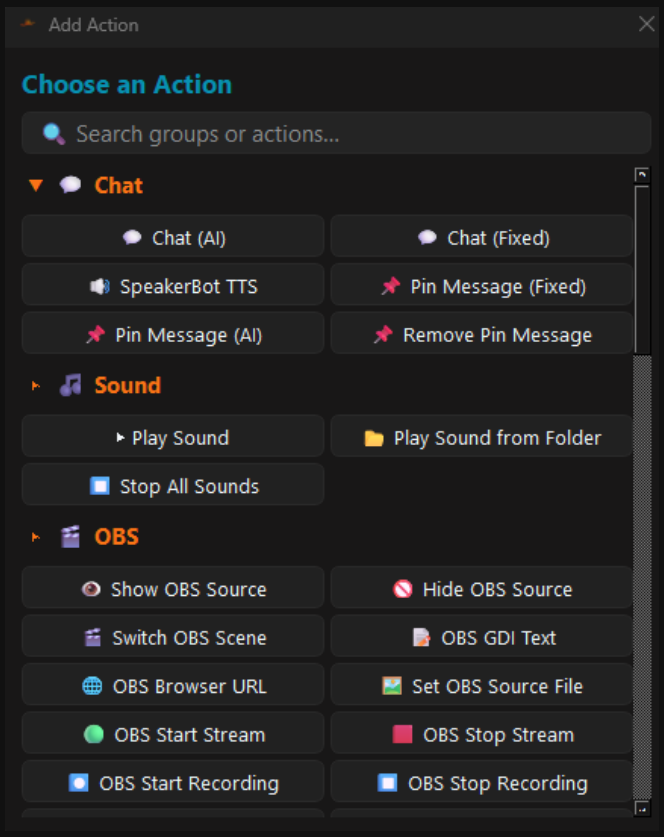
The header of the Action Sequence

- **+ Action** — adds an action.
- **+ IF/ELSE** — adds a condition (section 5.7).
- **Chance** — adds a random block (section 5.8).
- **Queue:** — which queue the event runs in. The queues are created in the settings (chapter Settings → Queues).
- **Skip queue** — the switch next to it lets the event skip the queue **entirely**: it starts right away and never occupies a slot. While it is on, the queue dropdown is greyed out — the assignment is kept, though, and applies again as soon as you switch it off.
- **Ignore:** — names, comma separated. These viewers do not fire the event. Applies **only to this one event**.

Skip queue is meant for **long-running** events. An event that shows an image for three minutes otherwise holds back every other event in its queue for those three minutes. With the switch you no longer need a separate queue just for that.

> **Skip queue** only lifts the ordering — **cooldown** and **ignore** still apply unchanged. Keep in mind, though: if the event fires again while it is still running, both runs happen **at the same time**. With events that involve waits, scene changes or sounds this can get tangled — which is exactly what the queue otherwise protects you from.

+ Action opens the **Choose an Action** window with all available actions, sorted into groups. There is a search field at the top, and the groups can be folded open and closed.



The "Choose an Action" window with all the action groups

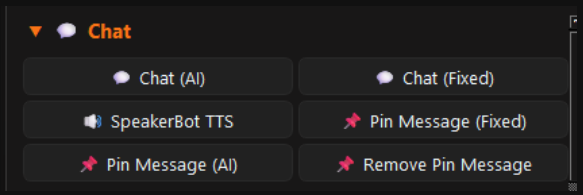
You sort inserted actions with the **handle** (⎇) by drag & drop, the arrows ▲▼ move them one position, and the red ✕ deletes them again.

> 💡 **All** actions may use variables — the prompts as well. Wherever there is a text field you can use `%UserName%`, `%UserInput%` and the rest.

5.6 The action groups

ASTO-BOT comes with ten action groups: **Chat**, **Sound**, **OBS**, **Clips**, **System**, **Twitch**, **Moderation**, **Chat Mode**, **Points** and **Now Playing**.

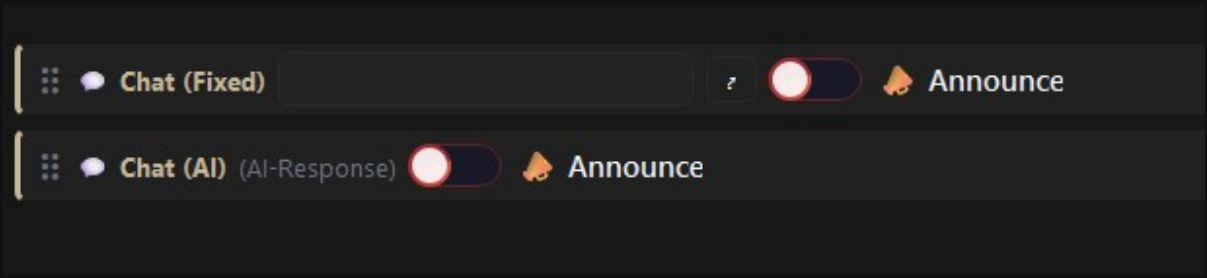
Chat



The Chat group

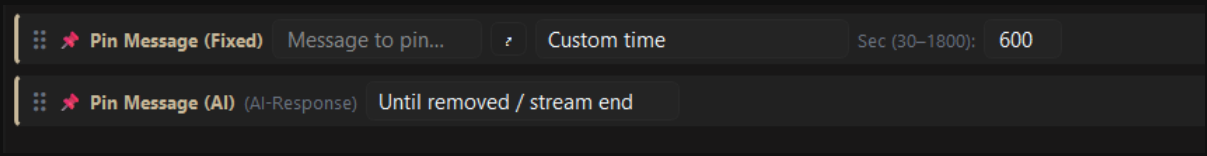
- **Chat (AI)** — sends a message generated by the AI into chat. There is **no** text field of its own: the text comes entirely from the **prompt** of the event.
- **Chat (Fixed)** — a fixed message that you write yourself. The small arrow opens a larger text field.
- **SpeakerBot TTS** — sends text to SpeakerBot so it gets read out loud.
- **Pin Message (Fixed)** and **Pin Message (AI)** — pin the message on Twitch.
- **Remove Pin Message** — removes the pinned message again.

> If the **prompt** of the event is empty, **Chat (AI)** cannot generate anything — instead of the message, a note from the AI arrives in chat. So the prompt wants to be filled in.



Chat (Fixed) and Chat (AI) with the Announce toggle

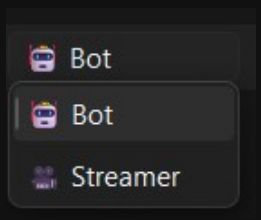
Both chat actions have an **Announce** toggle: the message then arrives on Twitch as an **announcement** — with a coloured background and far more noticeable than an ordinary chat line. When Announce is on, the **colour swatches** for the announcement colour appear next to it.



Pin Message (Fixed) with a fixed duration, Pin Message (AI) until it is removed

For the pin actions you choose the duration: **Custom time** in seconds (at least 30, at most 1800) or **Until removed / stream end** — then the message stays up until you remove it or the stream ends.

Who is speaking: bot or streamer? All four actions — **Chat (Fixed)**, **Chat (AI)**, **Pin Message (Fixed)** and **Pin Message (AI)** — have their own **sender selector**. It decides, per action, which name the message appears under in chat. You can mix freely within one event: the announcement from the bot, the personal reaction from the streamer account.



The sender selector on a chat action

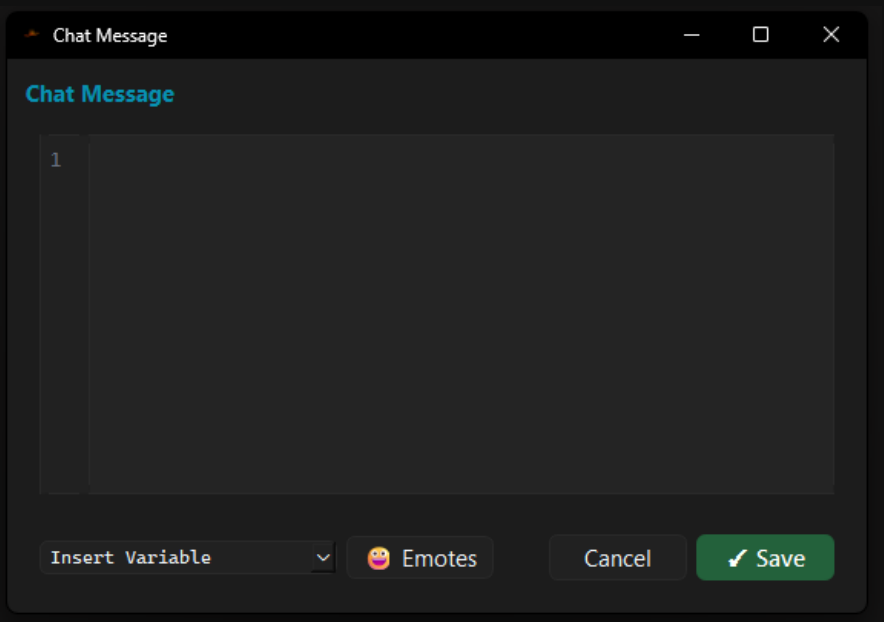
> If you have connected **only** your streamer account, the selector is **greyed out** and everything is sent from the streamer account. You connect a bot account in **Settings** under *Twitch*.

The choice is binding: set to **Streamer**, it will not quietly fall back to the bot even if sending fails. Actions from older configurations whose selector you never touched behave exactly as before.

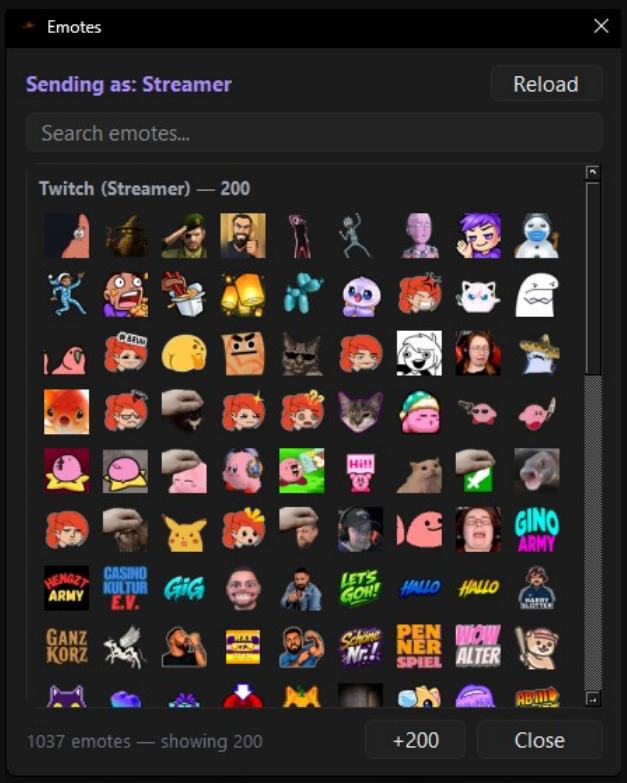
> ⚠ **Announcements need moderator rights.** If **Announce** is on and the sender is the bot, the bot has to be a **moderator** in your channel. If it is not, the announcement will not arrive — in that case simply switch the action to **Streamer**.

Inserting emotes

The text windows of **Chat (Fixed)** and **Pin Message (Fixed)** have an **Emotes** button next to *Insert Variable*. It opens a picker with every emote the selected sender account is allowed to use.



The Emotes button next to Insert Variable



The emote picker with a search field and the emotes of the selected account

- **Sending as** in the top left shows which account the selection applies to. Switch the sender selector on the action and the offered list changes with it.
- The emotes of **your own channel** come first, then sub and follower emotes of other channels, then the global Twitch emotes. **BTTV / FFZ / 7TV** sit at the very bottom.
- The **search field** always filters the complete list, not just what is currently drawn.
- For speed, 200 emotes are drawn at a time. **+200** fetches the next batch.
- **Reload** loads the emotes again — handy right after you have subscribed somewhere or created a new channel emote.
- The window stays **open** after a click so you can insert several emotes in a row.

> **Why the sender selector matters here:** Twitch checks on sending whether the sending account owns the emote at all. If the bot sends a sub emote of your channel without being subscribed to you, only the **bare name** appears in chat instead of the image. That is why the picker only ever offers what the selected account can really use.
>
> **BTTV, FFZ and 7TV** are not Twitch emotes but replacements in the viewer's browser. Every account may write them — but only viewers with the matching extension installed will see them. This list fills up once ASTO-BOT is connected.

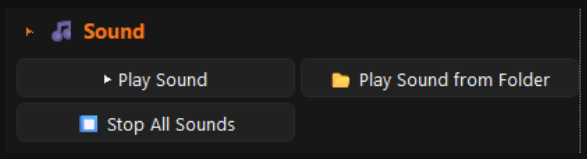
The **Prompt Editor** has the Emotes button as well. There the names are not scattered through the text but appended to a line reading *Only use these emotes:*. As a closed list, the AI sticks to it far more often instead of inventing emotes of its own. If several AI actions of the same event consume the prompt and they are set to **different** senders, the picker shows only the **intersection** — that is, what works for every account involved.

> For the emote list ASTO-BOT needs the Twitch permission `user:read:emotes`. After updating to this version you have to **sign out and sign in again once** with your accounts, otherwise the Twitch part of the picker stays empty. Whether something is missing is written to the **log** when connecting.

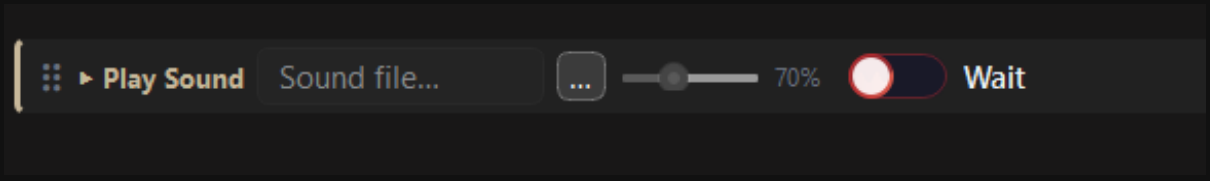
An emote does not travel with you. If you switch the sender selector of an action afterwards, a ⚠ appears next to it. It is a reminder that the emotes in the text may no longer fit the new account. A click hides it, and it is gone at the next program start at the latest.

> **The most important rule about TTS:** *SpeakerBot TTS* always reads out the **last chat action** before it — no matter how many other actions sit in between. Two chat actions right after each other and then a TTS: **only the second one** gets read out, the first is lost. The right way is to alternate: chat action → TTS → chat action → TTS.

Sound

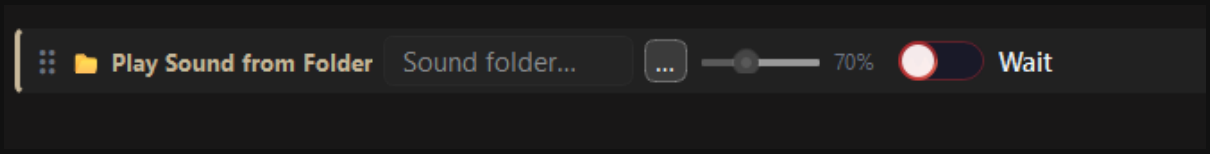


The Sound group



Play Sound with file selection, volume and Wait

- **Play Sound** — plays a sound file from your PC. You pick the file with the `...` button, next to it sit the **volume slider** and the **Wait**toggle.
- **Wait** — the event **pauses** until the sound has finished and only then moves on to the next action.
- **Stop All Sounds** — immediately stops **all** running sounds, no exceptions.

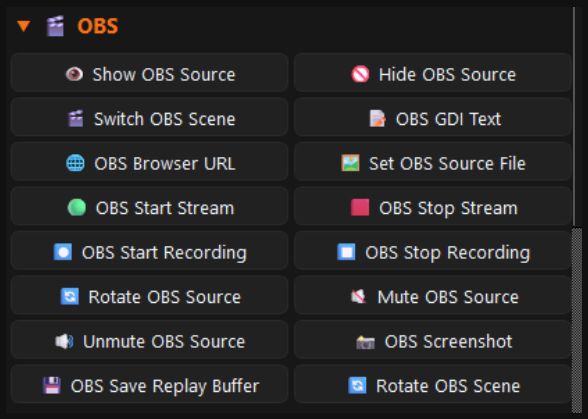


Play Sound from Folder plays all the sounds in a folder

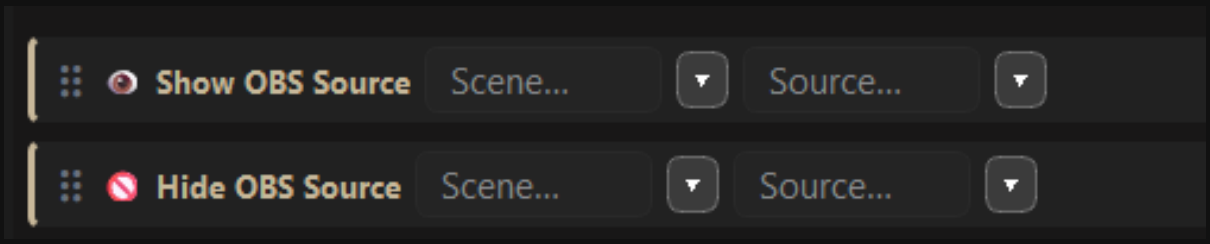
Play Sound from Folder plays **all** the sounds in the chosen folder **one after another** — in the stored order, from top to bottom. With **Wait**, things only continue once the last one is done.

> 💡 The **volume slider** only takes effect on the **next** playback. You cannot turn down a sound that is already running with it.

OBS

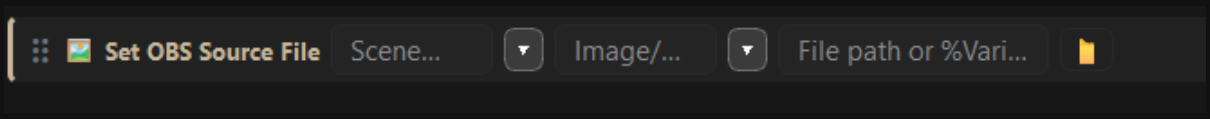


The OBS group with 14 actions



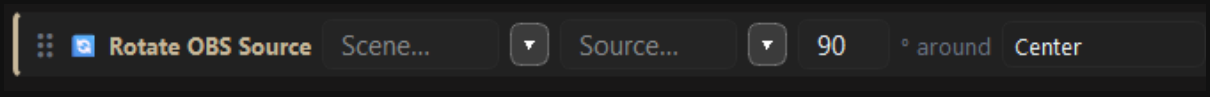
Show OBS Source and Hide OBS Source — pick scene and source

- **Show OBS Source** / **Hide OBS Source** — shows or hides a source. You pick the scene and the source.
- **Switch OBS Scene** — switches to the chosen scene.
- **OBS GDI Text** — writes text into a text (GDI+) source. Variables are allowed.
- **OBS Browser URL** — writes a URL into a browser source.
- **OBS Start Stream** / **OBS Stop Stream** and **OBS Start Recording** / **OBS Stop Recording**.
- **Mute OBS Source** / **Unmute OBS Source** — mute and turn back up.



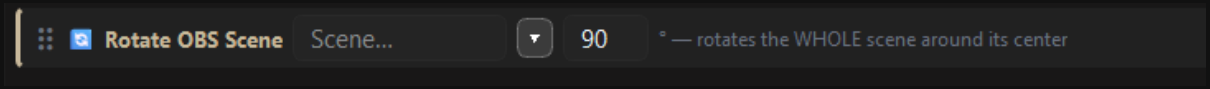
Set OBS Source File swaps the image or sound of a source

Set OBS Source File puts a new image into an **image** source or a new sound from your PC into a **media** source. You pick scene, source and the file path — a `%Variable%` is allowed here too.



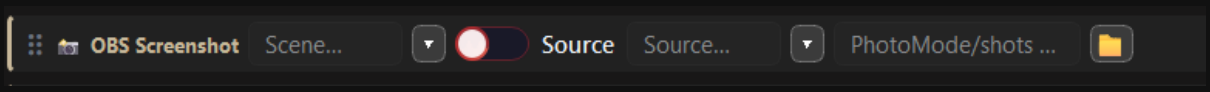
Rotate OBS Source rotates a single source

> Use **Rotate OBS Source** with care: OBS does not rotate individual sources cleanly, and the result often does not look the way you expect. If you want to be safe, take **Rotate OBS Scene**.



Rotate OBS Scene rotates the whole scene around its centre

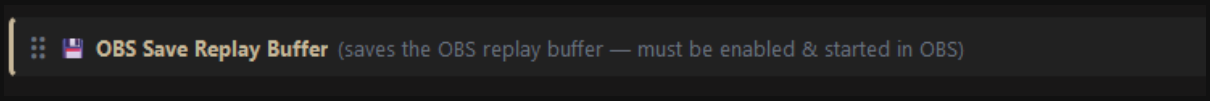
Rotate OBS Scene rotates the **entire scene** around its centre — and it does so reliably.



OBS Screenshot — either the whole scene or a single source

OBS Screenshot takes a still image. You pick the scene, but with the toggle you can switch to a single **source** instead. On the right you set the target folder.

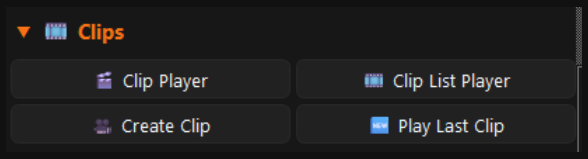
> **OBS Screenshot** has nothing to do with **Photo Mode** — those are two different things. Photo Mode has a chapter of its own.



OBS Save Replay Buffer saves the last few seconds

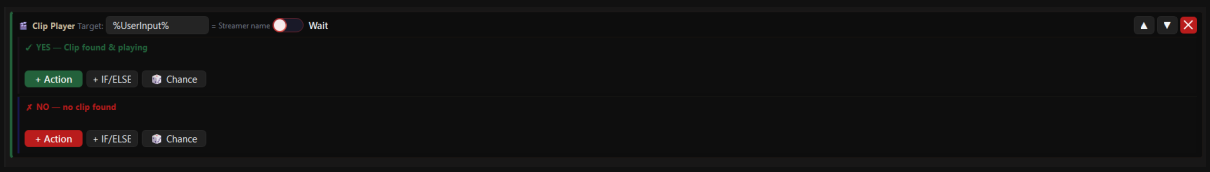
> **OBS Save Replay Buffer** only works if the replay buffer is **enabled and started** in the **OBS settings**. If it is not started, nothing happens.

Clips



The Clips group

Three of the five clip actions — **Clip Player**, **Clip List Player** and **Play Last Clip** — have a special feature: they are split into two branches.

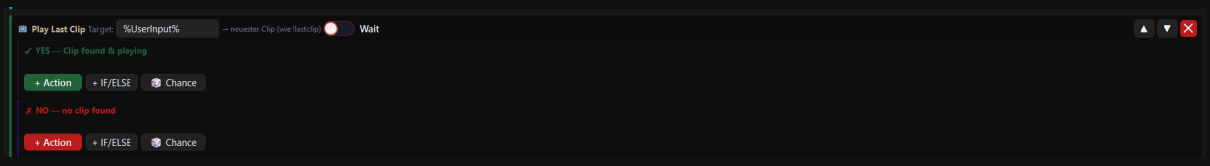


The Clip Player with its YES and NO branch

- **YES — Clip found & playing** — this is where the actions go that should run **if** a clip was found.
- **NO — no clip found** — and here the actions for the case that **none** was found.
- Both branches have their own buttons **+ Action**, **+ IF/ELSE** and **Chance**.

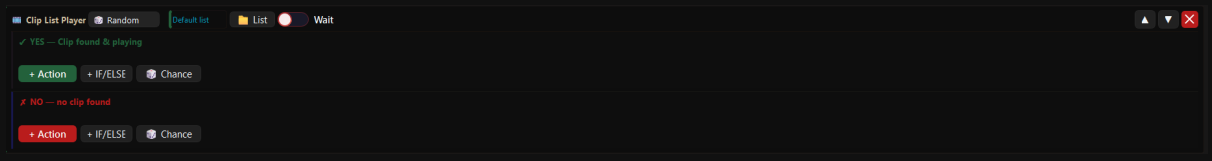
> 💡 Use the NO branch. A viewer redeems a clip reward but there is no clip — without a NO branch simply nothing happens, and the viewer thinks it is broken. A short chat message is enough.

Clip Player plays a **random** clip of the target person. The field **Target** says whose clips are meant: `%UserInput%` (the viewer types a streamer name into the reward) or `%UserName%` (the person who fires the event). The **Wait**toggle waits until the clip has finished — if no clip is found, things continue right away.

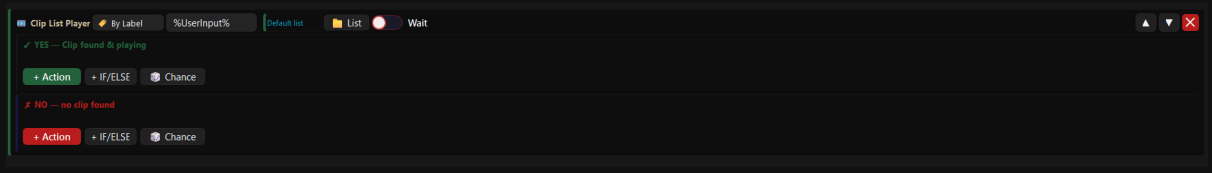


Play Last Clip plays the newest clip instead of a random one

Play Last Clip works the same way, but always plays the **newest** clip — the same thing `!lastclip` does.

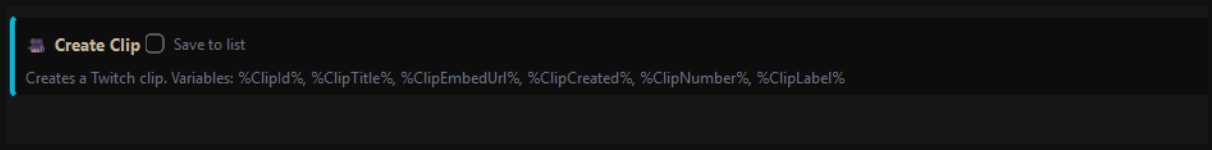


Clip List Player in Random mode

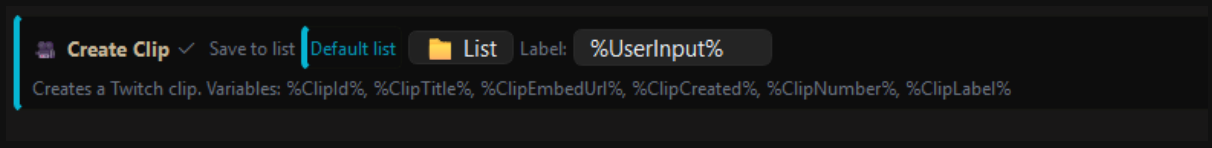


Clip List Player in By Label mode

- **Clip List Player** plays clips from a **list**.
- you have created. Mode **Random** — a random clip from the list.
- you have created. Mode **By Label** — a **particular** clip. Either through a variable (for example `%UserInput%`, when the viewer types the clip name into chat) or entered fixed — then the same clip always plays.
- The **List** button opens the **Clip List Manager**.



Create Clip — without saving into a list

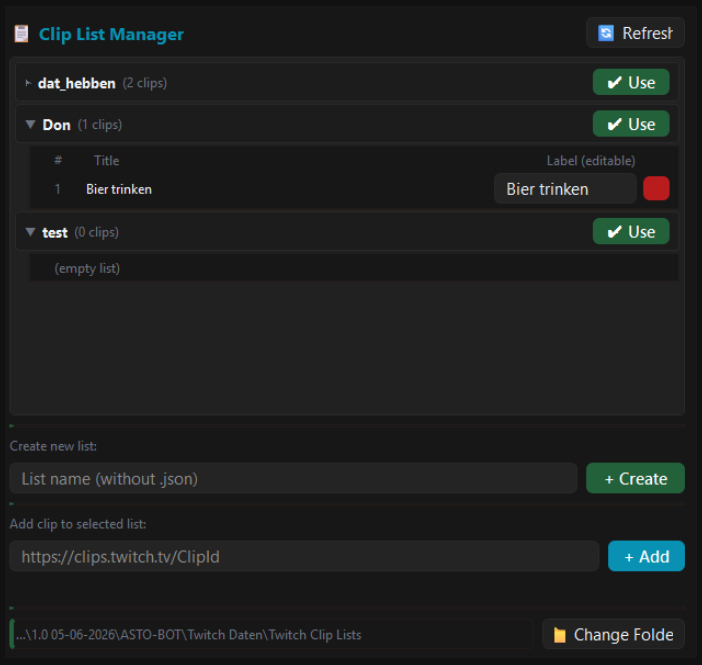


Create Clip with "Save to list" and a label from %UserInput%

- **Create Clip** creates a clip on Twitch.
- With the tick box **Save to list** the clip also lands in a list. You then pick the list and give it a **label** — for example `%UserInput%`, that is the name the viewer types into chat.
- It delivers: `%ClipId%`, `%ClipTitle%`, `%ClipEmbedUrl%`, `%ClipCreated%`, `%ClipNumber%` and `%ClipLabel%`.

Clip Playlist controls the playlist from the Clips page (chapter 8.5): **Start**, **Stop** or **Toggle**. So it can start by itself as soon as you switch to a particular OBS scene — and stop again when you leave it.

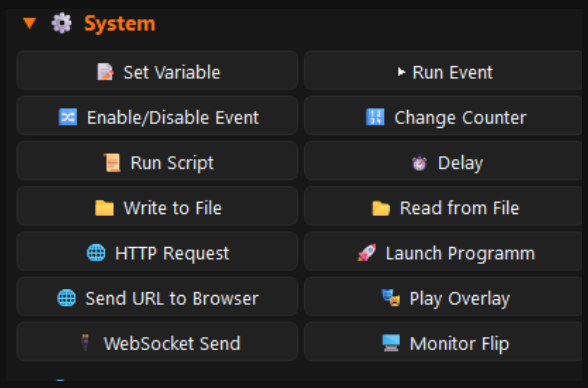
The Clip List Manager



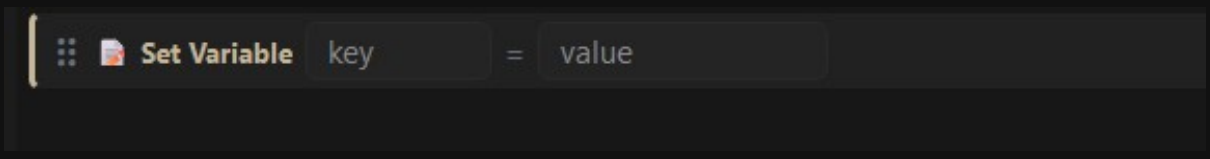
The Clip List Manager with an open list and editable labels

- You can create **several lists**. **+ Create** makes a new one — the name is entered without `.json`.
- **Use** picks the list that is worked with.
- When you fold a list open, you see the clips with **#**, **Title** and **Label (editable)**. The **label can be changed at any time** — it is the name *By Label* uses to find the clip.
- The **red button** deletes a clip from the list.
- **Add clip to selected list:** paste the clip URL (`https://clips.twitch.tv/ClipId`) and press **+ Add**. Alternatively the action **Create Clip** enters the clips automatically.
- **Change Folder** decides where the lists are stored and loaded from. **Refresh** reloads them.

System

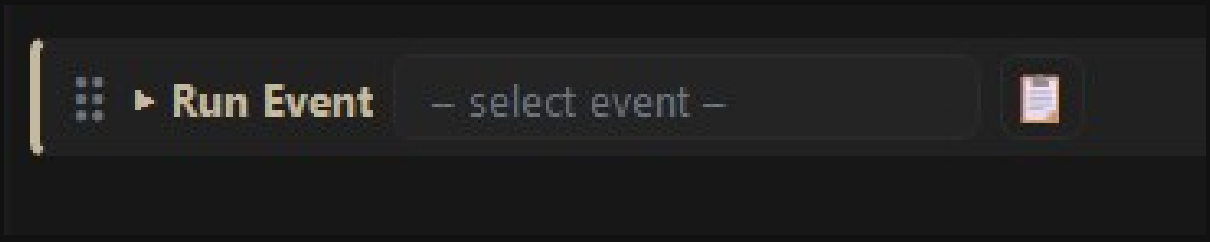


The System group with 12 actions



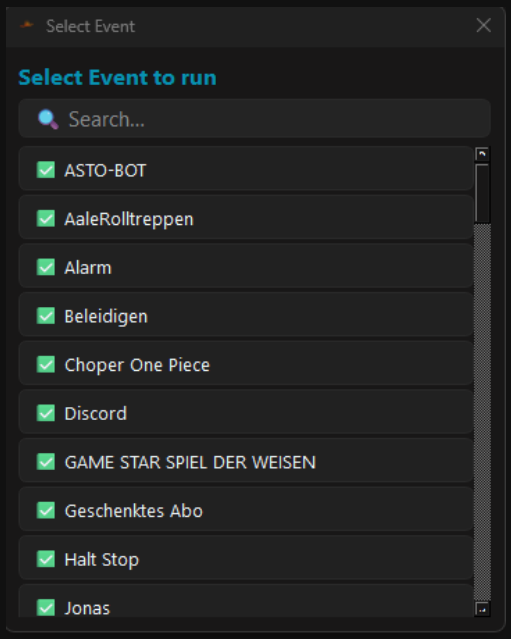
Set Variable — key = value

> **Set Variable** creates a variable that is only valid **inside this event**. It comes into being when the event starts and is deleted afterwards. Outside the event it **cannot** be used.

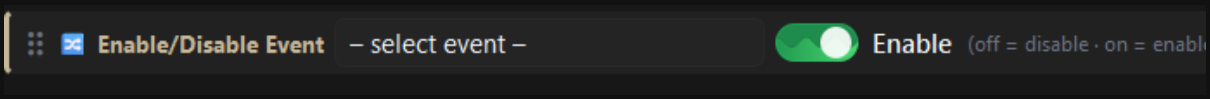


Run Event starts another event

Run Event starts another event. The small list button next to it opens a list of all events with a search field.

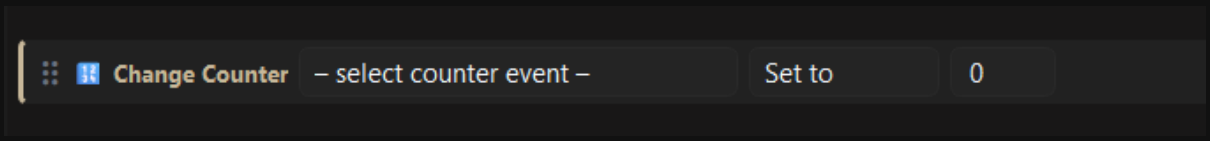


The event selection with a search field



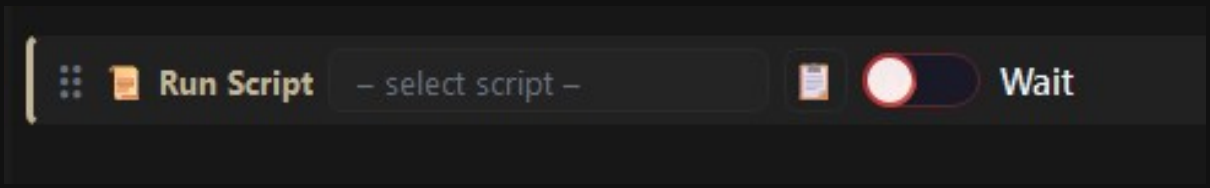
Enable/Disable Event — the toggle decides the direction

Enable/Disable Event switches an event on or off. The toggle decides the direction: green (`on`) = switch on, red (`off`) = switch off.



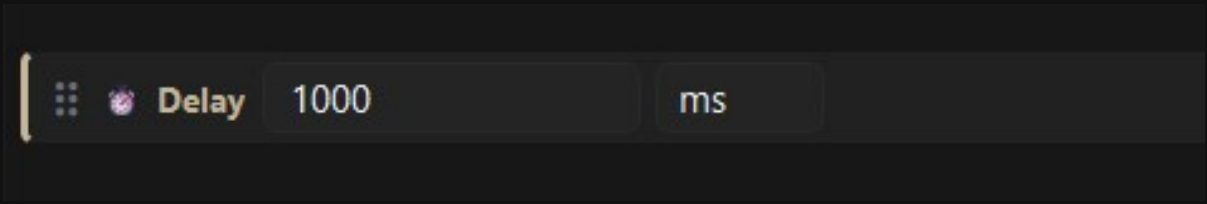
Change Counter changes the counter of another event

Change Counter changes the counter trigger of an event. The options are **Set to**, **Add** and **Reset to 0**.



Run Script runs an ASTOSCRIPT

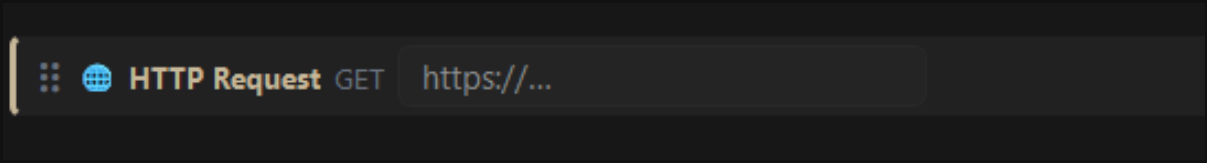
Run Script runs a script from the Scripts area (see the Scripts chapter). With **Wait** the event waits until the script has finished.



Delay — the unit can be switched

Delay makes the event wait. The unit can be switched: milliseconds, seconds, minutes or hours.

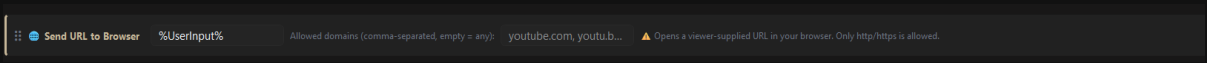
- **Write to File** — writes into a text file. Careful: the content is **overwritten**, not appended.
- **Read from File** — reads text from a file. The result ends up in `%line0%`, `%line1%` … or in `%FileContent%` and can be used further, in a chat action for example.



HTTP Request — GET only

HTTP Request calls a URL. Only **GET** is supported.

🚀 **Launch Programm** starts a program that lives on your PC — a game or a tool, for example.



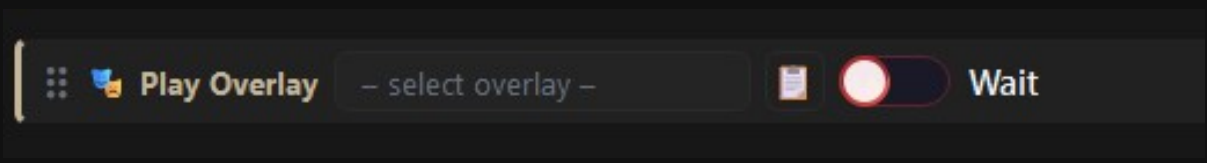
Send URL to Browser with a list of allowed domains

- **Send URL to Browser** opens a URL in your browser — typically one that a **viewer** has sent along, for instance through a channel point reward with `%UserInput%` (a YouTube link, say).
- In the field **Allowed domains** you enter, comma separated, which domains are allowed — `youtube.com, google.com`. Only `http` and `https`addresses are accepted.

> If you leave **Allowed domains** empty, **every** domain is allowed — a viewer can then open any page at all on your streaming PC, live in front of an audience. Enter a whitelist.

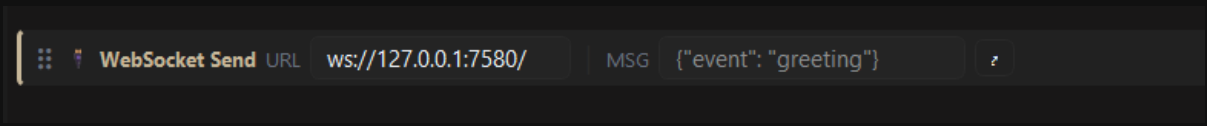
Send URL to Browser Overlay sends a URL to the **Browser Overlay** (Chapter 9.8) — so the page is shown directly on your stream, not opened in the normal browser.

> Unlike **Send URL to Browser**, there is **no** built-in domain list here. If you want viewers to be able to supply a URL, filter **beforehand with IF/ELSE** which pages are allowed — otherwise anyone can put any page at all on your stream.



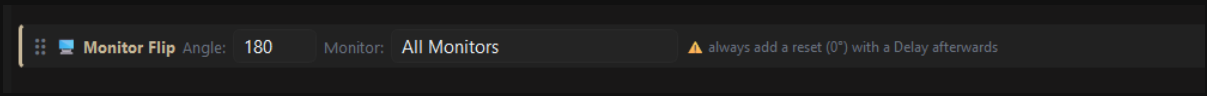
Play Overlay starts an overlay from the Overlay area

Play Overlay starts an overlay that you have built in the Overlay area (see the Overlay chapter). With **Wait**, things only continue once the overlay has run through.

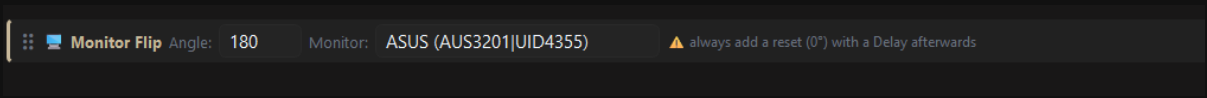


WebSocket Send sends a command to another program

WebSocket Send sends a websocket command to the given **URL** — to SpeakerBot or Streamer.bot, for example. The **MSG** field holds the message, usually as JSON. The small arrow opens a larger text field.



Monitor Flip — here all monitors are rotated

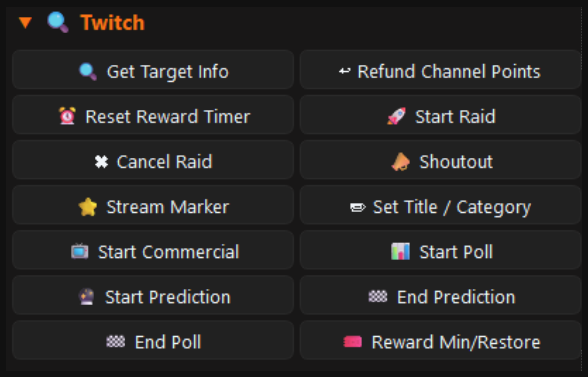


Monitor Flip on a particular monitor, recognised by its hardware ID

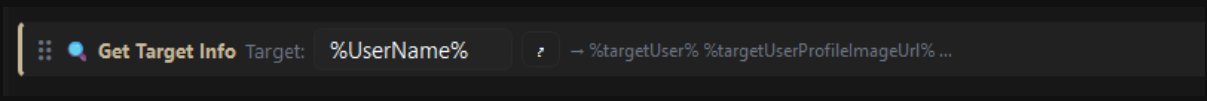
Monitor Flip rotates a monitor. Under **Angle** you set the angle, under **Monitor** you either pick a particular screen — it is recognised by its hardware ID, `ASUS (AUS3201|UID4355)` for example — or **All Monitors** for all of them at once.

> **Monitor Flip always belongs in the event at least twice.** The first action rotates the monitor (to `180`, for example), then comes a **Delay**, and the last action rotates it back with `0`. If you forget to rotate it back, your picture stays upside down — in the middle of the stream, and you have to put it straight again while looking at it head over heels.

Twitch

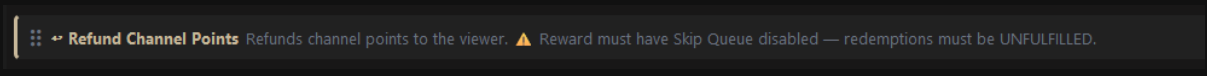


The Twitch group and its actions



Get Target Info fetches the Twitch data of a person

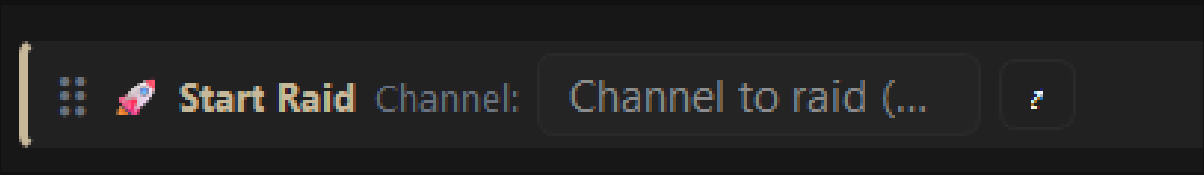
Get Target Info fetches all the Twitch information about a person: biography, profile picture, games played and more. The **Target** field says who it is about — `%UserName%` (whoever fired the event) or `%UserInput%` (the name somebody typed in). The data ends up in variables such as `%targetUser%` and `%targetUserProfileImageUrl%` and can then be used in the prompt.



Refund Channel Points gives back the channel points that were spent

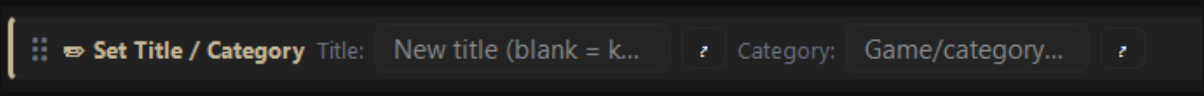
Refund Channel Points gives the viewer back the channel points they spent on the event.

> **Refund Channel Points** only works if **Skip Queue is switched off** on the reward. The redemption must still be **unfulfilled** on Twitch — whatever has already been confirmed can no longer be refunded.



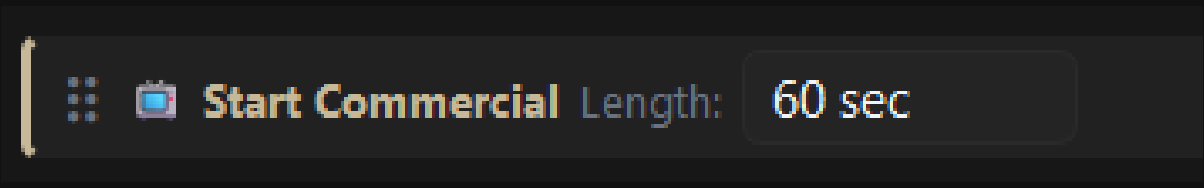
Start Raid — a fixed channel name or a variable

Start Raid starts a raid. You can enter the channel fixed or use a variable: with `%UserInput%` and a chat command like `!raid CaptainApolloo`, exactly the channel named in chat gets raided.

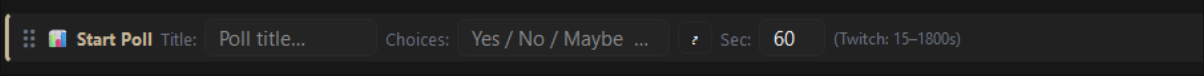


Set Title / Category — both or just one of the two

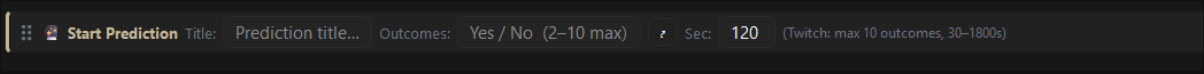
Set Title / Category changes the title, the category or both on Twitch. An empty field stays untouched. Variables are allowed — with `%UserInput%` and a command like `!title New Stream Title` the chat sets the title.



Start Commercial — ad length in seconds



Start Poll with title, answers and runtime



Start Prediction — two to ten outcomes

- **Start Commercial** — starts the ad for the length you set.
- **Start Poll** — starts a poll. **Title**, **Choices** (the answers) and **Sec** for the runtime (Twitch allows 15–1800 seconds).
- **Start Prediction** — starts a prediction. **Outcomes** are the possible results (2 to 10), **Sec** the runtime (30–1800 seconds).
- **End Poll** and **End Prediction** — end the poll or the prediction early.

Reward On/Off switches channel point rewards on, off or to paused. The dropdown offers **Enable**, **Disable**, **Pause** and **Resume**; below it you pick the rewards concerned — several at once are allowed.

The point of it shows the moment you run more than one chat game: while your maze is running, nobody should be able to start the other game alongside it. So **Disable** the other reward at the start of the event and **Enable** it again at the end. **Pause** is the gentler route — the reward stays visible but cannot be redeemed. **Enable** lifts a pause along with it.

> Twitch only lets an application change rewards **it created itself** — the same rule as with Refund Channel Points. A reward you made by hand in the Twitch dashboard is out of ASTO-BOT's reach; the log then says *read-only (created by another app)*. Recreate it through the Reward Editor in that case.

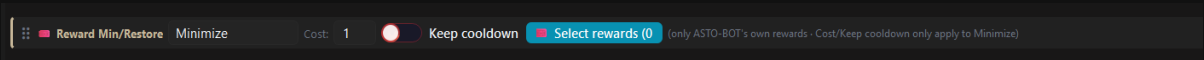
> 💡 Mind where the **Enable** sits. If it follows a **Run Script** in the same event, it fires at once, because Run Script does not wait for the script — the reward would be free again before the game had even begun. Either switch on **Wait** at the Run Script, or put the Enable into a separate closing event that your script fires at the end.

The remaining Twitch actions explain themselves as soon as you open them:

- **Reset Reward Timer** — puts the cooldown timer of a channel point reward back to its full value. If three of five minutes have already run down, five minutes are due again afterwards.
- **Cancel Raid** — cancels a running raid.
- **Shoutout** — gives a Twitch shoutout.
- **Stream Marker** — sets a marker so that later, in the VOD, you find more quickly where something happened.

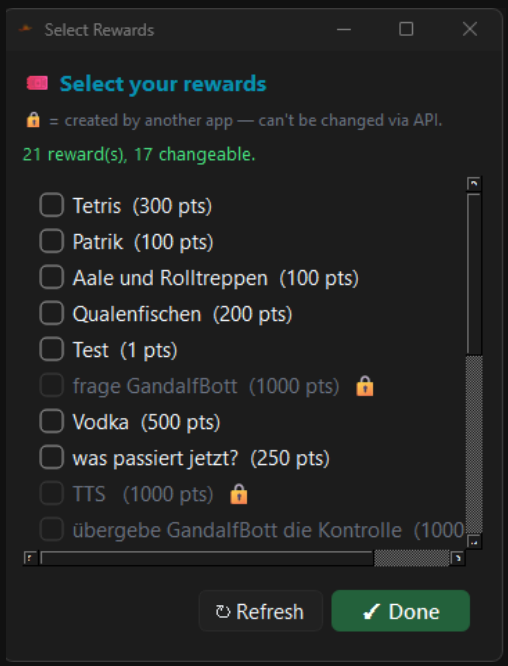
Reward Min/Restore

This action temporarily lowers the **cost** and the **cooldown** of channel point rewards — and puts them back afterwards.



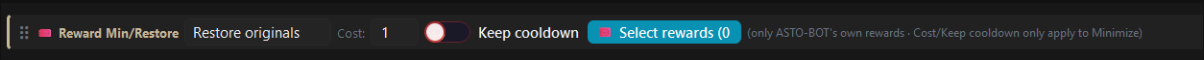
Minimize lowers the cost, here to 1 point

- In the mode **Minimize** you set the new **Cost**, for example `1` channel point.
- The toggle **Keep cooldown** keeps an existing cooldown. If it is off, the cooldown is set to `0`.
- **Cost** and **Keep cooldown** apply **only** to Minimize.
- With **Select rewards** you choose which rewards are affected — there may be several.



The selection window — the padlock marks rewards from other apps

> Only rewards that **ASTO-BOT owns** can be changed. Rewards created by another app are marked with a **padlock** in the selection window — Twitch does not allow touching them through the API.

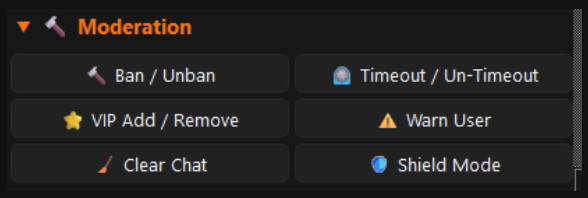


Restore originals puts the original values back

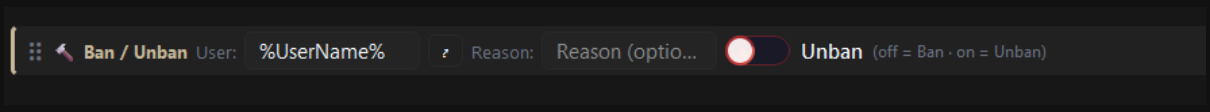
In the mode **Restore originals** the cost and the cooldown are set back to their original values.

> 💡 The usual pattern: **Minimize** at the **beginning** of the event, **Restore originals** at the **end**. In between sits whatever should happen — one minute, for example, in which a reward costs a single channel point.

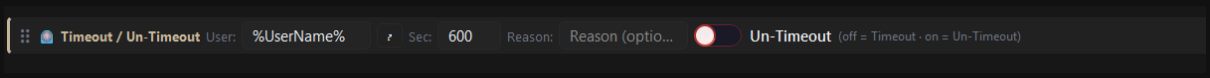
Moderation



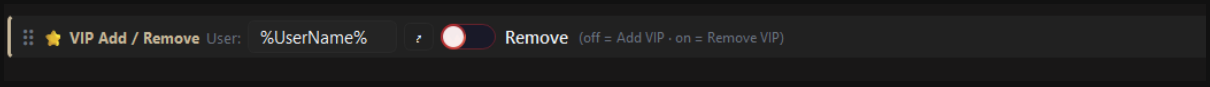
The Moderation group



Ban / Unban — the toggle decides the direction



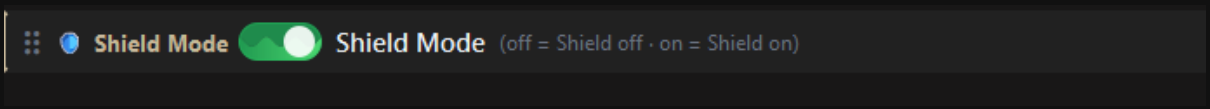
Timeout / Un-Timeout with duration and reason



VIP Add / Remove

Three of these actions can do **both** — the toggle on the right decides which way it goes:

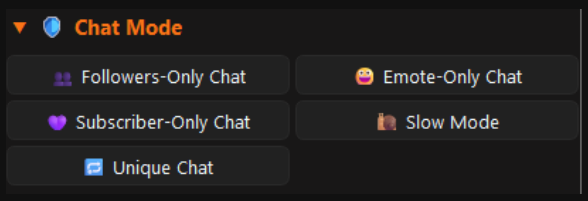
- **Ban / Unban** — toggle off = **Ban**, toggle on = **Unban**. In the field **Reason** you can optionally give a reason.
- **Timeout / Un-Timeout** — toggle off = **Timeout** for the seconds you set, toggle on = **Un-Timeout**. A reason is possible here too.
- **VIP Add / Remove** — toggle off = **grant VIP**, toggle on = **remove VIP**.
- In the field **User** says who it hits — usually `%UserName%`.



Switching Shield Mode on and off

- **Warn User** — issues a Twitch warning against the viewer.
- **Clear Chat** — clears the chat history for everyone.
- **Shield Mode** — switches the Twitch Shield Mode on or off.

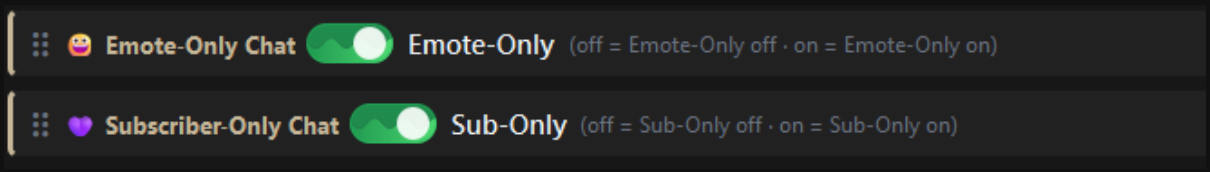
Chat Mode



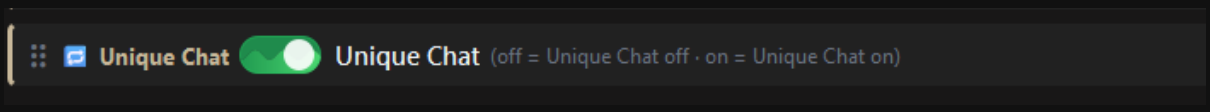
The Chat Mode group

Followers-Only Chat — the waiting time comes from the dropdown

- **Followers-Only Chat** — puts the chat into followers-only mode. In the dropdown you choose how long somebody has to have been following: `Off`, `0 min (all followers)`, `10 min`, `30 min`, `1 hour`, `1 day`, `1 week`, `1 month` or `3 months`. With `Off` you switch the mode off again.
- **Slow Mode** — switches slow mode on; the waiting time is chosen from a dropdown as well.



Emote-Only and Subscriber-Only — simple toggles

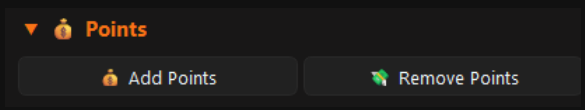


Switching Unique Chat on and off

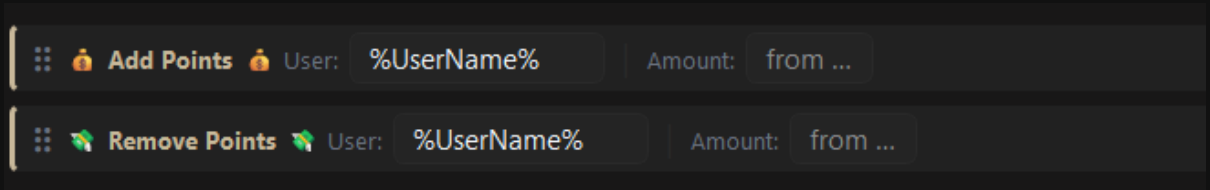
Emote-Only Chat, **Subscriber-Only Chat** and **Unique Chat** are plain toggles: on means switch on, off means switch off.

> 💡 A chat mode event goes well with the Timed trigger: switch emote-only on for a minute — with a second event that switches it off again. Or in one event with a **Delay** in between.

Points



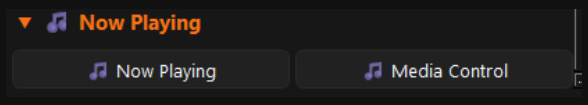
The Points group



Add Points and Remove Points

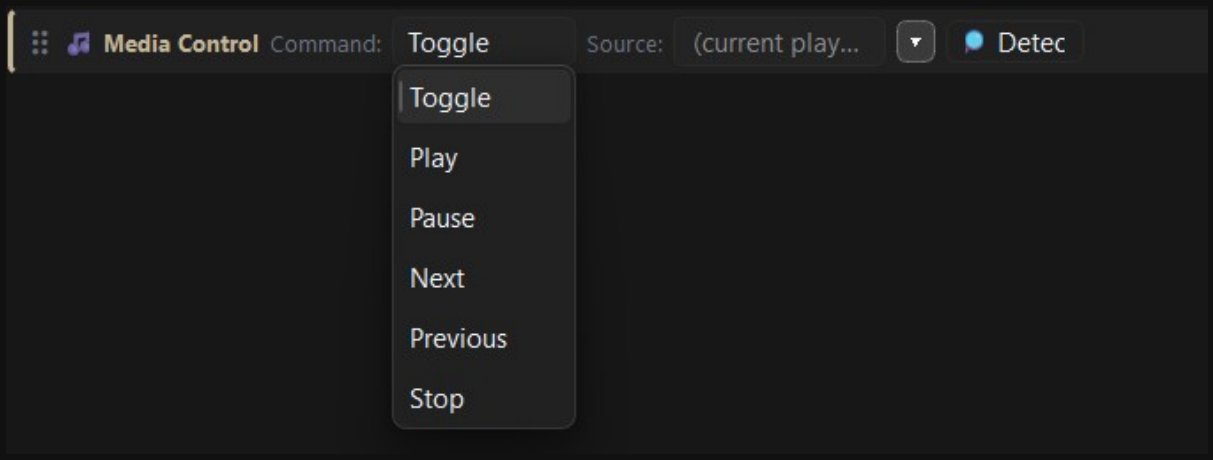
ASTO-BOT has a **points system** of its own, which is managed in the Giveaway area (see the Giveaway chapter). With **Add Points** you give a viewer points, with **Remove Points** you take them away again. The **User** field says who is meant, the **Amount** field how many points. For **Amount** both are allowed: a fixed number or a **variable**.

Now Playing



The Now Playing group

Now Playing fetches from the music program — Spotify, Apple Music or a browser — the information about what is currently playing and stores it in variables: `%NowPlaying%`, `%SongTitle%`, `%SongArtist%`, `%SongAlbum%`, `%MusicSource%` and `%MusicStatus%`. The action is built like the Music trigger.



Media Control drives the player directly

Media Control controls the program you pick in the dropdown. Under **Command** you find **Toggle**, **Play**, **Pause**, **Next**, **Previous** and **Stop**. With **Detect** ASTO-BOT looks for the running player.

> The same applies here: the program must be **playing sound at that moment**, otherwise Windows delivers no information — neither for **Now Playing** nor for **Detect**. A paused player is not found.

Giveaway

With the action **Draw Giveaway** ASTO-BOT settles a giveaway automatically — exactly as if you had pressed **Draw** on the Giveaway page yourself. That way a reward, a chat command or a timer can start the draw without you touching the machine.

The 📋 button opens the picker. It lists **all** giveaways — active and inactive — each with its status, number of entries and, if one has been drawn already, the current winner. The **active** switch only decides whether viewers can still enter; a giveaway can be drawn either way.

The **Announce** switch (on by default) decides whether the events bound to this giveaway as **Giveaway Winner** fire as well. Turn it off and the action only draws — you then make the announcement yourself in the following actions of the same event.

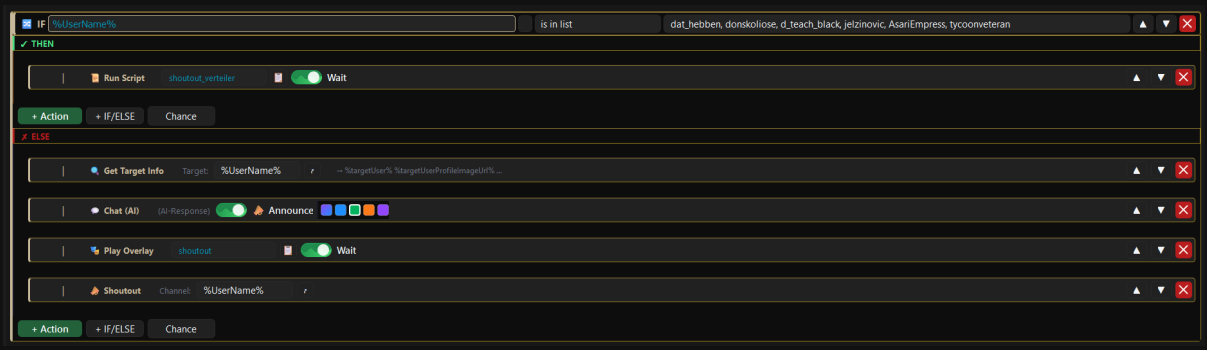
After the draw the results are ready as variables and can be used in **every following action** of the same event: `%GiveawayWinner%`, `%GiveawaySecond%`, `%GiveawayThird%`, `%GiveawayName%`, `%GiveawayEntryCount%` (number of entries), `%GiveawayResult%` (`drawn`, `not_found`, `no_entries`) and `%GiveawaySuccess%` (`true` / `false`).

> 💡 Put an **IF/ELSE** on `%GiveawaySuccess%` **directly behind** the action. If a giveaway is empty or the name is misspelled, your event would otherwise announce an empty winner. In the ELSE branch `%GiveawayResult%` tells you what went wrong.

> ⚠ **Careful — endless loop!** Never use the trigger **Giveaway Winner** and the action **Draw Giveaway** for the **same** giveaway inside **one** event while **Announce** is on. The draw fires the winner event, that event draws again, fires again — the event calls itself endlessly and floods your chat and log. **Pick one of these three instead:** split trigger and action across **two separate events** (one draws, a second reacts to Giveaway Winner) · or turn **Announce off** in the drawing event and put the announcement right behind it · or choose a **different giveaway** in the action than in the trigger.

5.7 IF/ELSE — conditions

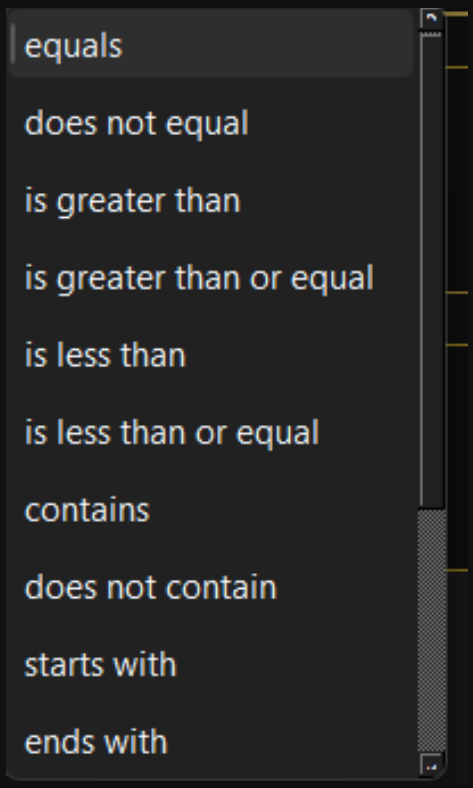
With **+ IF/ELSE** your event makes decisions. A condition always consists of three parts: a **variable**, an **operator** and a **value**. If it is true, the **THEN** branch runs — otherwise the **ELSE** branch.



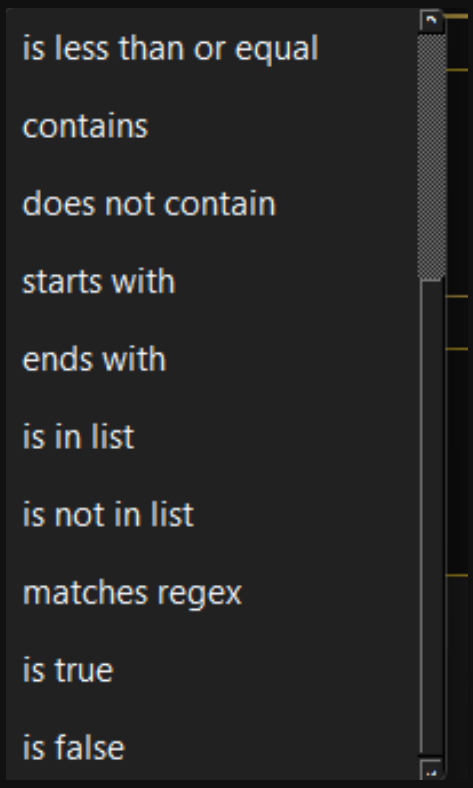
A finished IF/ELSE: the regulars get the script, everybody else the standard shoutout

On the left you enter the **variable** — `%UserName%`, `%CheerAmount%`, `%IsFollower%`, `%Role%` and everything else the Variables window has to offer. In the middle you pick the **operator**, on the right sits the **value** it is checked against.

The operators



The operators, part one



The operators, part two

- **equals** / **does not equal** — is exactly equal / is not equal.
- **is greater than** / **is greater than or equal** — greater than / greater than or equal. For numbers, bits for example.
- **is less than** / **is less than or equal** — less than / less than or equal.

- **contains** / **does not contain** — contains / does not contain.
- **starts with** / **ends with** — starts with / ends with.
- **is in list** / **is not in list** — is in a list / is not in it. You enter the list comma separated into the value field.
- **matches regex** — checks against a regular expression. For everyone who knows what they are doing.
- **is true** / **is false** — for variables that can only be true or false, such as `%IsFollower%` or `%IsGift%`. Here the value field stays empty.

Variable against variable

The value field on the right does not have to hold a fixed number — you can put **another variable** there. So `%FollowDays%` **is greater than or equal** `%GiveawayMinFollowDays%` compares how long someone has been following against what you set on the giveaway.

The point: the number then lives in **one** place only. Change the minimum follow age on the giveaway from 10 to 30 days and the condition follows along. With a fixed `10` typed in, you have to remember to change it in both places — otherwise ASTO-BOT rejects at 29 days while your message still talks about 10.

> Only names ASTO-BOT **knows** are resolved. A `contains` on `50%` therefore stays an ordinary text comparison, and a regular expression containing a percent sign behaves as before. Mistype a name and it stays put — so the mistake shows up instead of quietly matching nothing.

THEN and ELSE

You can put any number of actions into **both** branches — each branch has its own buttons **+ Action**, **+ IF/ELSE** and **Chance** at the bottom.

> Nesting goes **three levels deep**, and each level takes **several** IF/ELSE and Chance blocks side by side — the buttons no longer disappear once one has been placed.
>
> What does **not** belong inside a branch are the clip actions: **Clip Player**, **Clip List Player**, **Play Last Clip** and **Create Clip** stay on the top level. Nested, their playback was unreliable, so they can neither be created nor dragged in there.

Three examples

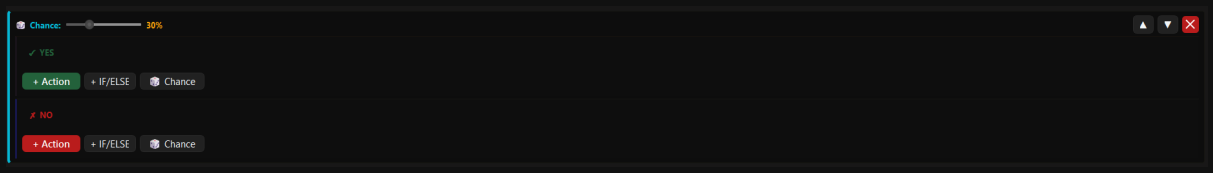
Example 1 — treating the regulars differently. That is exactly what the screenshot above shows: `%UserName%` **is in list** `dat_hebben, donskoliose, d_teach_black, ...` → in the **THEN** branch a script runs (`shoutout_verteiler`) that does something special for these people. Everybody else lands in the **ELSE** branch and gets the standard route: **Get Target Info** → **Chat (AI)** → **Play Overlay** → **Shoutout**.

Example 2 — rewarding generous bits. Condition: `%CheerAmount%` **is greater than or equal** `1000`. In the **THEN** branch: a louder sound, a **Pin Message (AI)** and an overlay. In the **ELSE** branch: an ordinary **Chat (AI)** message. That way the big cheer gets its moment without every small cheer setting off the whole circus.

Example 3 — picking up non-followers kindly. Condition: `%IsFollower%` **is false**. In the **THEN** branch a chat message that points to the follow button in a friendly way. In the **ELSE** branch the ordinary greeting. Important: with `is false` the value field stays empty.

5.8 Chance — leaving it to luck

The **Chance** button inserts a random block. At the top sits a **percentage slider**: it decides how likely it is that the **YES** branch runs. At 30 % that happens on average on three out of ten triggers — in the other seven cases the **NO** branch runs.



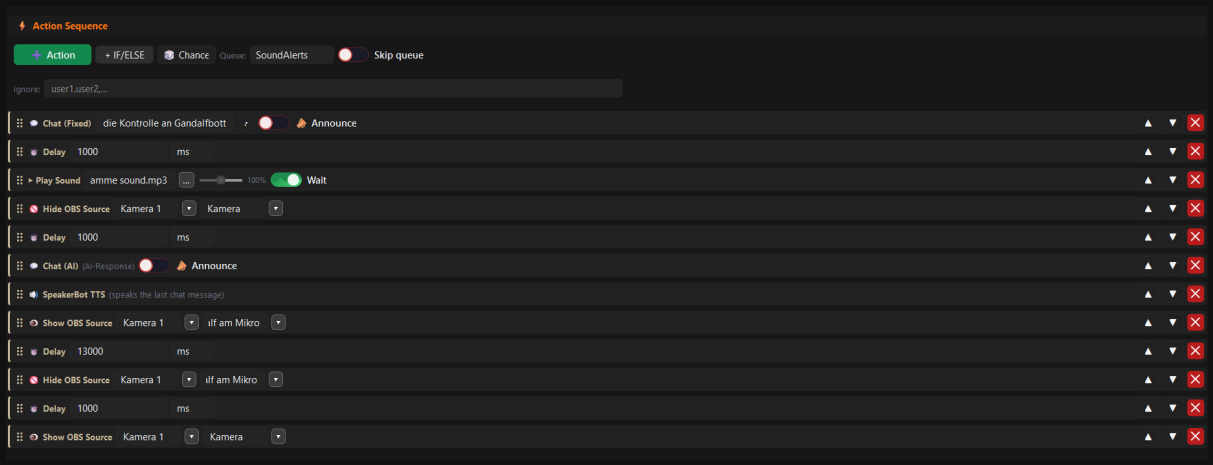
The Chance block with the percentage slider, YES branch and NO branch

As with IF/ELSE you can put actions into both branches — and in there again IF/ELSE blocks and further Chance blocks.

> 💡 Chance brings events to life. A follow sound that is a different one 10 % of the time creates exactly the small surprises your chat talks about.

5.9 An event from start to finish

To finish, a complete event as it looks in practice. The viewer fires it, ASTO-BOT announces it in chat, plays a sound, hides the camera, has the AI say something, reads it out, shows a different camera angle and tidies up again at the end.



A finished event — twelve actions in a row

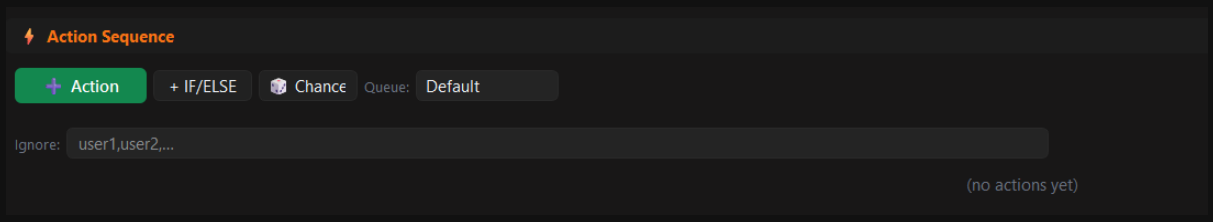
Read from top to bottom:

- **Chat (Fixed)** — a fixed announcement in chat.
- **Delay 1000 ms** — one second of pause so the message arrives before things move on.
- **Play Sound** with **Wait** — the sound plays and the event waits until it is done.
- **Hide OBS Source** — the camera is hidden.
- **Delay 1000 ms**
- **Chat (AI)** — the AI writes its message, steered by the prompt of the event.
- **SpeakerBot TTS** — reads out exactly that message. It sits directly above, which is why it works.
- **Show OBS Source** — a different source is shown.
- **Delay 13000 ms** — thirteen seconds during which you can see it.
- **Hide OBS Source** — and off again.
- **Delay 1000 ms**
- **Show OBS Source** — the camera comes back. The starting state is restored.

To the right of every line you see ▲▼ for moving and the red ✕ for deleting. At the top sits the **queue** the event runs in — *SoundAlerts* here.

> 💡 Two things this example shows nicely: first, a clean event tidies up after itself — whatever you hide, you show again. Second, **Chat (AI)** and **SpeakerBot TTS** sit right after one another, so that what gets read out really is what the AI has just written.

An empty event looks like this at the start — and from here you build downwards, action by action:

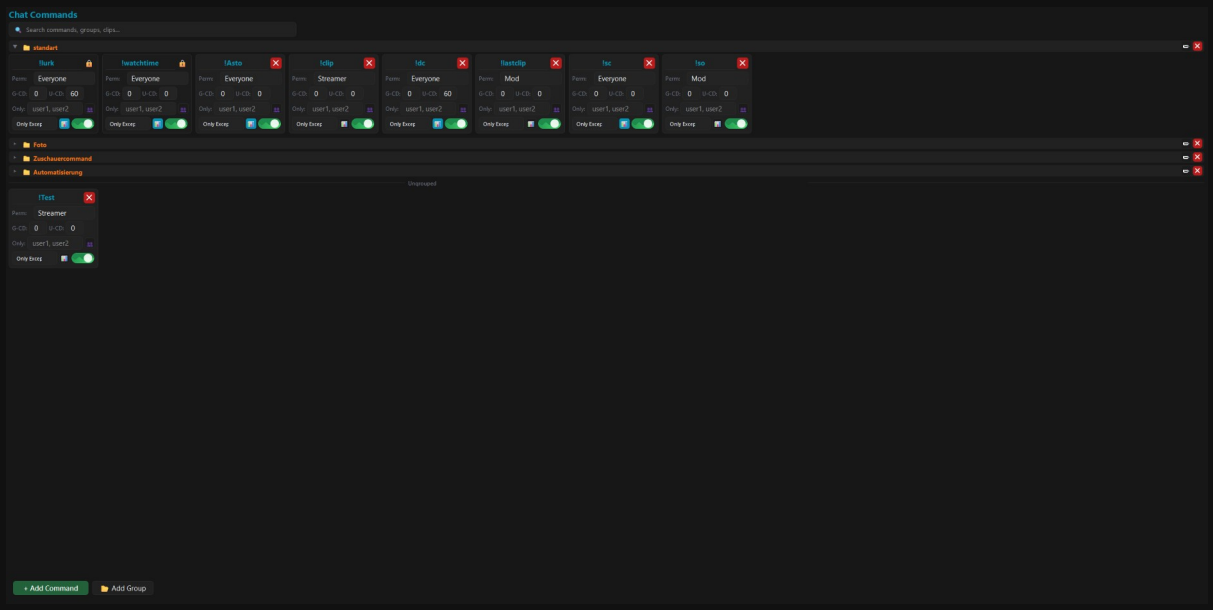


An empty Action Sequence — this is where every event begins

6. Command

In the **Command** area you create the chat commands of your channel — `!clip`, `!lurk`, `!discord`, whatever you need. What matters is what a command is **not**: it does **nothing** by itself.

> 💡 **A command is only a trigger.** What should happen when somebody types it in chat is built in the **event** — there you pick the command under the trigger **Chat Command** (see chapter 5.4.1). Here you only decide: what is the command called, who may use it, and how often?



The Command area with groups, cards and the “Ungrouped” section

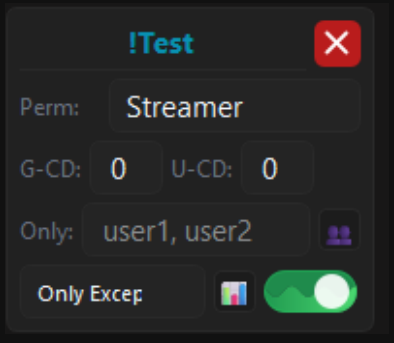
6.1 How the page is built

- At the very top sits the **search field** — it searches commands, groups and clips.
- Below that are the **groups**. Every group can be folded open and closed, renamed and deleted.
- Commands without a group gather at the bottom under **Ungrouped**.
- Bottom left: **+ Add Command** and **Add Group**.

+ Add Command immediately creates a new, empty card — no dialog in between. You name it right on the card. Moving works with **drag & drop**: simply pull the card into another group.

6.2 The command card

Every command is a small card. Everything you can set sits right on it.



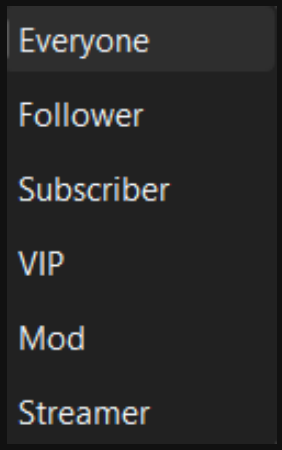
A command card with all its fields

name

At the top in blue sits the **name** of the command — and that is where you change it too. A command has exactly **one** name; **there are no aliases**, so `!so` and `!shoutout` would be two separate commands.

Who — who is allowed?

The **Who** dropdown decides who may fire the command — in older versions it was called *Perm*. The levels build on each other: whoever stands higher up may always do what is allowed further down as well. Hover the label or the dropdown and a tooltip shows you the whole ladder.



The permission levels

- **Everyone** — anybody in chat.
- **Follower** — followers, subscribers, VIPs, moderators and you. So everybody except the viewers who do not follow you yet.
- **Subscriber** — subscribers, VIPs, moderators and you.
- **VIP** — VIPs, moderators and you.
- **Mod** — moderators and you.
- **Streamer** — only you.

> Whether somebody is a follower is looked up at Twitch and remembered for a short while. The check only runs when a command really asks for **Follower** and the viewer is not a subscriber, VIP or moderator anyway.

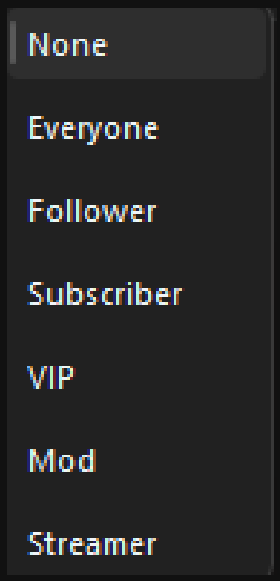
G-CD and U-CD — the cooldowns

- **G-CD** — the **global cooldown**: after it has been fired, the command is locked for **everyone**.
- **U-CD** — the **user cooldown**: it applies to **every viewer individually**. So one person can use the command while another is still waiting.

> 💡 The two together give you the usual combination: a small global cooldown against spam, a bigger user cooldown against the one viewer who types the command twenty times in a row.

Only and Only Exception

In the **Only** field you enter names — comma separated if there are several. Then **exclusively** this circle of people may use the command. The button with the two heads next to it opens a bigger text field when the list gets longer.



Only Exception — a role on top of the names

Only Exception adds a **role** to those names. The default is **None**: then the Only list applies **strictly** — nobody but the people listed there may use the command. If you pick *Subscriber* instead, the names you entered **and** all subscribers may use it; with *Follower* all followers join in, with *Streamer* only you. Here, too, a tooltip under the mouse pointer explains the whole list.

Statistics and the on/off switch

The little **table button** switches the statistics on: ASTO-BOT then counts how often the command has been used. The numbers end up in the **Command Ranking** card on the Dashboard (Chapter 3). The **toggle** on the far right enables and disables the command.

The warning symbol

If a command is linked in **several places**, a **warning symbol** appears on the card. Clicking it shows you everything it is connected to. That is not an error — it is a hint. If one command starts three events at once, you want to know that before you start wondering.

6.3 Groups

Like events, commands can be sorted into **groups** — *Viewer commands*, *Photo* or *Automation*, for example. **Add Group** creates a new group, groups can be renamed and deleted, and you drag cards from one group into the next. Whatever sits in no group appears under **Ungrouped**.

6.4 The built-in Commands

On the first start ASTO-BOT brings four commands with it: **!lurk**, **!watchtime**, **!clip** and **!lastclip**.

- **!lurk** — an ordinary command. You can ignore it or link it to a lurk event that thanks the viewers who leave the stream running in the background.
- **!watchtime** — records how long each viewer has been watching you. In events you output the value with the variable `%Watchtime%`.
- **!clip** and **!lastclip** — belong to the clip system. What they do and how you use them is described in the Clips chapter.

> **!lurk** and **!watchtime** carry a **padlock**: they **cannot be deleted**. **Renaming** them is possible at any time, though — if you prefer `!watchedtime`, take `!watchedtime`.
>
> The watchtime is counted from the moment ASTO-BOT ran along in a stream for the **first time**. It cannot know anything from before that — so the numbers only grow into something meaningful over time.

6.5 From command to event

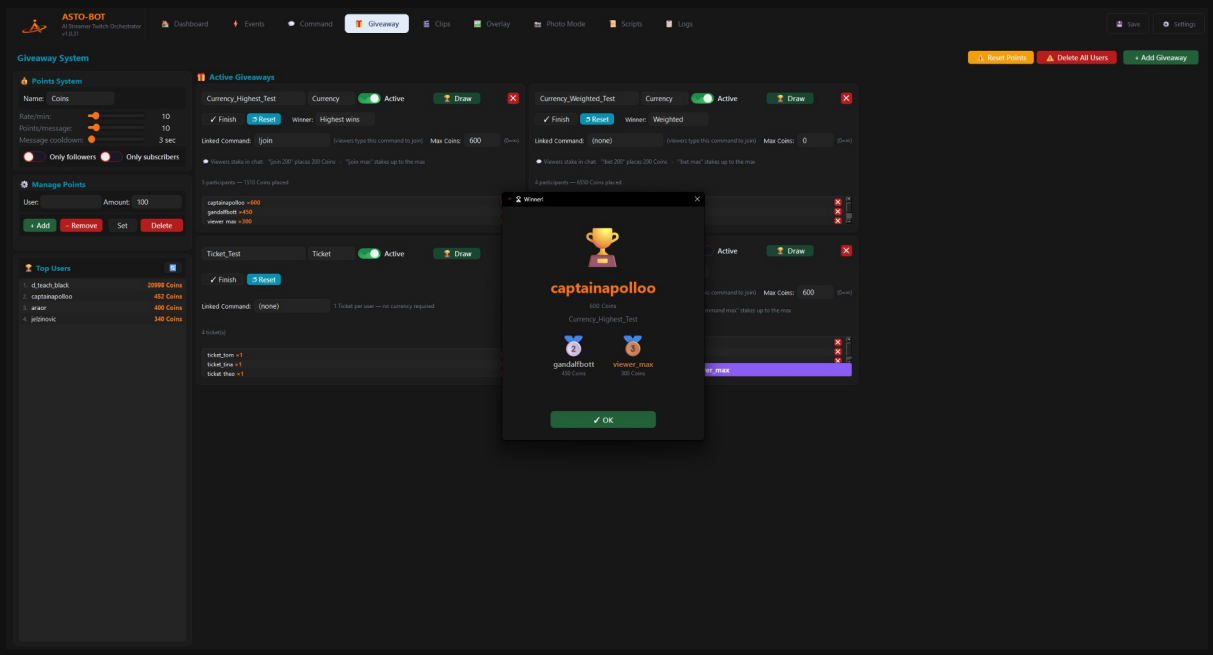
A new chat command always comes about in two steps:

- **First, here: + Add Command**, give it a name, set **Perm**, enter the cooldowns.
- **Second, in the event:** create an event (or take an existing one), pick the new command on the right in the trigger area under **General** at **Chat Command** — and decide in the Action Sequence what should happen.

> 💡 If the command is typed in chat and nothing happens, it is almost always one of three things: the command is **switched off**, there is **no event** attached to it — or a **cooldown** is still running.

7. Giveaway & Points

The Giveaway area consists of two parts that belong together: a **points system** in which your viewers collect currency for watching — and the **giveaways** in which they can spend that currency again.



The Giveaway area: the points system on the left, the active giveaways on the right

7.1 The points system

- **Name** — what your currency should be called. The default is **Coins**, but nothing stops you from calling it *Talers*, *Stars* or *Beer Mats*.
- **Rate/min** — the slider decides how many points every viewer gets **per minute of watching**.
- **Points/message** — extra points for **every chat message**. This rewards active chatters, not just silent viewers. `0` turns it off.
- **Message cooldown** — the lockout in seconds between two messages that earn points. It counts **per viewer**: with `30`, the same viewer earns from at most one message every 30 seconds. `0` means every message pays.
- **Only followers** — if the toggle is on, only followers collect points — both per minute and per chat message.
- **Only subscribers** — if the toggle is on, only subscribers collect points. With **both** toggles on, a viewer has to be a follower **and** a subscriber.

> So that **Points/message** cannot be farmed by spamming, a message only pays when it is at least **10 characters** long and contains at least **4 different** letters or digits — `aaaaaaaaaa` earns nothing. Chat commands (`!…`) and a message repeated back to back do not count either.
>
> Points only go to whoever is **watching right now** — and only while your **stream is live**. When the stream is offline nobody collects points, neither per minute nor per message, and **watchtime** does not keep counting either. That is the point of it: the points are a reward for being there in the stream.

Manage Points

This is where you step in by hand. You enter a **User** and an **Amount** and then choose:

- **+ Add** — gives the viewer points on top.

- **– Remove** — takes points away from them.
- **Set** — sets their score to exactly this value.
- **Delete** — removes the viewer **completely** from the points system.

> 💡 You do not have to type the name: a **click on a name** in the Top Users list takes it straight into the User field.

Top Users

The ranking shows who has the most points. The small button in the top right of the card **refreshes** the list — you rarely need it, that happens by itself as well.

Reset Points and Delete All Users

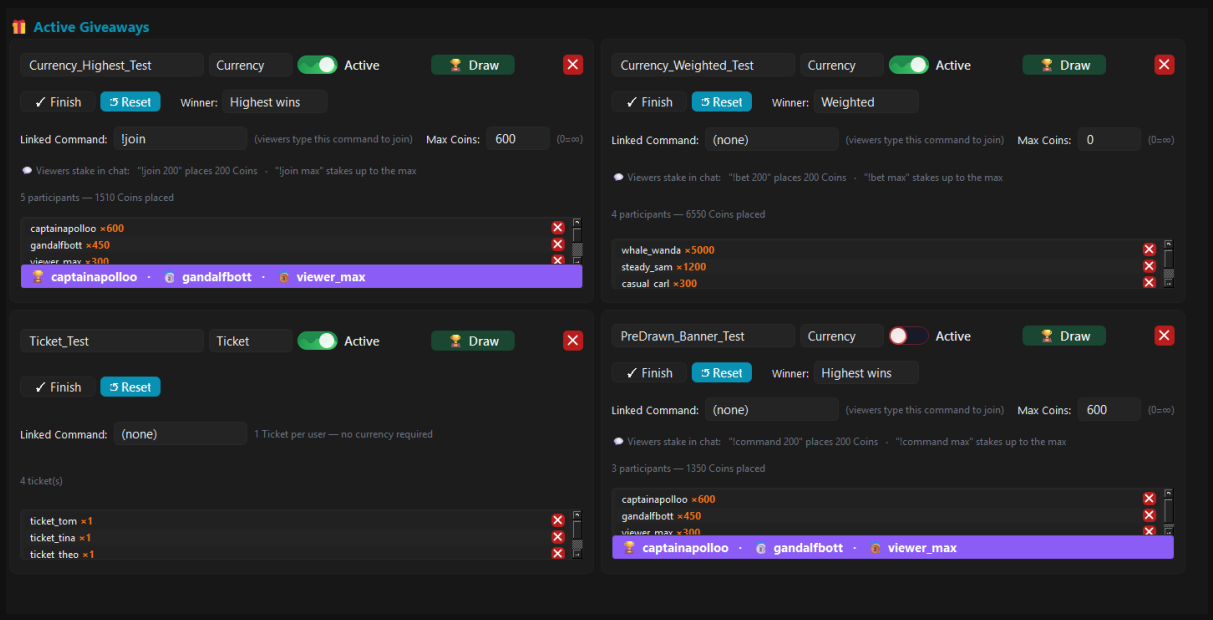
In the top right of the page sit two buttons that affect the **entire** points system:

- **Reset Points** — sets the points of **all** viewers to 0. The viewers themselves stay in the list.
- **Delete All Users** — removes **all** viewers from the points system.

> Both buttons act on **every** viewer at the same time. If your community has been collecting points for months, one click makes it all disappear. Look twice before you press.

7.2 Creating giveaways

+ Add Giveaway in the top right creates a new giveaway. You can create **any number** of them — each is shown as its own card, can be renamed, activated, deactivated and deleted again with the red ✕.



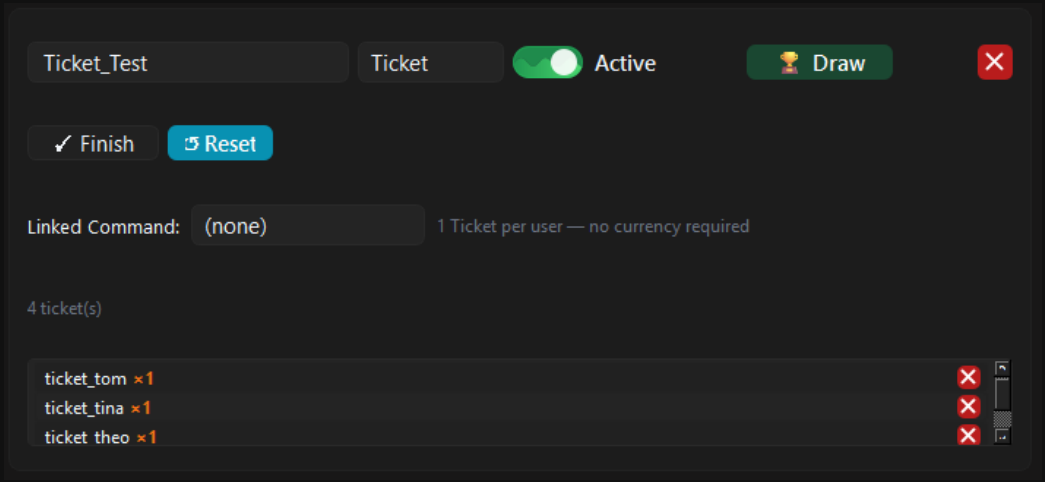
Several giveaways side by side — here in Ticket mode and in Currency mode

> All giveaways draw on **the same** points system. There is no separate currency per giveaway — the coins a viewer spends in one giveaway are missing in the other.

7.3 Ticket or Currency — the most important switch

Every giveaway runs in one of two modes. The difference is big, bigger than it looks.

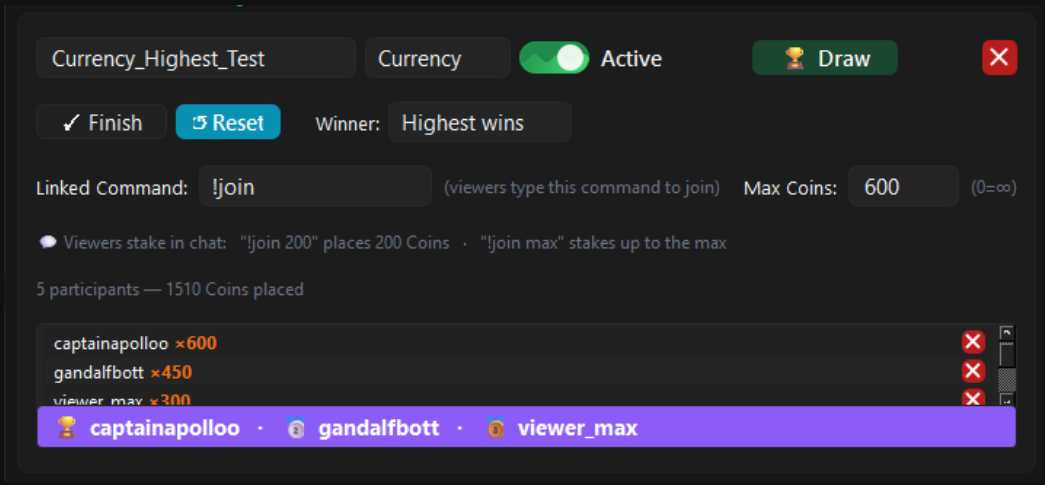
Ticket — the real draw



A giveaway in Ticket mode

In **Ticket** mode every participant has exactly **one** ticket. It costs no currency, and the draw is decided by **chance**. The viewer dropping in for the first time has the same chance as your most loyal regular. To join, the command alone is enough.

Currency — the stake counts



A giveaway in Currency mode — the participant has staked 500 coins

In **Currency** mode the viewers stake the points they have collected. **Max Coins** decides how much a single viewer may invest at most — `0` means unlimited. Below the card you see who is taking part and how much they have staked.

In the top right of the card, next to **Reset**, sits the **Winner** dropdown — it decides **how** the draw works in Currency mode:

- **Highest wins** — the highest stake **always** wins, no chance involved. If A stakes 500 coins and B stakes 499, A wins. Every time — an auction, not a raffle.
- **Weighted** — chance decides, but **weighted by the stake**: whoever stakes more has better odds, but never 100%. Someone who stakes 10 keeps a tiny chance against someone with 100000. Still, exactly **one** person wins.

> 💡 **Highest wins** rewards whoever risks the most — exciting, but predictable. **Weighted** keeps it open: small stakes can win too, big ones just far more likely. Anyone who wants a true even chance for everyone takes **Ticket**.

> The staked points are **deducted** from the viewer. They are spent — even if they do not win.

7.4 Linked Command — how viewers join

In the **Linked Command** field you enter the chat command people join with. `!join` is only an **example** — you can take any command you like.

- In the mode **Ticket** mode the command alone is enough: `!join`.
- In the mode **Currency** mode the stake comes after it: `!join 500` stakes 500 points.
- `!join max` stakes **everything** the viewer has — capped by **Max Coins**.

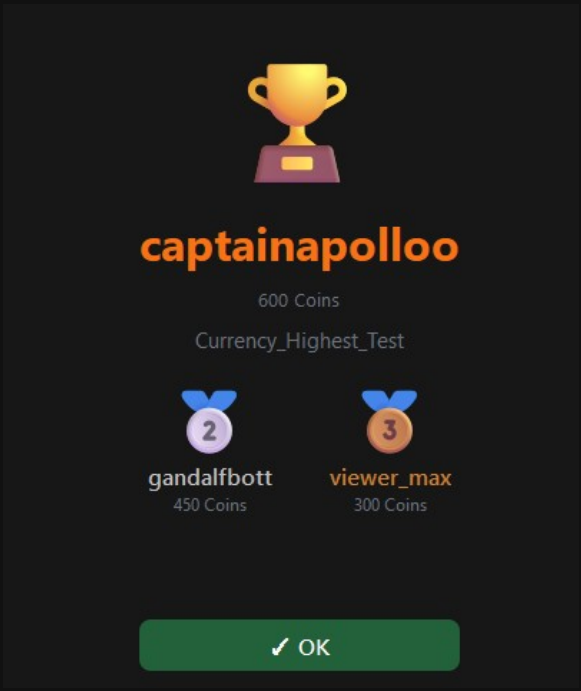
Below the card the list of participants runs along. The **red button** on a line takes **a single** participant out again.

When the list gets long, 🪀 **Open list** next to the participant count opens a window of its own with everybody in it. At the top sits a **search field**: type a name and the list filters along immediately. Deleting works exactly as it does on the card, and the changes are saved right away.

7.5 Draw, Finish and Reset

- **Draw** — draws the winner. They appear in a window of their own. If a winner was already drawn, ASTO-BOT asks first whether you really want to **re-draw** — so a stray click never overwrites the result.
- **Finish** — ends the giveaway and **deactivates** it at the same time.
- **Reset** — resets the winner and **all** participants. The giveaway is then back at zero.

The winner window



The winner window with the podium: gold, silver and bronze

At the top sits the **winner** (1st place) with their stake. Below — provided there were enough participants — the **podium**: **silver** (2nd place) on the left, **bronze** (3rd place) on the right, each with name and stake.

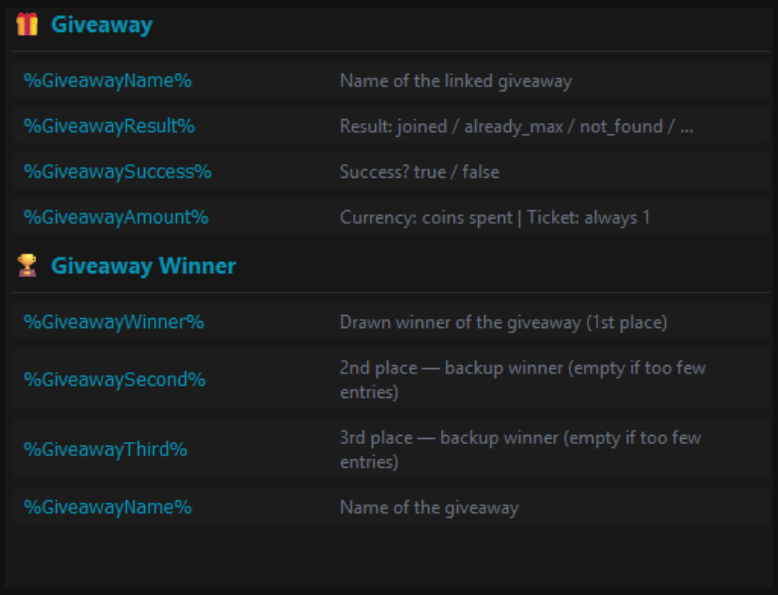
> 💡 The podium is your **safety net**: if the winner is no longer in chat or cannot be reached, 2nd and 3rd place give you an instant backup. **A click on a name** — gold, silver or bronze — copies it to the clipboard.

> In **Ticket** mode the window shows only the names, no stakes — everyone has exactly one ticket there.

The podium stays visible on the card itself too: after the draw a banner appears there in the form 🏆 **Winner** · 🥈 **Second** · 🥉 **Third**.

7.6 Celebrating the winner on stream

The winner window is seen only by **you**. So that your chat gets something out of it, you link the giveaway to an event: in the event you pick the trigger **Giveaway Winner** and select the giveaway there. As soon as you press **Draw**, the event fires — and ASTO-BOT can announce the winner, play a sound and show an overlay.



The giveaway variables in the Variables window

Two categories that should not be mixed up:

- **Giveaway** — these variables describe the **joining**, that is the moment somebody types `!join`: `%GiveawayName%` (the linked giveaway), `%GiveawayResult%` (`joined`, `already_max`, `not_found` …), `%GiveawaySuccess%` (`true` / `false`) and `%GiveawayAmount%` (in Currency the coins staked, in Ticket always `1`).
- **Giveaway Winner** — these describe the **draw**: `%GiveawayWinner%` is the winner that was drawn, `%GiveawayName%` the giveaway.
- New here are **`%GiveawaySecond%`** and **`%GiveawayThird%`** — the **second** and **third** place of the podium. This lets your event announce the backup winners too, in case your actual winner cannot be reached. If there were too few participants, they stay empty.

> 💡 The winner's name is in **`%GiveawayWinner%`** — not in `%UserName%`. If your winner event announces the wrong name, that is almost always the cause.

Letting it draw automatically

You do not have to press **Draw** yourself. The action **Draw Giveaway** (chapter 5.6, group *Giveaway*) draws the giveaway from inside an event — started by a reward, a chat command or a timer. Winner and podium end up on the Giveaway page just as if you had pressed the button.

Besides `%GiveawayWinner%`, `%GiveawaySecond%` and `%GiveawayThird%` the action also provides `%GiveawayEntryCount%` — the number of participants. That turns a dry announcement into a small show: *"🎉 The winner is %GiveawayWinner% out of %GiveawayEntryCount% entries!"*

> ⚠ An event that listens to **Giveaway Winner** must not draw the same giveaway with **Draw Giveaway** and **Announce** at the same time — it would trigger itself endlessly. Split it across two events, or switch **Announce** off in the drawing event.

If you prefer writing to clicking, ASTOSCRIPT does the same: `GIVEAWAY DRAW $winner "Name"` draws and stores the winner in a variable of your own, while `GIVEAWAY ENTRIES`,

`GIVEAWAY WINNER` and `GIVEAWAY ACTIVE` read a giveaway without drawing it. Add `ANNOUNCE` and the winner events fire as well. All commands with examples are in the tutorial on the Scripts page and in chapter 11.

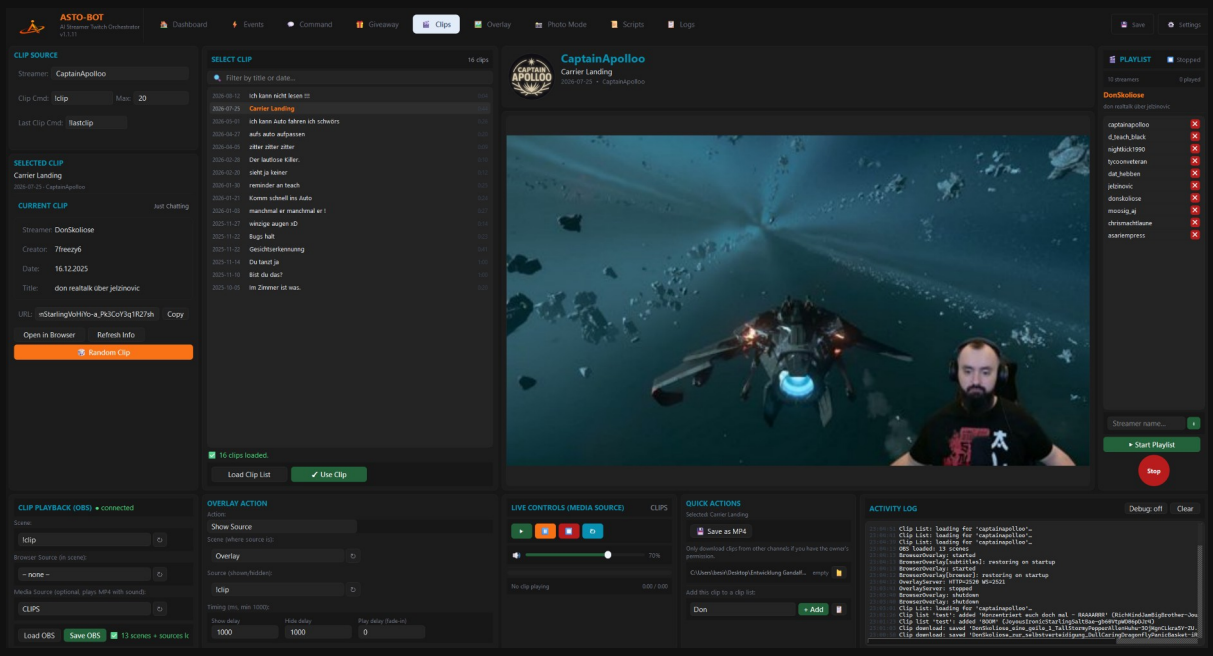
Handing out points from events

The points system does not only hang on the clock. In every event you can give away or take away points with the actions **Add Points** and **Remove Points** (group *Points*, chapter 5.6) — as a bonus for a sub, for a raid or as a prize in a chat game, for example.

> 💡 A nice loop: one event hands out 500 coins on every follow, a second event announces the winner after the draw. Between the two sits nothing but the points system — and your chat has a reason to stick around.

8. Clips

The Clips area is command centre and workshop in one: here you pick clips out and play them — and here you decide **how** clips appear in your stream in the first place.



The Clips area with clip list, preview, playback settings and playlist

> 💡 The settings under **Clip Playback** and **Overlay Action** apply to **all** the ways a clip can be played — to this page, to `!clip` and `!lastclip`, to the actions **Clip Player**, **Clip List Player** and **Play Last Clip** from the Events chapter and to the playlist. Everything runs through **the same** source and is shown and hidden the same way. Set it up properly once and you will have peace.

8.1 Streamer and chat commands

In the top left you enter a name at **Streamer:** — any name at all, as long as that person has clips. This field applies **only to this page**: it decides whose clips you load and play here.

> The **Streamer** field has **nothing** to do with the two chat commands. With those, the viewer writes the name after the command in the Twitch chat themselves.

- **Clip Cmd** — the command for a **random** clip. If somebody types `!clip CaptainApolloo` into chat, a random clip from that channel plays.
- **Max Clips** — how many clips it picks from. If it says `20` there, ASTO-BOT rolls the dice among the **20 newest** clips.
- **Last Clip Cmd** — the command for the **newest** clip: `!lastclip CaptainApolloo`.

Both commands can be renamed at any time. `!clip` and `!lastclip` are only the defaults, not a rule.

A clip command can fire an **event** as well — to announce the clip in chat, for instance. You link the command to an event in the **Command** section like any other; the clip still plays. The event fires once the clip is **known**: only then do `%ClipStreamer%`, `%ClipTitle%`, `%ClipGame%` and the other `%Clip…%` variables describe the clip that is starting rather than the one before it. If no clip is found the event still runs — with `%ClipFound%` set to `false`, which is what you build a fallback announcement on.

8.2 Loading and picking clips

Load Clip List fetches the clips of the streamer entered above — practically all of them, sorted by date. On channels with many clips this takes a moment, reckon with 5 to 20 seconds.

- A **click** on a clip **selects** it — it shows up on the left under **Selected Clip**.
- A **double-click** — or the **Use Clip** button — plays it in OBS.
- A click on the large **thumbnail** opens the clip in the browser.

Above the list sits a **search field**. It filters the clips already loaded by title, date or creator — it does not fetch anything new. The top right of the card shows how many clips are currently listed, and each row shows the length of the clip on the right. While **Load Clip List** is running, a bar below the list shows how far ASTO-BOT has got.

> Twitch does not report a total number of clips. The bar therefore does not show "347 of 594" — it shows how far back towards 2016 ASTO-BOT has searched, together with the number of clips found so far.

Selected Clip and Current Clip

These two blocks look alike but mean different things:

- **Selected Clip** — the clip you **clicked** in the list. Changes with every click.
- **Current Clip** — the clip that was **played** last. Streamer, game, creator, date and title, below it the **Clip URL** with a **Copy** button.

> **Current Clip** is more than a display: these details live in the files in the **Twitch Daten** folder and feed `%ClipTitle%` and the other clip variables in events and overlays — and `!lastclip` plays exactly this clip. That is why merely clicking a row in the list does **not** change it.

- **Open in Browser** — plays the clip in your default browser.
- **Random Clip** — plays a random clip.
- **Refresh Info** — reloads the information in case that did not happen by itself.

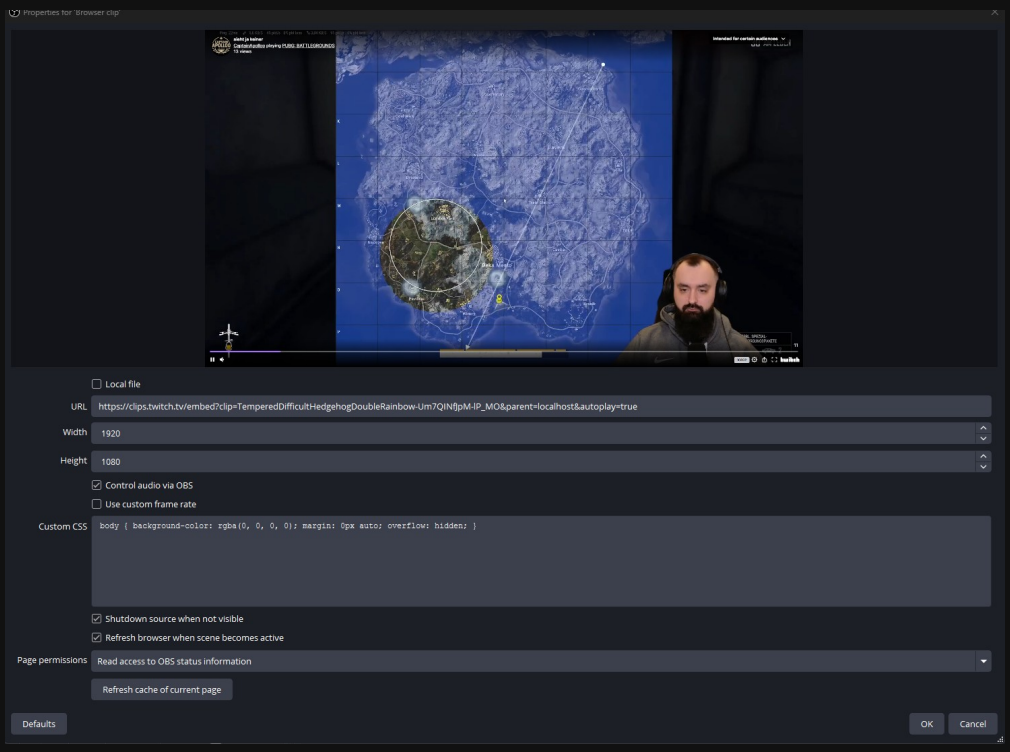
8.3 Clip Playback — setting up the OBS source

For a clip to be seen on stream, ASTO-BOT needs a source in OBS. At the bottom left you first pick the **scene** the source sits in, and then one of two options:

- a **Browser Source** — plays the clip through the Twitch player,
- or a **Media Source** — plays the MP4 file directly, with sound.

> If you set **both**, the **Media Source wins**. If you do not want that, set the other source to **– none –**.

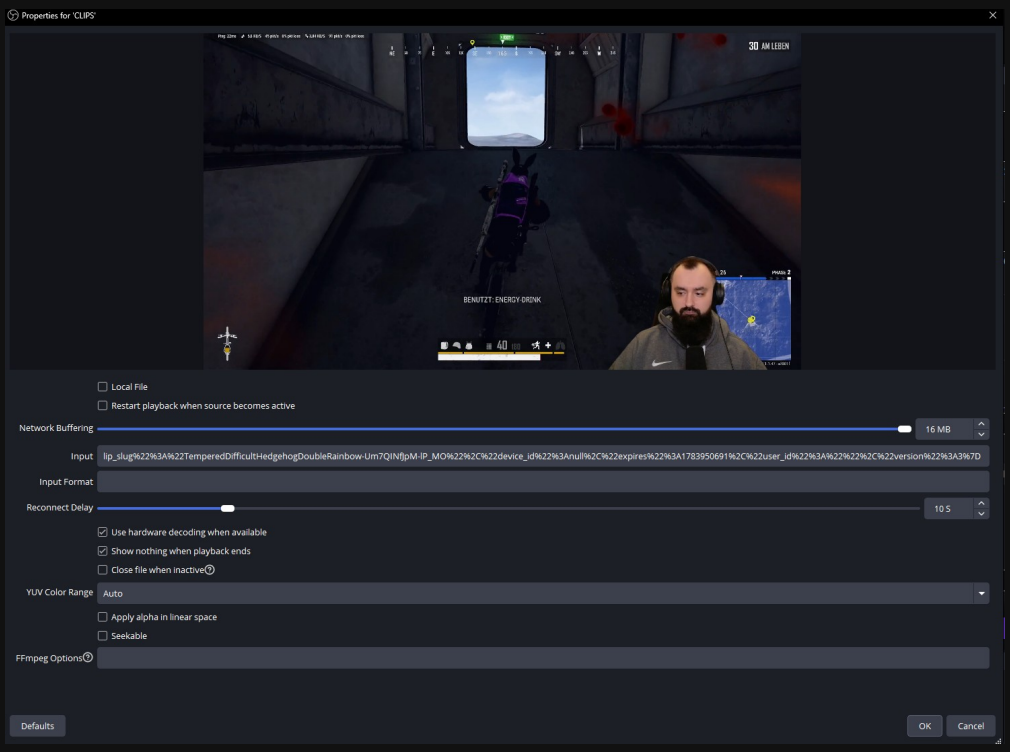
The browser source in OBS



The properties of the browser source in OBS

- **Local file — off.**
- **Control audio via OBS — on.**
- **Shutdown source when not visible — on.**
- **Refresh browser when scene becomes active — on.**

The media source in OBS



The properties of the media source in OBS

- **Local File — off.**

- **Network Buffering** — at least **4 MB**.
- **Use hardware decoding when available** — **on**.
- **Show nothing when playback ends** — **on**.
- **YUV Color Range** — **Auto**.

> The URL, or the input, is entered by **ASTO-BOT itself**. You do not have to type anything there — the screenshots only show something because a clip happens to be playing.

Live Controls

Under **Live Controls (Media Source)** you drive the running clip by hand: **Play**, **Pause**, **Stop** and **Reset** (plays the last clip again). Next to it sits the **volume slider**. The top right of the card names the media source that is being driven.

Below that a **progress bar** runs along with the title and the time. It covers **every** clip — whether you start it here, whether `!clip` triggers it, a channel point reward or the playlist.

Is OBS reachable?

The header of the **Clip Playback (OBS)** card carries a dot: **green** means connected, **red** means not reachable, **grey** means not checked yet. ASTO-BOT asks once a minute and on every real call to OBS.

8.4 Overlay Action — everything around the clip

The scene in which the clip is **played** does not have to be the scene your audience **sees**. That is exactly what the **Overlay Action** is for: when a clip starts, it shows the right thing — and hides it again afterwards.

- **Action** — what should happen: **Show Source**, **Switch Scene** or **nothing**.
- **Scene** and **Source** — what exactly is shown or switched to.

Timing

- **Show delay** — how long **before** the start of the clip it is shown.
- **Hide delay** — how long is waited **after** the end of the clip before it is hidden again.
- **Play delay (fade-in)** — how long is waited after showing it until **Play** is pressed automatically.
- For **Show delay** and **Hide delay** at least **1000 ms** is possible.

> 💡 Set **Show delay** and **Hide delay** to the same values as the **show and hide transition** of your source in OBS. Then the fade-in fits the start of the clip and nothing stutters against each other.

Load OBS loads all scenes and sources from OBS, **Save OBS** stores your settings.

8.5 The Clip Playlist

On the right edge sits the **Clip Playlist** — a rolling programme of other people's clips, ideal for the break, the start of the evening or simply as a backdrop.

- Enter any number of **streamer names** below. The green button adds one, the red ✕ removes it again.
- **Start Playlist** starts the programme, **Stop** ends it.
- ASTO-BOT then draws a **random name** from the list and from that a **random clip** out of the **100 newest** of that channel.
- While one clip is playing, ASTO-BOT already picks the next one — so there is no gap in between.

> The playlist runs **endlessly** until you press **Stop**. It does not stop by itself.

> 💡 A nice gesture for the community: enter the names of the people who hang around in your channel and let the playlist run during the break. Then your chat sees the clips of its own people.

8.6 Quick Actions — saving and sorting clips

The **Quick Actions** card always works on **one** clip. Which one is named at the top of the card: if a clip is selected in the list, that one counts — otherwise the one under **Current Clip**. That lets you click through the list and save clips away without having to play each one first.

Save as MP4

Saves the clip as a video file on your drive.

- The target folder starts out as **Downloaded Clips** next to ASTO-BOT.exe. The **folder button** picks any other one — the choice is remembered.
- Next to it you always see how many files are in the folder and how much space they take.
- While downloading you see the progress in percent.
- A clip already sitting in the folder is **not** downloaded again.

> Only download clips from **other** channels if you have the owner's permission. A clip being publicly playable does not make it yours.

> 💡 A cancelled download leaves no half file behind: while the clip is still loading the file is named `.part` and only gets its real name once it is complete.

Add this clip to a clip list

The dropdown lists all existing **clip lists**. **+ Add** puts the current clip into the selected one.

- The 📋 button opens the **Clip List Manager**, where you create lists, rename clips or delete them.
- A ✔ **Use** in the manager takes that list over — the dropdown in Quick Actions jumps to it.
- If the clip is already in the list, ASTO-BOT says so and does not add it twice.

> These are the same lists the **Clip List Player** plays in the events. Whatever you sort in here is available there straight away.

8.7 Activity Log

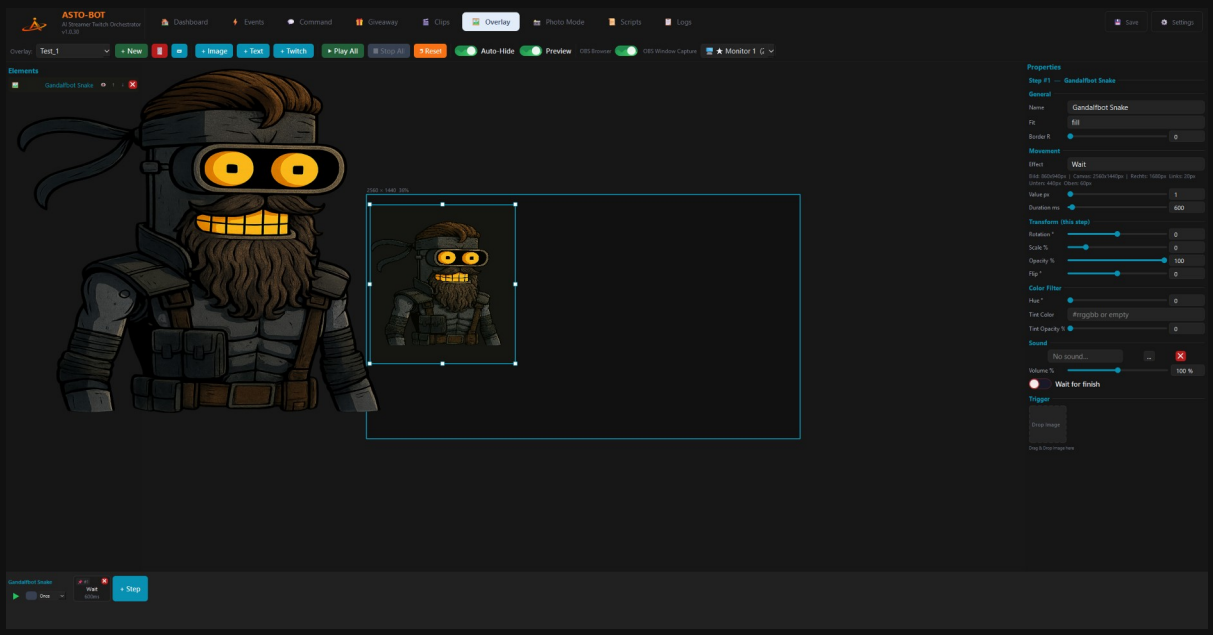
At the bottom right the **Activity Log** runs along — the latest messages about clips, OBS and the playlist, **newest line on top**. It also shows clips triggered by an **event**, not just the ones from this page.

- **Debug: off / on** adds the technical lines. Useful when hunting a fault, too much for everyday use.
- **Clear** empties the **view** only. The log file itself is kept in full.

> 💡 When a clip does not show up, the reason is usually already here — a missing source, an OBS that cannot be reached, or an MP4 that could not be resolved.

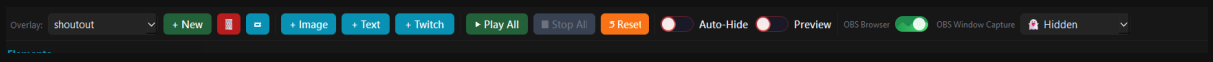
9. Overlay

In the Overlay area you build the things your stream shows: an image flying in, a text with the name of the donor, the profile picture of the viewer, plus a sound. An overlay is later started from an event — with the action **Play Overlay** (chapter 5.6).



The overlay editor: the elements on the left, the canvas in the middle, the properties on the right, the steps at the bottom

9.1 The header bar



The header bar of the overlay editor

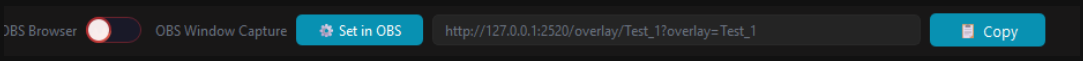
- **Overlay:** — the dropdown picks which overlay you are editing.
- **+ New** creates a new one, the red **trash can** deletes the selected one, the button next to it **renames** it.
- **+ Image** adds an image or GIF, **+ Text** a text field, **+ Twitch** the profile picture of a viewer.
- **Play All** plays all the animations of the overlay, **Stop All** stops them, **Reset** puts all elements back to their starting position.
- **Auto-Hide** — the overlay hides itself as soon as it has run through.
- **Preview** — shows the overlay so you can look at it.
- 🔒 **Lock** — locks the elements against the mouse. Selecting and typing still work; only moving, resizing and rotating are dead. It is per overlay, survives a restart and follows the overlay when it is renamed.

> 💡 You can also simply pull images and GIFs into the canvas with **drag & drop**. That is the fastest way.

9.2 How the overlay gets into OBS

On the right of the header bar you decide **how** OBS captures the overlay. The switch stands between **OBS Browser** and **OBS Window Capture** — and it does so **per overlay**: every overlay has its own setting.

OBS Browser



In browser mode ASTO-BOT hands you the URL straight away

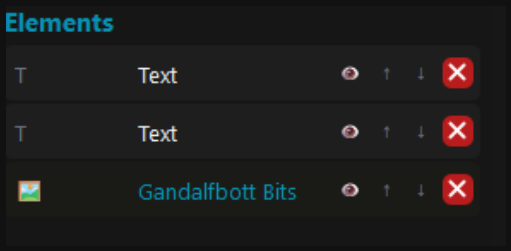
In browser mode the button **Set in OBS** appears: it enters the URL into the browser source in OBS all by itself. Next to it the same URL is shown for reference, with a **Copy** button in case you want to paste it by hand.

OBS Window Capture

In window capture mode ASTO-BOT shows the overlay as a window of its own. The dropdown next to it decides **which monitor** this window appears on — or you set it to **Hidden**, then it stays invisible and is only captured by OBS.

> 💡 Which mode is the right one depends on your setup. Setting up both sources in OBS is described in the chapter Settings → OBS.

9.3 Elements



The element list — eye, arrows, delete

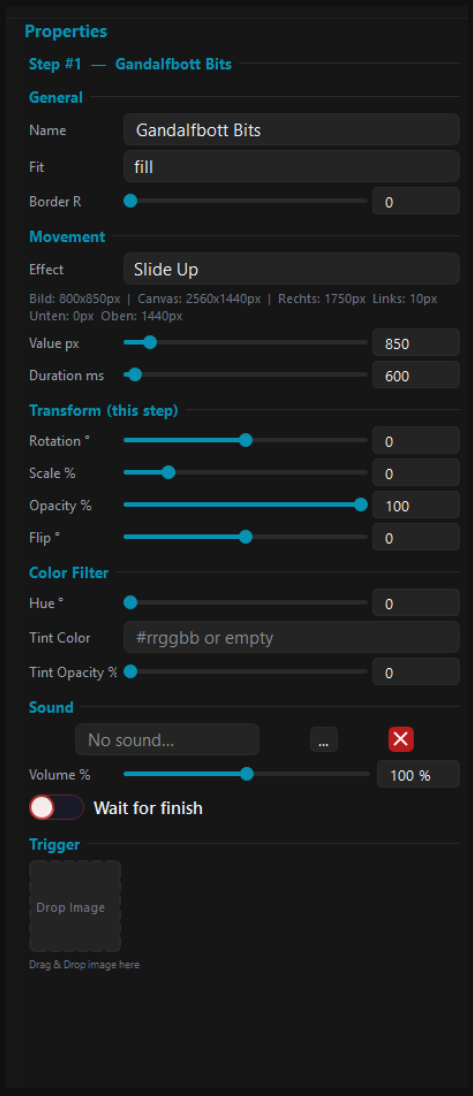
- The **eye** hides an element in the editor without deleting it.
- The **arrows** ⬆ ⬇ sort the elements on top of each other.
- The red ✕ deletes the element.

The Twitch element

+ Twitch places a placeholder for the **profile picture of a viewer** into the canvas. When you insert it you first see only a default image — that is normal.

> For the viewer's picture to really appear on stream, the action **Get Target Info** must sit **before** the action **Play Overlay** in the event. Get Target Info fetches the profile picture and sends it to the overlay. Without that order, the default image stays.

9.4 The properties of an image

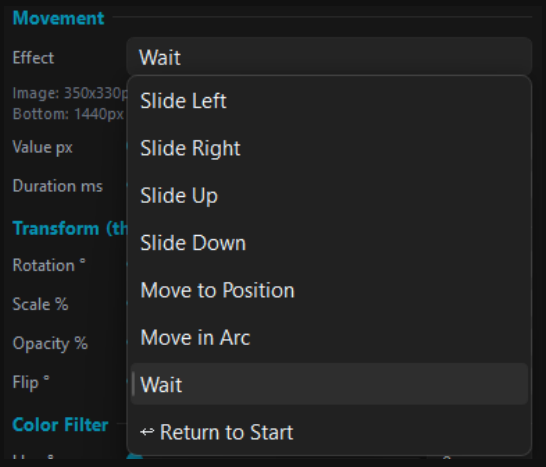


The properties of an image element

General

- **name** — what the element is called.
- **Fit** — how the image fits into its frame: `fill`, `contain`, `cover` or none. In practice you can usually leave it on **fill**, the difference hardly matters.
- **Border R** — rounds off the corners.

Movement — the motion



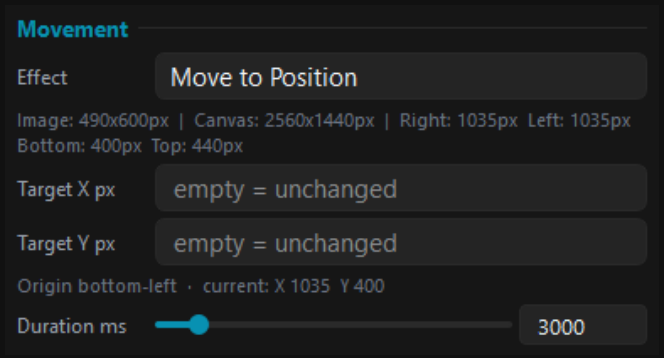
The movement effects in the dropdown

- **Slide Left**, **Slide Right**, **Slide Up**, **Slide Down** — the element travels in the respective direction.
- **Move to Position** — the element travels to a **fixed spot** on the canvas (see below).
- **Move in Arc** — the element travels along a **curve** instead of a straight line (see below).
- **Wait** — nothing happens, the element stays where it is. This is how you build pauses.
- **Return to Start** — the element jumps back to its starting point.
- **Value px** — how far it moves, **Duration ms** — how long the movement takes.

> The input fields change with the effect: with **Move to Position** and **Move in Arc** the **Value px** field disappears and the matching fields show up instead. Whatever you type there is kept — switch the effect back and your values are still there next time.

Move to Position — travel to a fixed spot

Instead of „how far" you enter „where to" here. That helps when an element should always end up at **the same place** — no matter where it currently is or what earlier steps did to it.



Move to Position — Target X and Target Y

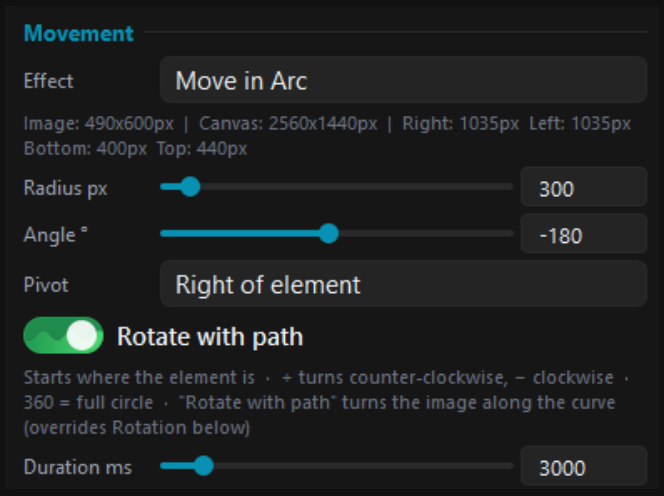
- **The origin sits in the bottom left** of the canvas: X counts to the **right**, Y counts **up**. You will find a cyan axis cross with two arrows down there so you never lose your bearings.
- The reference point on the element is its **bottom left corner** as well. So `X 0` and `Y 0` place the element exactly in the bottom left canvas corner.
- **Leaving a field empty** means: that axis stays as it is. Fill in only Y and the element moves **vertically** — fill in only X and it moves **horizontally**.
- **Negative values are allowed.** That is how you send an element off screen on purpose, for example `X -400` to make it leave the stream to the left.

- The small line below the fields shows the **current position** in the same system, so you can read off straight away which target value makes sense.

> 💡 Because X and Y change at the same time, filling in both fields produces a **diagonal movement**. The element then runs in a straight line to the target — not sideways first and upwards afterwards.

Move in Arc — travel along a curve

Here the element does not move in a straight line but along a **curve**. It starts right where it stands, runs the arc and stops at the end — from there you can simply carry on in the next step, with a straight move for instance.



Move in Arc — radius, angle, pivot and rotate with path

- **Radius px** — how far the pivot point sits away from the element. A small radius gives a tight curve, a large one a wide, gentle sweep.
- **Angle °** — how far around it goes. **Positive** turns **counter-clockwise**, **negative** turns **clockwise**. `90` is a quarter circle, `360` a full turn, `720` two turns.
- **Pivot** — which **side** of the element the pivot point sits on: to its right, left, above or below.
- **Rotate with path** — the element **turns along with the curve**, like a car going round a bend. Without the switch it keeps its orientation.

> Why the **Pivot** field is needed: radius and angle alone do not define the arc yet. Around any given point there are countless circles with the same radius — only the side the pivot sits on turns that into one clear path. Together with the sign of the angle you can reach every direction from there.

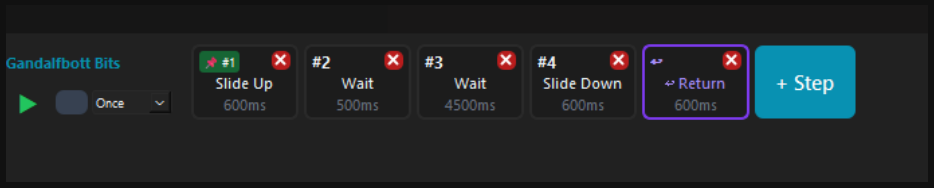
> 💡 **Rotate with path** does the maths for you: the rotation is derived from the arc automatically and runs evenly along with it. Careful — the switch **overrides** a fixed **Rotation** set in the same step. Leave it off if you want to set the rotation yourself.
>
> 💡 A floating effect works nicely with two arcs in a row: first `Angle 60` with pivot **above**, then `Angle -60` with pivot **below**. That gives a gentle wave instead of a rigid circle.
>
> 💡 Put **Return to Start** as the **last step** whenever you can. Then image, text and Twitch picture end up back at their starting point — and the overlay looks the same next time as it did the first time.

Transform, Color Filter and Sound

- **Transform (this step)** — applies **to this one step**: **Rotation**, **Scale %**, **Opacity %** and **Flip**.
- **Color Filter** — **Hue**, **Tint Color** (as hex) and **Tint Opacity**.
- **Sound** — every step can play its **own sound**, with a **Volume** slider.
- **Wait for finish** — the next step only starts once the sound has finished.

9.5 The step timeline

At the bottom sits the **timeline**. Every **step** is one effect with a duration, and the steps run one after another from left to right. **+ Step** adds another one.

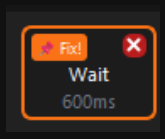


A timeline with five steps — Slide Up, two pauses, Slide Down, Return

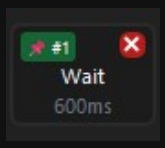
- The **green play button** on the left starts **only this one element** — handy while fine-tuning. Next to it sits the stop button.
- The dropdown beside it knows two values: **Once** (run through once) and **Loop** (repeat endlessly).

The pin — the most important little thing

At the **first step** sits a small **pin**. Its colour tells you whether the position of your element has been saved.



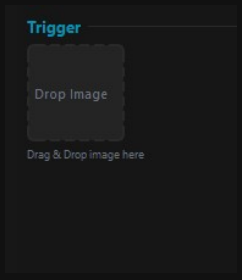
*Orange: the position has **not** been saved*



Green: the position has been saved

> Whenever you **move an element by hand** in the canvas, the pin turns **orange**. That means: the new position is **not** saved — and on the next playback the element shifts. Click the pin until it is **green**. Then the position is fixed.

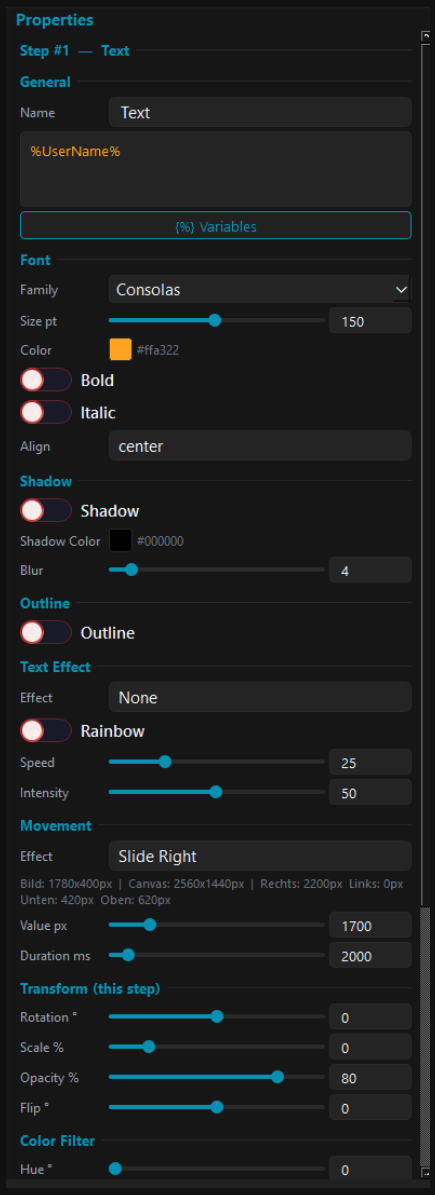
Several timelines one after another



The trigger area

At the very bottom of the properties sits the **Trigger** area. If you pull a **new image** into it with drag & drop, a **new timeline** appears below it, **starting from the step currently selected**. That way you chain any number of sequences together: image 1 comes in, and at a certain point of its movement image 2 starts — and from there the next one.

9.6 Text elements



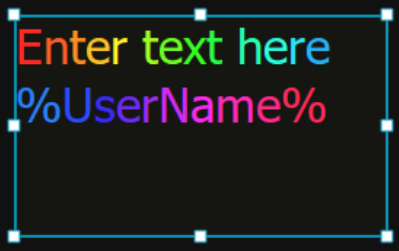
The properties of a text element

- At the top sits the **text field**. The button **{%} Variables** opens the variable overview.
- **Font** — **Family**, **Size pt**, **Color**, **Bold**, **Italic** and **Align**.
- **Shadow** — a shadow with colour and **Blur**. **Outline** — an outline around the text.
- **Movement**, **Transform** and **Color Filter** work just like they do for an image — so text can move as well.

Variables in the text

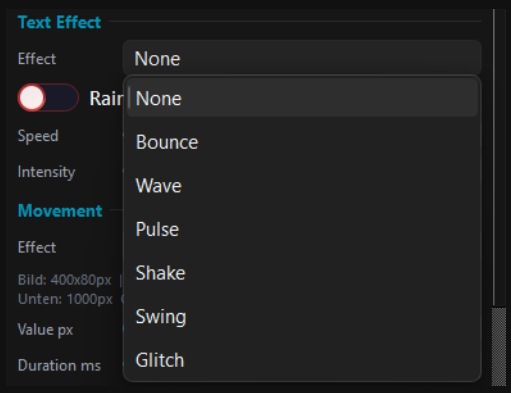
Variables work in the text: `%UserName%`, `%CheerAmount%` and all the others. In the canvas you still see the variable itself — that is on purpose. It is only resolved where it matters: in the window and in the browser source, that is, on stream.

Colours and text effects



Individual words can be coloured separately

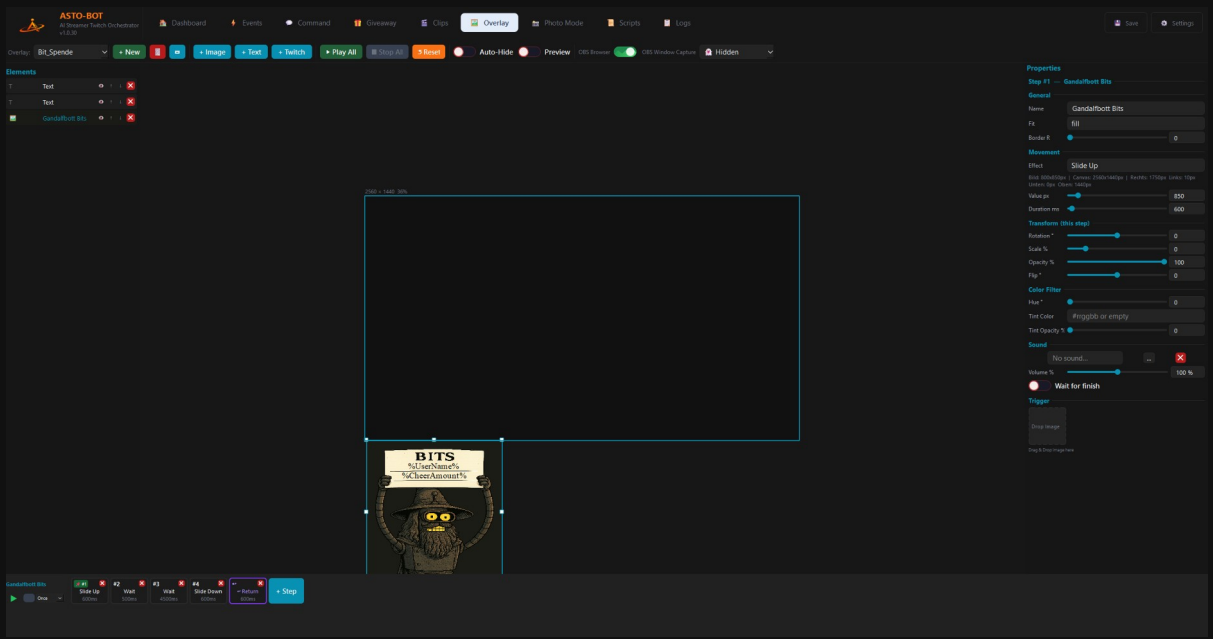
You can colour not only the whole text but also **individual words**: select the word, pick a colour — done.



The text effects

- **Text Effect** — **None**, **Bounce**, **Wave**, **Pulse**, **Shake**, **Swing** or **Glitch**.
- **Rainbow** — with the toggle on, the text runs through changing colours. **Speed** and **Intensity** control how fast and how strong.

9.7 An overlay in action



The overlay “Bit_Spende”: two texts, one image, five steps

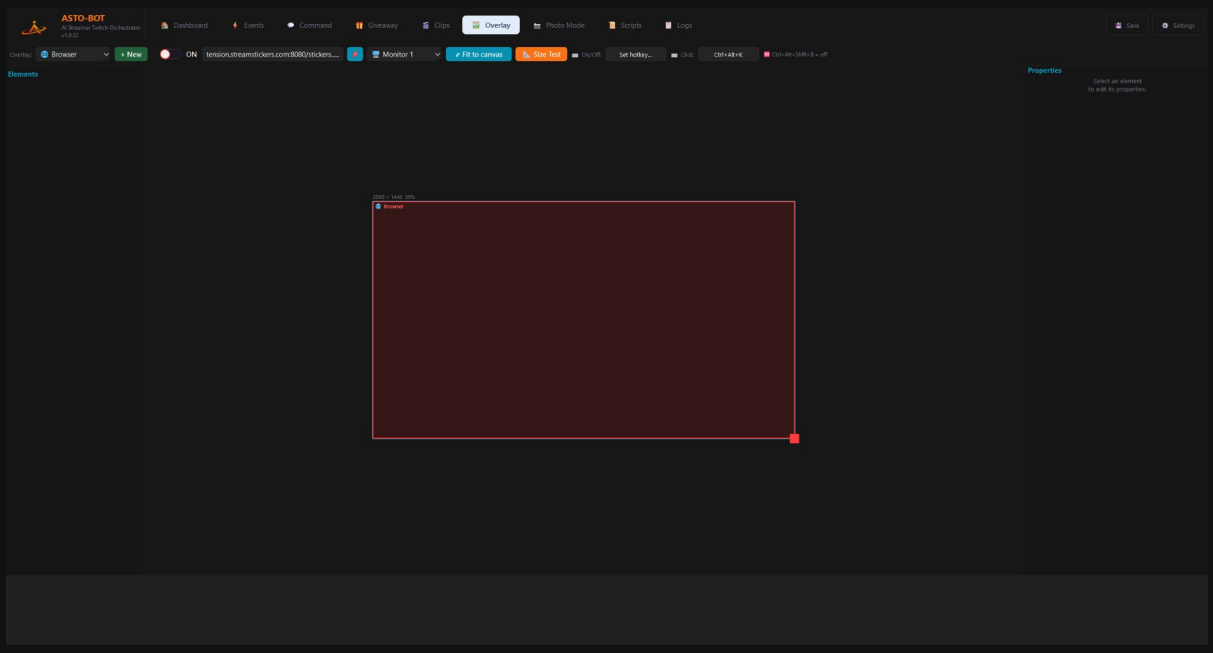
The example shows how the parts come together: an image with a sign, on it two text elements with `%UserName%` and `%CheerAmount%`. The timeline for it is five steps long — **Slide Up** (600 ms) drives the sign into the picture, two **Wait** steps let it stand there (five

seconds in total), **Slide Down** drives it out again, and **Return** puts everything back to the beginning.

In the event only a single action hangs on it: **Play Overlay** with this overlay — and, if a Twitch picture is involved, a **Get Target Info** before it.

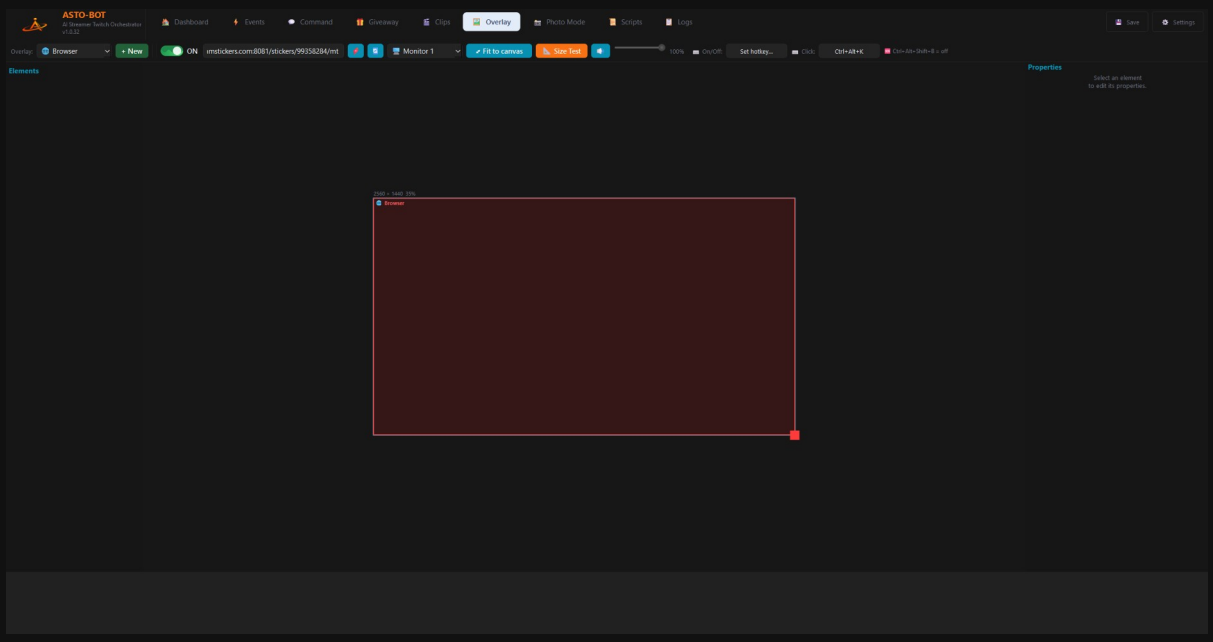
9.8 The Browser Overlay

The **Browser Overlay** is a special, built-in overlay called **Browser** — you find it at the top of the dropdown. Unlike the other overlays you do **not** assemble images or text here: it loads a **web page via its URL** and shows it — StreamStickers, for example. The trick: the page runs in its **own window directly on your monitor**, not only in OBS.



The Browser Overlay in the editor — the red rectangle is the area the web page takes up

The header bar



The header bar of the Browser Overlay

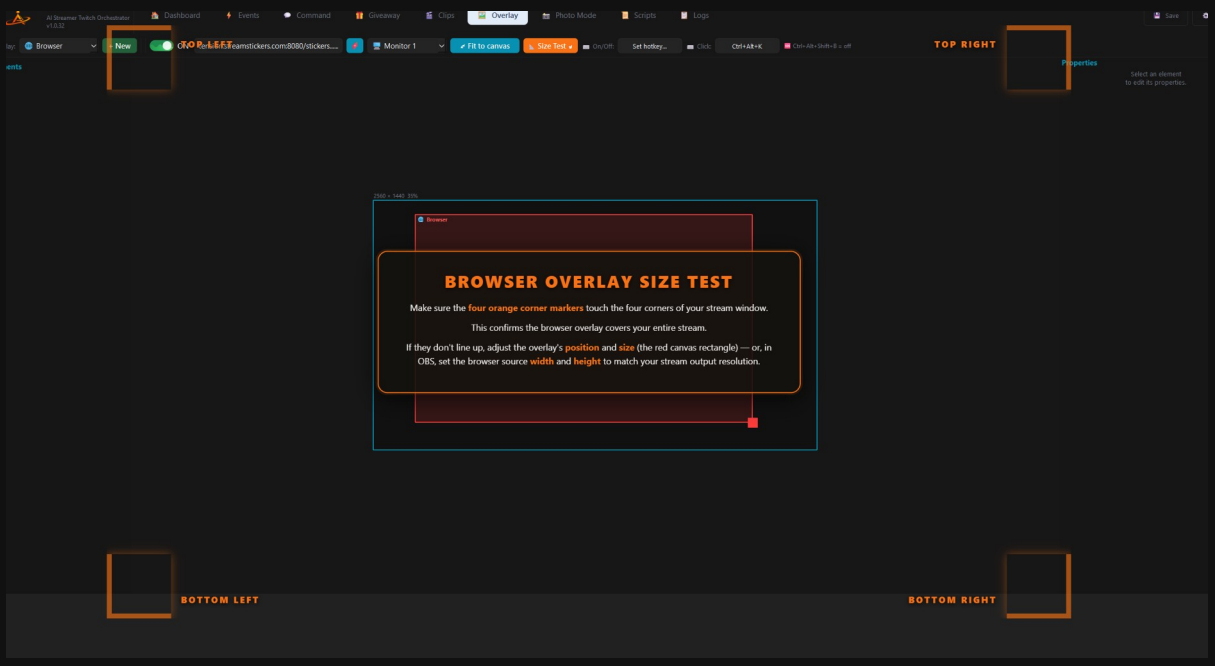
From left to right:

- **ON / OFF** — switches the browser window on and off.
- **URL field** — here you enter the address to be shown.
- **URL list** (the blue button with the red icon) — opens a list where you can store **several URLs**. Only **one** is ever active at a time, though, never more.
- **Refresh** — reloads the current page.
- **Monitor** — which screen the window appears on. If you pick **Hidden**, it stays invisible and is only captured by OBS.
- **Fit to canvas** — automatically lays the window over the whole screen. Alternatively you drag it by hand to the size and position you need.
- **Size Test** — shows an alignment helper (see below).
- **Mute** and the **volume slider** — mute the page or adjust its volume.
- **On/Off hotkey** and **Click hotkey** — keyboard shortcuts for switching on/off and for enabling clicks (more on that in a moment).

> The window is **click-through** by default — you click straight through it as if it were not there. With the **Click hotkey** you make it clickable for a moment to operate something on the page.
> **Emergency off:** if you load a page that is **not transparent** (e.g. YouTube) and it lands right over the ASTO-BOT window, `**Ctrl+Alt+Shift+B**` gets you back out immediately — it hard-disables the overlay.

Size Test — line it up cleanly

Pressing **Size Test** shows a display in the browser window with four orange corner markers and a hint text. Move the position and size of the red rectangle (or, in OBS, the width and height of the source) so that the four markers sit **exactly in the corners of your stream picture**. Then the overlay covers your stream with no gaps.

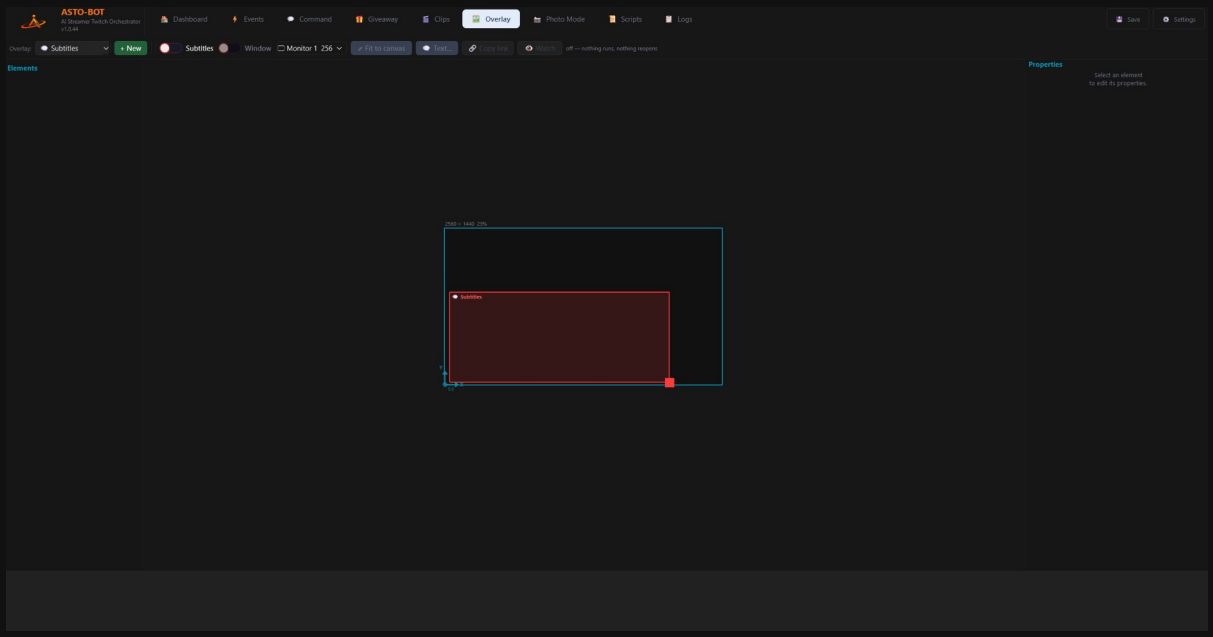


The Size Test with the four corner markers and the hint text

> The Browser Overlay runs in its **own window** and therefore needs its **own OBS source**. How to set it up as a window capture is described in Chapter 4.3.4.

9.9 Subtitle Overlay

The subtitle overlay is called 💬 **Subtitles** and, like the Browser Overlay, sits permanently in the dropdown at the top left. This is where the lines go that the Live Translator sends through its **To Subtitles** switch.



The subtitle overlay toolbar with the red text area on the canvas

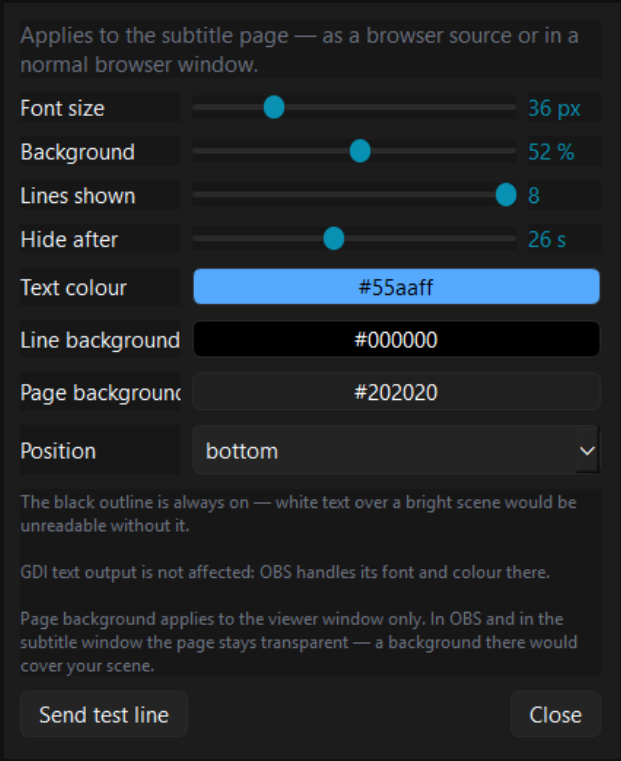
The **red rectangle** on the canvas is the area the text appears in. Move or resize it and the text follows immediately — in the window on your monitor as well as in a browser source in OBS. Its position is stored as a percentage, so it works the same on 1080p, 1440p and 4K.

The toolbar

- **Subtitles** — arms the overlay.
- **Window** — opens the subtitle window on your screen. It is transparent and click-through, so you can keep working normally.
- **Monitor** — which screen the window sits on.
- **Fit to canvas** — resets the red area to the full surface.
- **Text…** — font, colours and position, see below.
- **Copy link** — the address for a **browser source in OBS**. That page is transparent: it shows the text and nothing else.
- **Watch** — opens the subtitles in your normal browser. This variant paints a background, because a browser renders transparency as white.

> **Watch** is the route for anyone with neither OBS nor a Twitch account — during a video call, say, or when you want subtitles on a video you are watching.

Text… — the appearance



The settings for font, colours and position of the subtitles

- **Font size** — the text size. It scales by itself: 36 px on your canvas looks the same on a 4K source.
- **Background** — how opaque the box behind each line is.
- **Lines shown** — how many lines stay on screen at once.
- **Hide after** — how many seconds before a line disappears. At **0** it stays.
- **Text colour** — the font colour.
- **Line background** — the colour of the box behind each line.
- **Page background** — the background for the **Watch** viewer window **only**. In OBS and in the subtitle window the page stays transparent; a background there would cover your scene.
- **Position** — top, middle or bottom within the red area.

Send test line sends a sample line without you having to speak. Use it to judge size and colour against your real scene — and to see at once whether the connection is up at all.

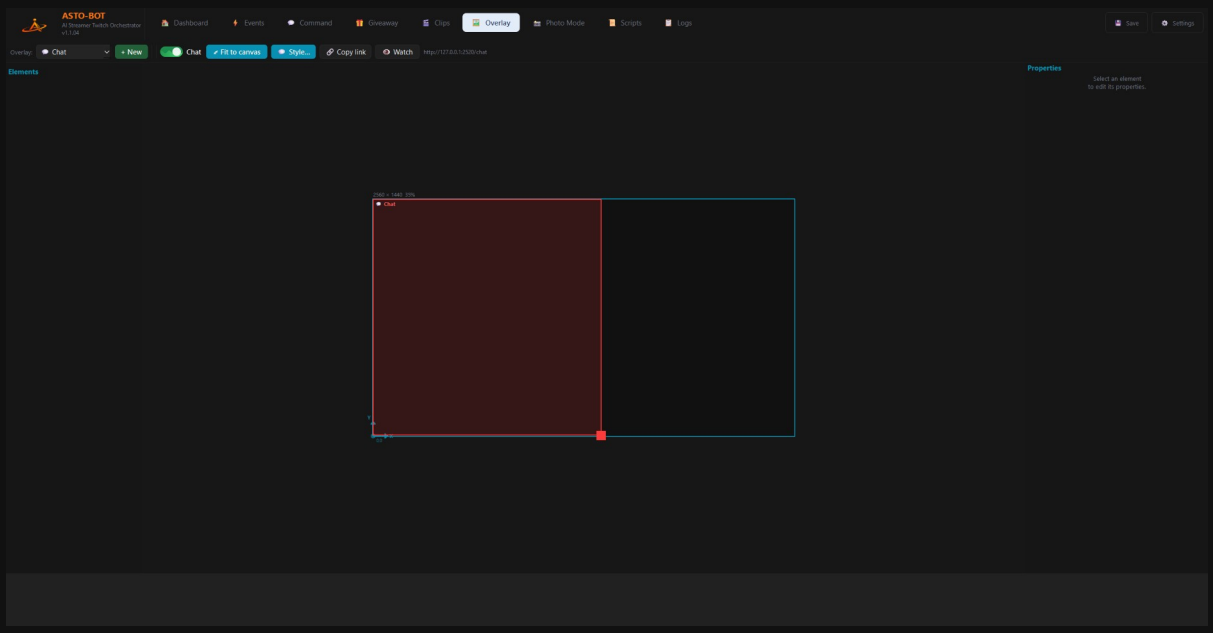
> 💡 The black outline around the text is always on. White text over a bright scene would be unreadable without it.

> These settings do **not** apply to the **To GDI Text** output. There OBS decides font and colour.

9.10 Chat Overlay

The chat overlay is called 💬 **Chat** and, like the Browser Overlay and the subtitles, sits permanently in the dropdown at the top left — it cannot be deleted. It puts your **Twitch chat straight into the stream**: with the real colours of the viewer names, their badges and their emotes. And not only the Twitch emotes, but those from **7TV**, **BTTV** and **FrankerFaceZ** as well.

So you need no outside service and no second browser page. ASTO-BOT serves the chat page itself — it is already attached to your chat, and it is the only thing that knows which account is your bot and what was a command. An external overlay can only guess at that.



The chat overlay in the Overlay area: the toolbar and the red area on the canvas

The **red rectangle** on the canvas is the area the chat runs in — exactly as with the subtitles. Move or resize it and the chat follows immediately. Its position is stored as a percentage, so it sits in the same place on 1080p, 1440p and 4K.

> Chat Overlay, Subtitle Overlay and Browser Overlay are **independent of one another**. Each has its own rectangle, its own settings and its own page. You can run all three at the same time.

The toolbar

- **Chat** — the master switch. Off means: no more messages are sent **and** the lines still on screen are cleared. Otherwise the chat would stay up after switching it off, which rather defeats the point.
- ⛶ **Fit to canvas** — resets the red area to the full surface.
- 💬 **Style…** — font, colours, designs and filters, see below.
- ☐ **Show area** — draws the chat area as a dashed frame with a label, in OBS as well. That is how you place it without waiting for someone to write. Press again to turn it off — the button then reads ■ **Hide area**, so you always know where you stand.
- 🔗 **Copy link** — copies the address of the chat page. That is exactly what you need in OBS in a moment.
- 👁 **Watch** — opens the chat in your normal browser. This variant paints a background, because a browser renders transparency as white.
- To the right of it stands the current address — or *off* when the switch is off.

How the chat gets into OBS

The chat overlay is a **web page** that ASTO-BOT serves itself — not a window and not a file on your hard drive. In OBS you set up a **browser source** for it and enter the address:

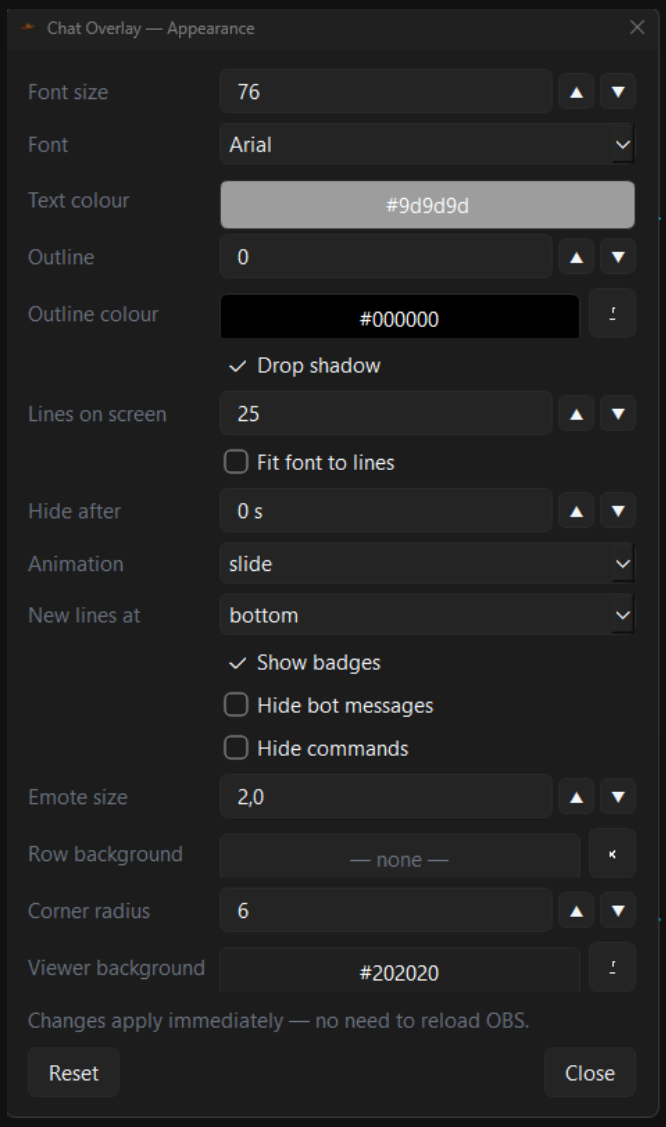
- In ASTO-BOT press 🔗 **Copy link**. Your clipboard now holds `http://127.0.0.1:2520/chat`.
- In OBS click the **+** under **Sources** and choose **Browser**. Give it a name, *ASTO Chat* for example.
- Paste the copied address into the **URL** field.
- Set **Width** and **Height** to the resolution of your scene — usually `1920` × `1080`. The page converts the font size by itself, there is nothing to adjust.
- Confirm with **OK**. The moment someone types in chat, it is on screen.

> **Important:** the **Local file** tick box must stay **off**. This is not an HTML file you go and pick from somewhere, it is an address — and it belongs in the **URL** field and nowhere else. With *Local file* on, OBS looks for a file on your hard drive, finds nothing and shows an empty source.

> 💡 Size and position in the picture are **not** set in OBS but with the red rectangle in ASTO-BOT. Just leave the source in OBS at full size — then the two always match.

> ASTO-BOT has to be running for the page to be served. With ASTO-BOT closed the browser source stays empty. On shutdown ASTO-BOT clears the chat out of the picture by itself, so nothing is left hanging there.

Style… — the appearance



The settings for font, colours, number of lines and filters

Every change takes effect **immediately** — you need neither to save nor to reload the source in OBS. Simply put ASTO-BOT and OBS side by side and turn the dials until it fits.

- **Design** — the overall look of the chat. Nine designs to choose from, from **Classic** — the look it always had — through bubbles, a glowing edge, paper and an instant photo. What makes each one is written underneath the picker. Exactly **one** is active at a time; switching reloads the browser source by itself, so you do not have to touch OBS.
- **Font size** — the text size, chosen against the width of your canvas. The page converts it, so that 40 px on a 2560 canvas still looks like 40 px when the source renders in 4K.

- **Font** — the font family. The dropdown shows every font in its own typeface, so you see what you are getting before you pick it. Empty means the built-in default stack.
- **Text colour** — the colour of the messages. The names always keep their real Twitch colour.
- **Outline** — the outline around the text, in pixels. **0** turns it off.
- **Outline colour** — the colour of that outline. Black is the safe choice on almost any scene.
- **Drop shadow** — a soft shadow under the text, on top of the outline.
- **Limit by** — what decides when an old message disappears. **Fit to area** keeps as many as fit inside the red area: the oldest goes the moment the next one would run over the top edge. There is nothing to work out — the rectangle you drag **is** the limit. **Messages on screen** uses a fixed number instead.
- **Messages on screen** — the fixed number for that second mode. Older messages drop off the top. Be aware that 25 short messages fill a third of the box while 25 long ones run out of it — same setting, very different picture. That is exactly why **Fit to area** is the default.
- **Fit font to lines** — turns the relationship around: instead of the font size deciding how many lines fit in the box, the number of lines decides the font size. 25 lines then really means 25 visible lines. Belongs to **Messages on screen** and is greyed out under **Fit to area**, where the height of the rectangle decides.
- **Hide after** — how many seconds before a line fades out. **0** means never.
- **Animation** — how new lines arrive: *slide*, *fade*, *zoom* or not at all.
- **Slide from** — which side lines come in from and go back out to: left or right. Only affects sideways movement.
- **New lines at** — whether new lines come in at the bottom (like the real Twitch chat) or at the top.
- **Show badges** — shows mod, sub and VIP badges in front of the name.
- **Emote size** — the size of the emotes as a multiple of the line height. **1.5** is about the size in the Twitch chat.
- **Row background** — a box behind every line. Usually better left empty; use the outline instead. ✕ gets rid of it again.
- **Corner radius** — how round the corners of that box are.
- **Viewer background** — the background for viewing through 👁 **Watch only**. In OBS the page stays transparent; a background there would cover your scene.
- **Reset** — puts everything back to the defaults.

> Your colours and sizes are kept when you switch, so trying a design out costs you nothing. Whatever a design sets for **itself** is **greyed out** in the dialog rather than overwritten: a Terminal without fixed-width type would no longer be a Terminal. Switch back and your own values are there again straight away.

> 💡 Only seeing a few lines in OBS although you set 25? Then the font is too large for the red area — the other lines are there, they just sit outside it. That is precisely what **Fit to area** is for: nothing can run out of the box there, because it measures instead of counting. If you stay on **Messages on screen**, the cures are to drag the rectangle taller, make the font smaller, or switch on **Fit font to lines**.

Your own designs

The nine designs that ship with ASTO-BOT are not the end of it. Below the list are three buttons for adding your own — and, worth more than any built-in editor: they let designs **travel between users**. One person builds one, drops it in the Discord, everyone else pulls it in.

- **Export current** — writes the design selected above out as a `.css` file. This is the way I would recommend: hardly anybody gets far starting from an empty file, but from Polaroid with three colours changed, everybody does.
- **Import design…** — loads such a file. You can also simply **drag the file onto this window**, which is the same route.
- **Remove design** — deletes an imported design again. Your own only; the built-in ones are protected and the button is greyed out for them.

Imported designs land in `Overlays/ChatDesigns/` and appear in the list at once, marked **(own)**. The header of the file is CSS comments, so it stays a valid CSS file and can be edited in any editor with syntax colouring:

```
/* @label  Retro Terminal    */
/* @info   Green on black    */
/* @owns   font, color       */
/* @leave  300               */

.line { color: #0f0; }
```

@owns names the settings the design decides for itself — those are greyed out in the dialog, exactly as with the built-in ones. **@leave** is the fade-out time in milliseconds.

> **Someone else's CSS is cleaned before it is served.** A design file is written into your overlay page, and it comes from the Discord, from a viewer, from anywhere — the sender need not even mean any harm. Removed are: anything that could break out of the stylesheet and put its own JavaScript into your overlay; loading further files from the internet; and image addresses pointing at other people's servers — those would otherwise see your IP address every time your stream starts. Images embedded in the file itself stay allowed. What was removed is written to the log. Your file on disk is left untouched, so you can keep editing it.
>
> If a design file goes missing later — deleted, moved, renamed — the chat falls back to **Classic** and writes a line to the log. It never leaves a black rectangle in your stream.

What does not make it on screen

- **Hide bot messages** — hides everything your **bot account** and the entries on your **ignore list** post. Handy when you do not want your automatic announcements showing up twice.
- **Hide commands** — hides every message starting with `!`. So `!timer` or `!join` never appear at all — your bot's answer to them still does.

> **Announcements** from the chat (`/announce`) appear with a coloured bar down the left-hand side, in the same colour you picked when sending them. Twitch delivers them through a different route than ordinary messages, which is why they are drawn differently too.

> 💡 You do not have to be at the machine to switch the chat on and off: there is a **websocket command** for it (chapter 4.6), and the ASTO-BOT **Stream Deck plugin** has a **Chat Overlay** key of its own. One press and the chat is on screen or gone again.

10. Photo Mode

Photo Mode literally brings your viewers into the picture: you start the mode, your Twitch profile picture appears in front of a background, the viewers join with a chat command — and then the shutter goes. The finished photo lands in your gallery.

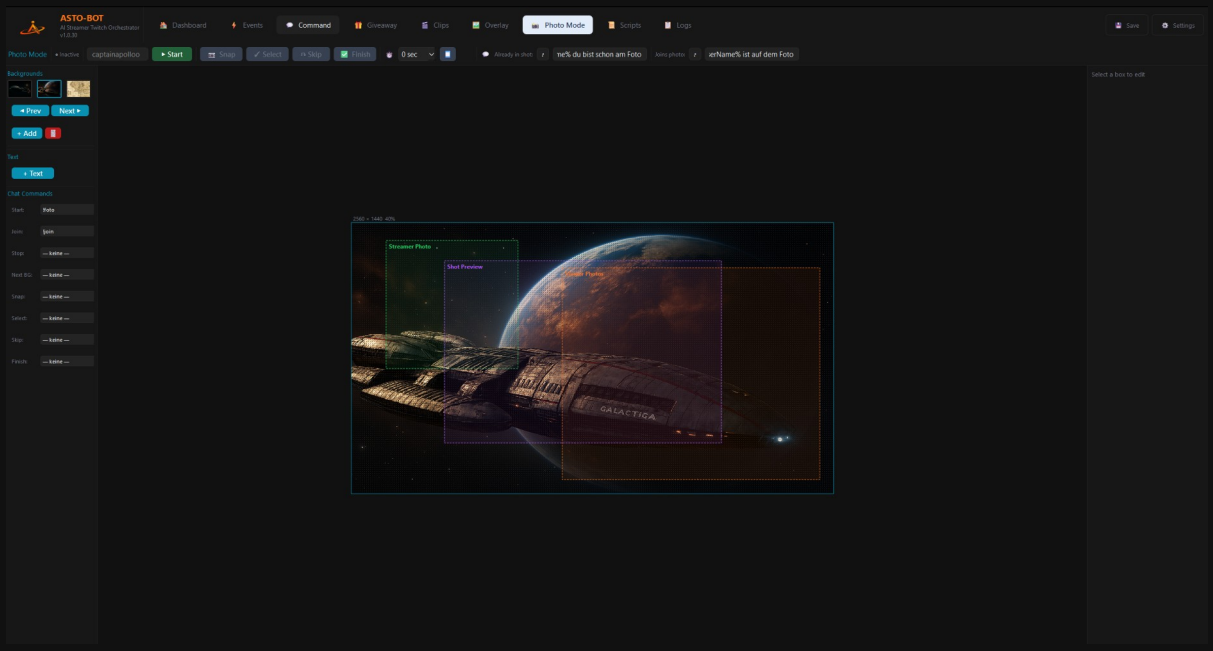


Photo Mode: backgrounds and commands on the left, the layout of the picture in the middle

> Not to be confused with the action **OBS Screenshot** from the Events chapter — that one only takes a still of your scene. Photo Mode is something else: a little photo session with your community.

10.1 The header bar

- **Active** — shows that the mode is running, together with the name it is about.
- **Snap** — takes the photo.
- **Select** — accepts the photo that was taken.
- **Skip** — discards it. A new one then has to be taken.
- **Finish** — ends Photo Mode.
- The **clock dropdown** is the self-timer: **0**, **3**, **5** or **10 seconds** after Snap is pressed.

The two chat texts

- **Joins photo** — the message that goes into chat when somebody joins the photo.
- **Already in shot** — the message for the case that somebody is already in the picture and wants to join again.

Variables are allowed in both texts — `%UserName%` practically always belongs in there.

10.2 Backgrounds and texts

- Under **Backgrounds** sit the background images. With **◀ Prev** and **Next ▶** you page through them.
- **+ Add** adds a new background, the **trash can** deletes the selected one.
- **+ Text** places a text field into the picture. The texts work exactly as in the overlay editor — the same effects, the same variables (chapter 9.6).

10.3 The three areas in the picture

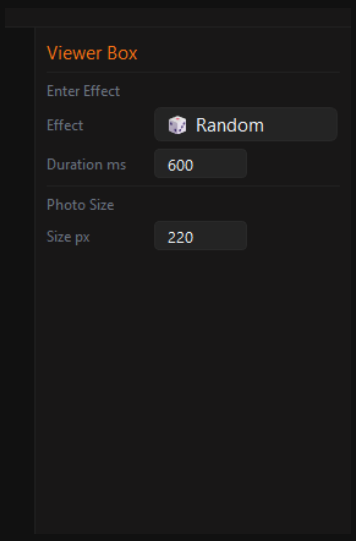
In the canvas you see three boxes with coloured borders. Each has its job, and each can be moved and resized freely.

Green — Streamer Photo

The streamer box

This is where **your** Twitch profile picture appears — the person the photo is about. Under **Enter Effect** you choose which direction it comes in from, **Duration ms** decides how long that takes.

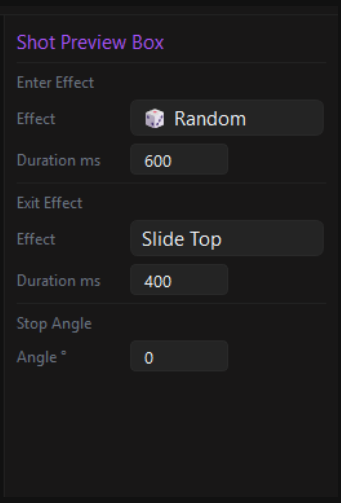
Orange — Viewer Photos



The viewer box

This is where the **Twitch profile pictures of the viewers** who have joined gather. Besides the enter effect there is also the **Photo Size** in pixels here — how large the individual pictures are shown.

Purple — Shot Preview



The shot preview box

The **photo that was taken** appears in this field. It sits in the **foreground**, in front of all the other pictures. This box is the only one that also has an **Exit Effect** — how the photo disappears again — and a **Stop Angle**, with which you can leave it slightly tilted, like a Polaroid tossed onto the table.

10.4 The chat commands

At the bottom left sits the complete list of commands. Every step of Photo Mode can be controlled from chat:

- **Start** — starts the mode, for example `!foto`. With a name after it: `!foto CaptainApolloo`.
- **Join** — viewers join with this command. **Without** anything else after it — so simply `!join` or `!join`.
- **Stop**, **Next BG**, **Snap**, **Select**, **Skip** and **Finish** — the same functions as the buttons at the top, just from chat.

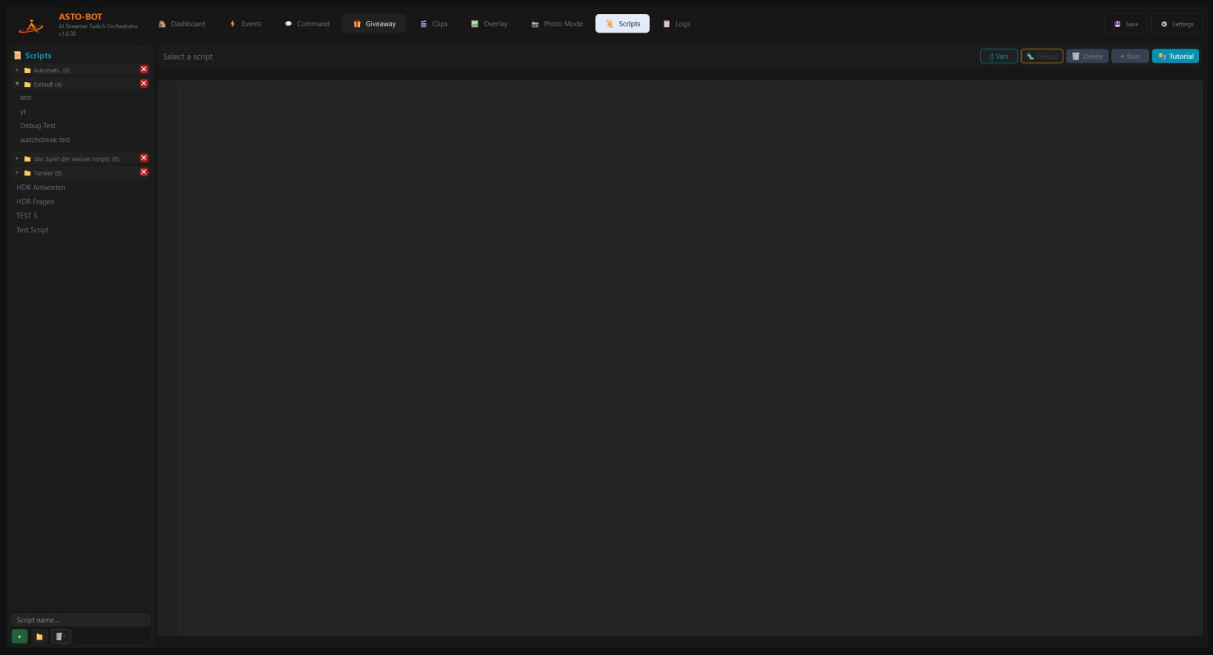
> 💡 You do not have to sit at the computer: the commands can also be fired by **websocket** — through a key on the Stream Deck, for example (chapter Settings → Websocket). Snap, Select and Finish on three keys, and you have both hands free for posing.

10.5 Where the photos end up

ASTO-BOT stores the finished photos in the folder **PhotoMode**, which sits next to the EXE. The card **Photo Gallery** on the Dashboard (Chapter 3) reaches into exactly this folder — the two belong together. Whatever you take here you can page through over there.

11. Scripts — ASTOSCRIPT

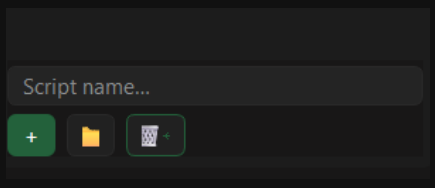
Everything you have seen so far can be clicked together. But at some point you reach a spot where that is no longer enough: you want a weather lookup in chat, you want to make your Elgato light blink, or you want to play a little game with your community. That is what **ASTOSCRIPT** is for — the scripting language built into ASTO-BOT.



The Scripts area: the script list on the left, the editor on the right

> 💡 The language itself is **not** explained in full in this manual — the **tutorial window** inside ASTO-BOT does that, and it does it better: it is always up to date, searchable and available in German and English. This chapter shows you everything **around** it: how you create scripts, test them, find errors and start them from events.

11.1 Creating and organising scripts

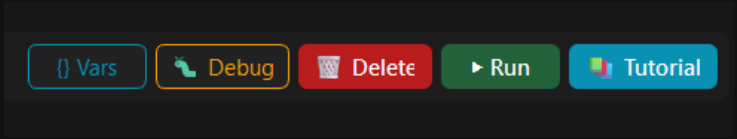


New scripts and folders are born in the bottom left

- Type a name into the **Script name...** field at the bottom and press the green **+** — your new script is done.
- The **folder button** next to it creates a new folder. You drag scripts into the folder they belong in with **drag & drop**.
- A **right-click** on a script in the list offers **rename** and **copy**.

> 💡 The **trash can** next to the folder button is not a delete button, it is the opposite: it **restores deleted scripts**. So if a script has gone missing on you — this is the first place to look.

11.2 The buttons in the top right



Vars, Debug, Delete, Run and Tutorial

- **Vars** — opens the same Variables window you already know from the events. A click copies.
- **Debug** — starts debug mode (section 11.5).
- **Delete** — deletes the script.
- **Run** — runs it.
- **Tutorial** — opens the tutorial and reference work (section 11.6).

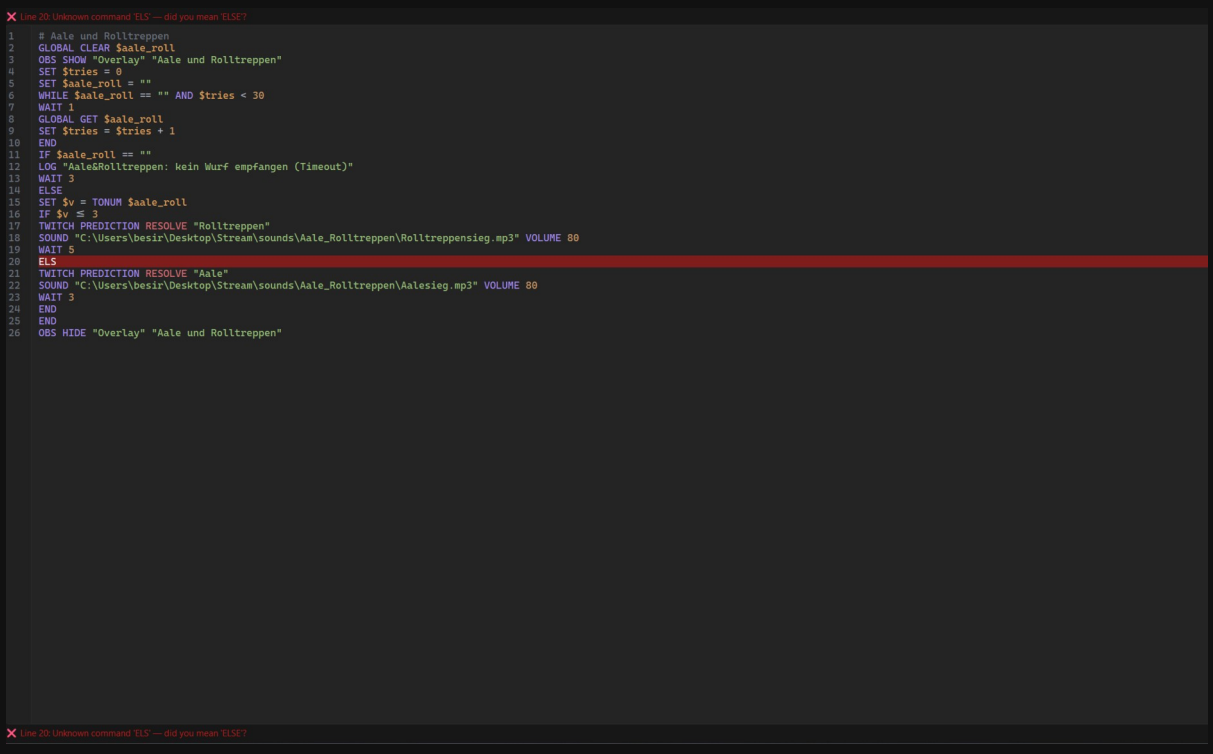
11.3 The editor lends a hand

While you type, the editor takes work off your hands:

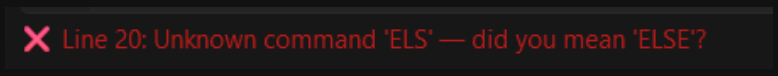
- It suggests **commands, conditions and variables** while you write.
- If you type a `"`, it immediately puts the **second quotation mark** after it — you simply keep writing in between. The same goes for brackets `()`.
- After an `IF`line it indents the next line automatically.

11.4 Finding errors

If you press **Run** and there is an error in the script, you do not have to guess: ASTO-BOT marks the line in question **red** and writes at the bottom what is wrong — and usually what was probably meant as well.



A typo in line 20 — the line is marked red

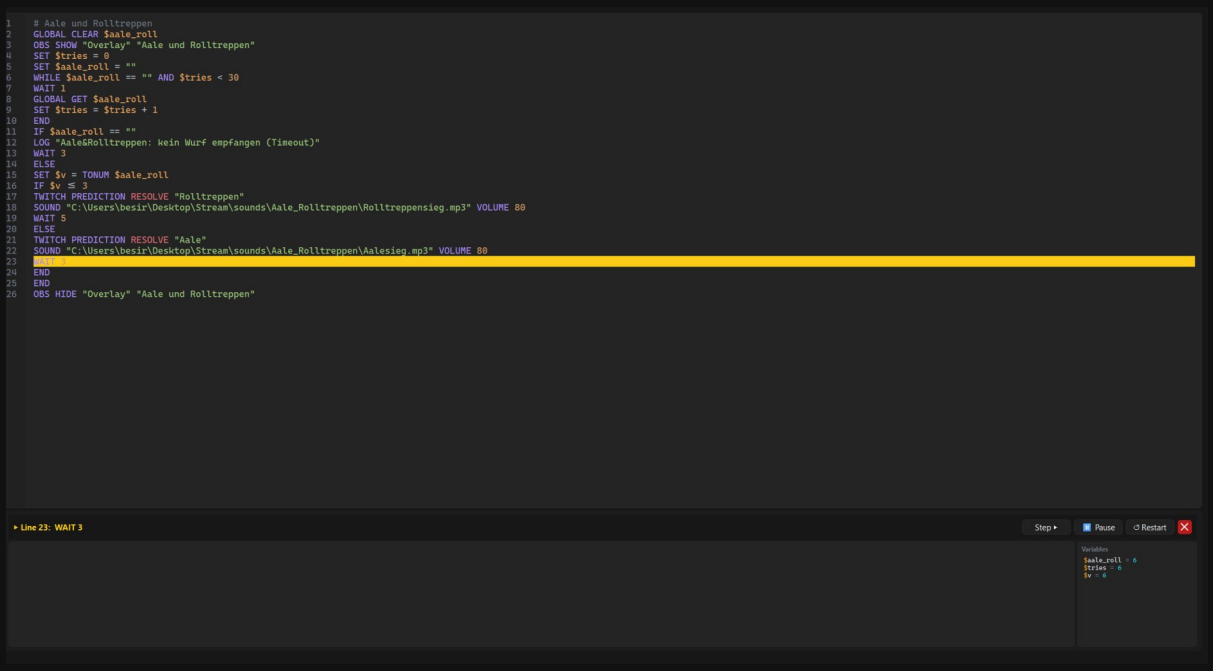


The error message names the line, the problem and a suggestion

Fix the error, press **Run** again — the marking disappears and the script runs.

11.5 Debug mode

Debug lets you look over the script's shoulder: it then runs line by line, and at any moment you see where it stands and what is inside the variables.



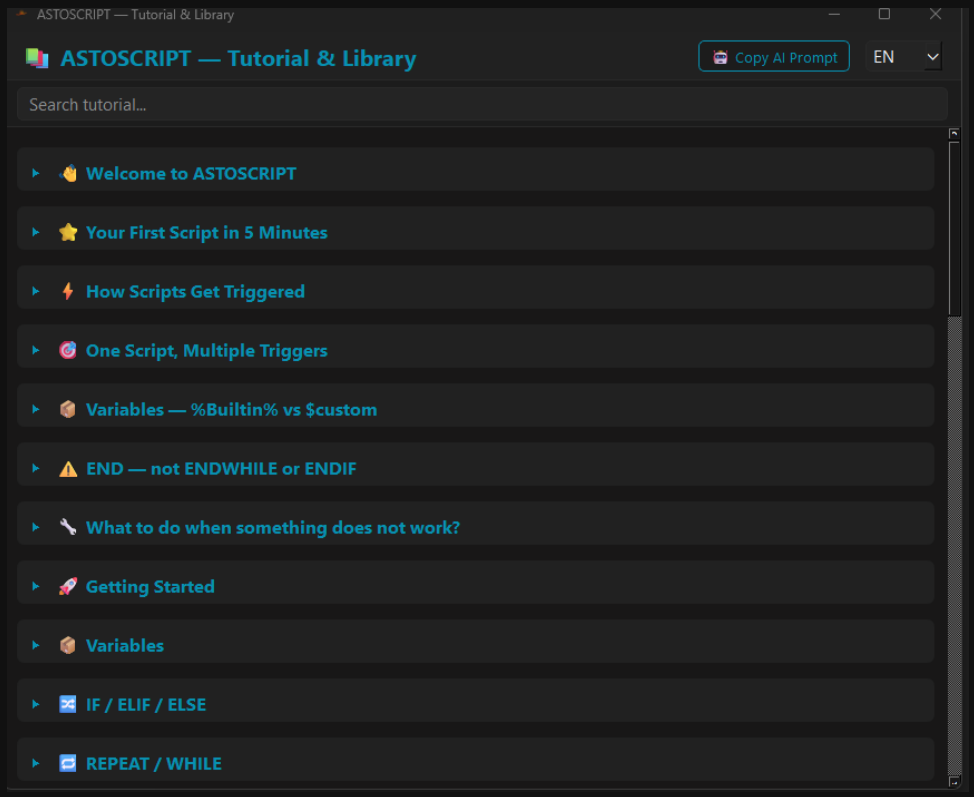
Debug mode: the current line marked, the controls and the variables at the bottom

- The line currently running is highlighted in **yellow**, and at the bottom left it is spelled out once more.
- **Step** — moves on one line.
- **Pause** — holds.
- **Restart** — starts over from the beginning.
- On the right, under **Variables**, you see **live** what value every variable currently holds.

> 💡 If a script runs but does something other than intended, debug mode is the shortest way to the answer. Usually you see after three steps that a variable holds something other than you assumed.

11.6 The tutorial window

The **Tutorial** button opens the built-in manual for ASTOSCRIPT — with a search field, sorted by topic, from the first steps to the fine points.



The tutorial — reference work and introduction in one

- In the top right you switch the language: **EN** or **DE**.
- The chapters range from *Welcome to ASTOSCRIPT* and *Your First Script in 5 Minutes* through *How Scripts Get Triggered* to the individual building blocks of the language — variables, `IF / ELIF / ELSE`, `REPEAT / WHILE` and so on.
- The search field at the top finds every command.

> **The complete list of commands is in the tutorial window, in the Glossary / Library section.** All ASTOSCRIPT keywords are sorted by category there and explained one by one — in German and English. This manual deliberately does not repeat them: the glossary in the program grows with every version and is therefore always more current than any printed list.

Copy AI Prompt

The **Copy AI Prompt** button is the secret star of this window. It copies a ready-made prompt to the clipboard that you can hand to any AI that can program — Claude, Gemini or whatever you use. After that the AI knows how ASTOSCRIPT works and can write scripts for you or repair yours.

> 💡 This is the fastest way in if you have never programmed: copy the prompt, give it to the AI, describe what you want — then paste the result in here and test it with **Run**.

11.7 Starting scripts from events

A script nobody starts does nothing. The way there is the same as with everything else: through an **event**. In the Action Sequence you insert the action **Run Script** (group *System*, chapter 5.6), pick your script — done.

The **Wait** toggle there decides whether the event waits until the script has run through or carries straight on with the next action.

One script, many events

A common misunderstanding: you do **not** need one script per event. The same script can be started by **any number** of events — by a chat command, by a channel point reward, by a timer, all at the same time. You write it **once** and hook it in wherever you need it.

So that the script knows **who** just started it, there is the variable **`%EventName%`** — it holds the ID of the firing event. That way a single script can take on several jobs and branch internally. What that looks like in practice is shown by the example in section 11.9.

And there is one more way to a script: events can also be fired from outside by **websocket** — with a key on the Stream Deck, for example (chapter Settings → Websocket). That lets you start a script without touching chat at all.

11.8 Two scripts from real life

To finish, two scripts that show what ASTOSCRIPT is good for — not to type out, but to look at.

Weather from chat

A viewer types the name of a city, the script fetches the coordinates, then the weather data — and has the AI turn it into a readable sentence for chat. Without an API key, using public interfaces only.

```
# City from chat (make spaces URL-safe)
SET $city = %UserInput%
SET $city = REPLACE $city " " "+"

# 1) City -> coordinates (Open-Meteo geocoding, no API key)
SET $geourl = "https://geocoding-api.open-meteo.com/v1/search?name=" + $city +
"&count=1&language=en&format=json"
HTTP GET $geourl $geo

SET $lat = JSON $geo "results[0].latitude"
SET $lon = JSON $geo "results[0].longitude"
SET $city_name = JSON $geo "results[0].name"
SET $country = JSON $geo "results[0].country"

IF $lat == ""
CHAT "City not found: " + %UserInput%
STOP
END

# 2) Weather for 3 days (Open-Meteo forecast, no API key)
SET $wurl = "https://api.open-meteo.com/v1/forecast?latitude=" + $lat + "&longitude=" + $lon
SET $wurl += "&daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max"
SET $wurl += "&forecast_days=3&timezone=auto"
HTTP GET $wurl $weather

# Sanity check: did a temperature come back?
SET $t0 = JSON $weather "daily.temperature_2m_max[0]"
IF $t0 == ""
CHAT "Weather data could not be retrieved."
STOP
END

# 3) Let the AI format it (the numbers are exact, it only dresses them up)
SET $p = "You are %BotName% and you answer in the style: %PersonaStyle%."
SET $p += " Real 3-day weather data for " + $city_name + " (" + $country + "): " + $weather
SET $p += " Give temperatures and rain probability EXACTLY as in the numbers, invent NOTHING."
SET $p += " Short clear answer for Twitch chat in English. 250 characters maximum."
AI PROMPT $p $answer
CHAT $answer
```

The interesting part is the division of labour: the **numbers** come from the interface and are exact, the **AI** is only allowed to wrap them into a nice sentence — and is expressly told to invent nothing.

Making the Elgato light blink

This script talks to an Elgato light directly on the network: a random number of blinks at a random rhythm, then a finale — a perfect channel point reward.

```
# Random Flash Script - random blink pattern
SET $url = "http://192.168.0.242:9123/elgato/lights"
SET $on  = "{\"numberOfLights\":1,\"lights\":[{\"on\":1,\"brightness\":100,\"temperature\":143}]}"
SET $off = "{\"numberOfLights\":1,\"lights\":[{\"on\":0,\"brightness\":100,\"temperature\":143}]}"

# Random number of blinks: 5-30
SET $blinks = RANDOM 5 30

REPEAT $blinks
HTTP PUT $url $on
SET $delay_on = RANDOM 50 1000
WAIT $delay_on MS

HTTP PUT $url $off
SET $delay_off = RANDOM 50 500
WAIT $delay_off MS
END

# Finale after a random pause (1-10 seconds)
SET $finale_pause = RANDOM 1 10
WAIT $finale_pause

# 3x quick blinks to finish
REPEAT 3
HTTP PUT $url $on
WAIT 100 MS
HTTP PUT $url $off
WAIT 100 MS
END

HTTP PUT $url $on
```

> 💡 Both scripts show the same pattern: ASTOSCRIPT talks to the world through `HTTP` — to weather services, to lamps, to anything with an interface. What you do with that is your business.

11.9 A real example: the YouTube song queue

To finish, a complete system that shows how the parts play together: viewers send YouTube links into chat, ASTO-BOT puts them into a queue, plays them one after another and says at any time what is playing and who it came from.

The special thing about it: it is **one single script** — but **four events** start it. Which one it was, the script learns from `%EventName%`, and it branches internally after that.

What you create for it

- Four **Events** with the IDs entered in the script above — in the example `test1` to `test4`. Each of them gets a **Chat Command** as its trigger and **one** action: **Run Script** with this script.
- `test1` — **add a song**. The viewer sends the link along, it ends up in `%UserInput%`.
- `test2` — **play the next song**.
- `test3` — **reset the queue**.
- `test4` — **what is playing right now?**

> In the script above, enter the `$evAdd`, `$evNext`, `$evReset` and `$evNp` the **real IDs of your events** — the ID sits on the right in the event panel below the name (chapter 5.2). If they do not match, nothing happens, because none of the four conditions is true.

How it works

- The queue lives in **two text files** — one with the links, one with the names of the viewers. Both lists are separated with `|` and go hand in hand line by line: the third link belongs to the third name.

- `FILE READ` brings them in, `LIST SPLIT` turns them into lists, `LIST APPEND` appends, `LIST POP` takes the next one out, `LIST JOIN` writes them back together — and `FILE WRITE` saves.
- The `TRY` / `CATCH` around it catches the case that the files do not exist yet — which is how it is on the very first run.
- `CLOSE_TASK` closes the browser, `OPEN` opens the next link. In between a `WAIT`, so the browser has time to shut down.
- `GLOBAL SET` remembers **who** sent in the song that is playing — beyond the end of the script. Only that way can event 4 still say later where the song came from.
- `MUSIC NOW` reads what is actually playing and fills `%SongTitle%` and `%SongArtist%`.

```
# ========================================
# SCRIPT: YT Song Queue (all-in-one, file based)
# Called by all 4 events.
# Branches on %EventName% (= signal ID of the event).
# The queue is stored in two text files, separated with "|".
# ========================================

# --- Configuration: enter your real event IDs here ---
SET $evAdd   = "test1"   # add video
SET $evNext  = "test2"   # next song
SET $evReset = "test3"   # reset queue
SET $evNp    = "test4"   # song info

# --- More variables ---
SET $browserProcess = "brave.exe"
SET $waitSeconds = 2
SET $domain1 = "youtube.com"
SET $domain2 = "youtu.be"
SET $sep = "|"

# --- File paths for the queue ---
SET $fileUrls  = "Config/song_queue_urls.txt"
SET $fileUsers = "Config/song_queue_users.txt"



# --------------- 1) ADD ---------------
IF %EventName% == $evAdd
IF CONTAINS %UserInput% $domain1 OR CONTAINS %UserInput% $domain2
SET $urlsRaw = ""
SET $usersRaw = ""
TRY
FILE READ $urlsRaw $fileUrls
CATCH
SET $urlsRaw = ""
END
TRY
FILE READ $usersRaw $fileUsers
CATCH
SET $usersRaw = ""
END
LIST SPLIT $urls $urlsRaw $sep
LIST SPLIT $users $usersRaw $sep
LIST APPEND $urls %UserInput%
LIST APPEND $users %UserName%
LIST JOIN $urls $sep $urlsOut
LIST JOIN $users $sep $usersOut
FILE WRITE $fileUrls $urlsOut
FILE WRITE $fileUsers $usersOut
LIST SIZE $urls $pos
CHAT "@" + %UserName% + " your song was added to the queue! Position: " + TOSTR $pos
ELSE
CHAT "@" + %UserName% + " that is not a valid YouTube link!"
END
END



# --------------- 2) PLAY NEXT ---------------
IF %EventName% == $evNext
SET $urlsRaw = ""
SET $usersRaw = ""
```

```
TRY
FILE READ $urlsRaw $fileUrls
CATCH
SET $urlsRaw = ""
END
TRY
FILE READ $usersRaw $fileUsers
CATCH
SET $usersRaw = ""
END
LIST SPLIT $urls $urlsRaw $sep
LIST SPLIT $users $usersRaw $sep
LIST SIZE $urls $qsize

IF $qsize == 0
CHAT "The queue is empty! No more songs."
ELSE
LIST POP $urls $nextUrl
LIST POP $users $nextUser
LIST JOIN $urls $sep $urlsOut
LIST JOIN $users $sep $usersOut
FILE WRITE $fileUrls $urlsOut
FILE WRITE $fileUsers $usersOut
GLOBAL SET $currentSongUser $nextUser
LIST SIZE $urls $remaining

CLOSE_TASK $browserProcess
WAIT $waitSeconds
OPEN $nextUrl

CHAT "Now playing: " + $nextUrl + " (requested by " + $nextUser + ", " + TOSTR $remaining + " still in the
queue)"
END
END



# --------------- 3) RESET ---------------
IF %EventName% == $evReset
FILE WRITE $fileUrls ""
FILE WRITE $fileUsers ""
CHAT "The song queue has been reset."
END



# --------------- 4) NOW PLAYING ---------------
IF %EventName% == $evNp
MUSIC NOW
GLOBAL GET $currentSongUser
IF %SongTitle% == ""
CHAT "No recognisable song is playing right now."
ELSE
IF $currentSongUser == ""
CHAT "Now playing: " + %SongArtist% + " - " + %SongTitle%
ELSE
CHAT "Now playing: " + %SongArtist% + " - " + %SongTitle% + " (requested by " + $currentSongUser + ")"
END
END
END
```

> 💡 The process name at `$browserProcess` has to match **your** browser — the example says `brave.exe`. For Chrome it would be `chrome.exe`, for Firefox `firefox.exe`. And bear in mind: `CLOSE_TASK` closes the browser **completely** — so use a browser in which nothing important is open.

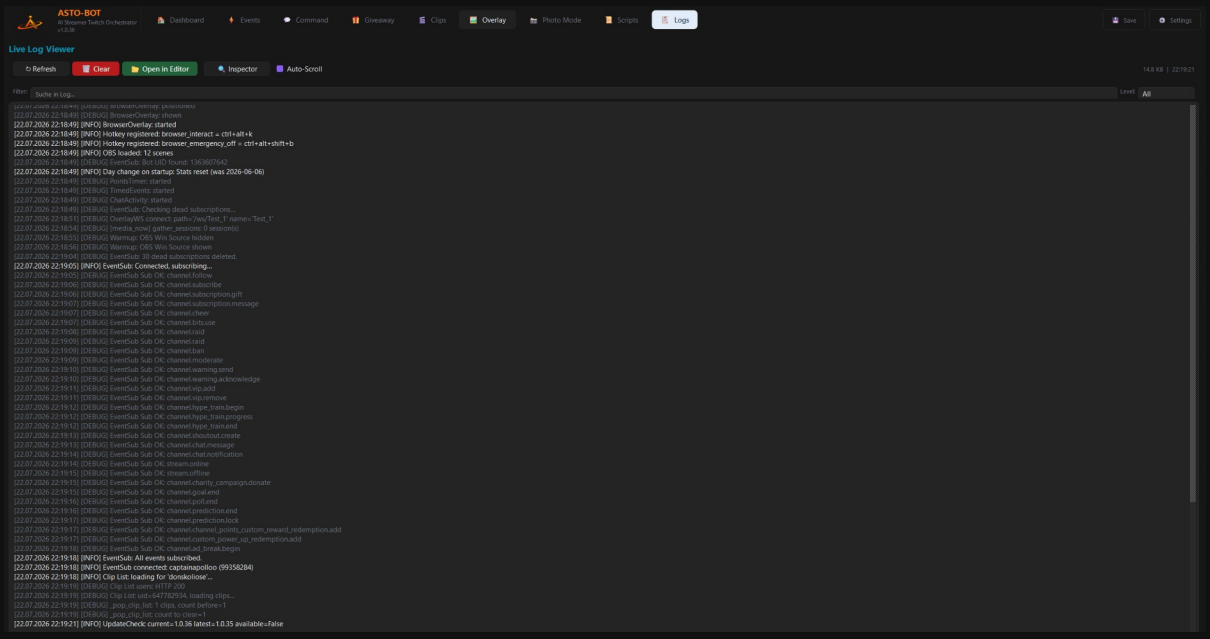
This is the point where ASTOSCRIPT shows what it is for: four chat commands, one script, two text files — and your community has a working song request. You will not get that by clicking things together.

12. Logs

The Logs page has **two views** and the button at the top switches between them: the **Live Log Viewer** — the running protocol — and the **Event Inspector**, which shows you why an event or command did *not* fire.

12.1 The Live Log Viewer

The **Live Log Viewer** is the log of ASTO-BOT: every connection, every event that fired, every setting that was saved and every error ends up here — with date, time and level.



The Live Log Viewer

- **Refresh** — reloads the log.
- **Clear** — empties the view.
- **Open in Editor** — opens the log file in your text editor.
- **Auto-Scroll** — the view jumps to the newest line automatically.
- **Filter** — searches the log for a term, the name of an event for example.
- **Level** — filters by level. In the top right you also find the size of the log file and the time of the last refresh.

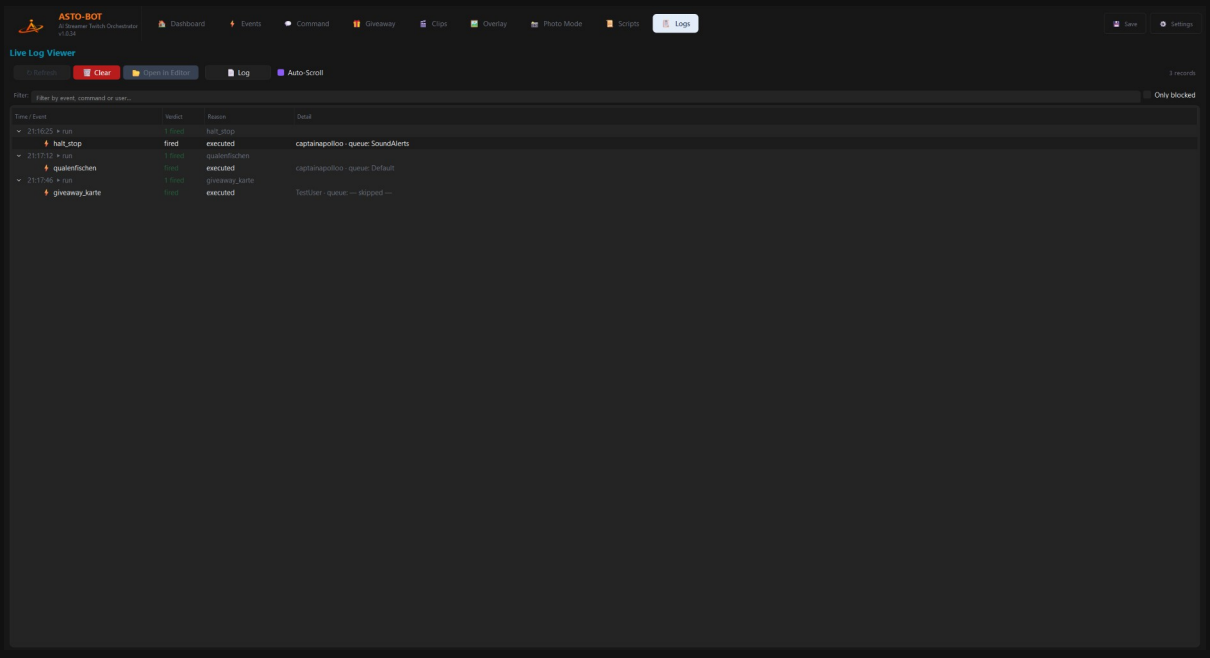
> 💡 Best to leave **Level** on **All** at all times — that way you see the most. The grey `[DEBUG]` lines may look unimportant, but they are often exactly where it says **why** an event did not fire.

And that is precisely what this area is for: when something does not work, the log is the first address. Type the name of the event into the filter and look at what ASTO-BOT did at that moment. Usually the answer is right there in plain words.

> If you ask for help on Discord, it is best to attach the relevant excerpt from the log right away. That saves a lot of back and forth.

12.2 The Event Inspector

The log answers the question **"what happened?"**. The Event Inspector answers the other one, which comes up far more often: **"why did nothing happen?"** — the command was typed but the bot stays silent; the event is set up but nothing occurs. Those are exactly the cases spelled out here.



The Event Inspector — every row shows what came in and what became of it

The 🔍 **Inspector** button at the top switches over; it then turns into 📄 **Log** for the way back. **Clear** always empties only the view that is currently open — never both.

How to read the tree

Every **top-level row** is one incoming trigger: a chat message, a trigger or an event run. Below it hang the **candidates** — everything that reacted to it, or would have. With IF/ELSE you can open one level deeper: that is where the individual **conditions** are.

- 💬 **chat** — a chat message came in. Next to it you see who wrote it and what it said.
- ▶ **run** — an event was executed.
- ❯ **Command**, 🔤 **word trigger**, ⚡ **event** — the kind of candidate.
- 🔀 **IF/ELSE** and 🎲 **Chance** — conditions and random blocks from the action sequence.

The **Detail** column shows, depending on the row, the viewer who triggered it and the **queue** the event ran on. If it says *— skipped —*, the event bypassed the queue (section 5.5).

The Verdict column

- Green **fired** — it went through.
- Red **blocked** — it was stopped. The **Reason** column tells you why.
- Orange **waiting** — neither executed nor discarded: the event sits in a **paused** queue and waits there until you switch it back on.

The most common reasons

- **disabled** — event or command is switched off. By far the most common case.
- **permission** — the role is not enough. Next to it you see what was required, for example *needs Follower*.
- **cooldown** and **user cooldown** — the lock is still running; the remaining time is shown next to it.
- **only-list** — the command is restricted to certain names.
- **event cooldown** — the cooldown of the event itself.
- **ignore list** — this viewer is on the ignore list of this event.

- **shared-chat scope** — the trigger is limited to your own channel but came from a shared chat.
- **queue paused** — the queue has been halted.
- **condition false** — a condition in the IF/ELSE was not met. Open the row: field by field it shows which **actual value** was compared against which **expected value**.

> 💡 The filter at the bottom searches event names, commands and viewer names all at once. And the **Only blocked** switch hides everything that worked — leaving exactly what you are looking for.

What the Inspector does not show

Only what was **a candidate at all** gets recorded. If somebody writes a sentence that matches no command whatsoever, it does not turn up here — otherwise every message would produce dozens of "did not match" rows. What you see are the **near misses**: cases where the trigger did match and something got in the way afterwards. And the successes, of course.

> The Inspector keeps the last **200** entries in memory and writes **nothing** to disk — after a restart of ASTO-BOT the list is empty. For the permanent record the Live Log Viewer is in charge.

13. When something does not work

Almost all the problems that end up on Discord are a handful of the same old stumbling blocks. Here they are — with the solution next to them.

13.1 Chat, AI and TTS

SpeakerBot reads out the wrong message

TTS always reads the **last chat action before it**. If two chat actions sit one after the other and then a TTS, only the second one is read out. Build it alternating: chat → TTS → chat → TTS.

The AI answers with a notice instead of a message

Then the **prompt of the event is empty**. **Chat (AI)** has no text field of its own — the text comes entirely from the prompt in the trigger area.

Chat says the AI is currently unavailable

Then **every** configured API key has failed — or none is entered at all. Check the keys under **Settings** → **AI** (the **API Key** field and the optional **Backup API Key**) and take a look at the **log**, where the exact reason is written (e.g. an expired key or an exceeded quota). If a second key is set, it steps in automatically; the notice only appears when none of them work anymore.

13.2 Events and commands

The chat command does nothing

Three causes, check them in this order: the command is **switched off**. There is **no event** attached to it (trigger *Chat Command*). Or a **cooldown** is still running — G-CD applies to everyone, U-CD per viewer.

The event does not fire at all — and I do not know why

Look into the **log** (Chapter 12), **Level: All**, type the event name into the filter. It says there in plain words what ASTO-BOT did at that moment.

After a scope change nothing works any more

When new permissions are added, Twitch requires a **complete re-login of both accounts** — streamer and bot.

13.3 Rewards and giveaways

I cannot edit or delete a reward

Then it was not ASTO-BOT that created it, but Twitch itself or another tool. Such rewards can only be **linked**. In the selection window they carry a **padlock**. That is a rule from Twitch.

My changes to the reward are gone

Press **Save Reward before** you close the window.

Refund Channel Points does not work

The reward must have **Skip Queue switched off**. A redemption that has already been confirmed can no longer be refunded.

The same person always wins the giveaway

Then it is set to **Currency** with **Winner: Highest wins** — and there the **highest stake always wins**, with no chance involved. Set **Winner** to **Weighted** if smaller stakes should have a chance too, or to **Ticket** for a real raffle with an equal chance for everyone.

My winner event announces the wrong name

The winner is in **`%GiveawayWinner%`** — not in `%UserName%`.

13.4 OBS, clips and overlays

The clip is not played

Check the source under **Clip Playback**: if **both** sources are set, the **Media Source** wins. Set the other one to **– none –**. And go through the checklist for the OBS source (chapter 8.3) — **Local file** must be **off**.

OBS Save Replay Buffer saves nothing

The replay buffer must be **enabled and started** in OBS. Enabled alone is not enough.

My overlay element moves on every playback

The **pin** on the first step is **orange** — the position has not been saved. Click it until it is **green**.

The overlay always shows only the default profile picture

In the event, **Get Target Info** must sit **before Play Overlay**. Only then is the viewer's Twitch picture fetched and sent to the overlay.

After the event my screen is upside down

Monitor Flip belongs in the event at least **twice**: once for the rotation, then a **Delay**, and at the end once more with `0` to rotate it back.

Rotate OBS Source looks broken

OBS does not rotate individual sources cleanly. Take **Rotate OBS Scene**, that works reliably.

13.5 Music and scripts

ASTO-BOT does not find my music player

At that moment the player must **actually be playing sound**. If it is merely open or paused, Windows reports nothing — and ASTO-BOT sees nothing. This applies to the Music trigger, to **Now Playing** and to **Detect** in Media Control.

I deleted a script by accident

The **trash can** next to the folder button in the Scripts area restores deleted scripts.

My script runs but does something other than intended

Start **debug** mode and go through it with **Step**. On the right you see live what is in the variables — usually it is clear after three steps where it is stuck.

13.6 And if nothing helps?

Then come to the **Discord** — https://discord.gg/n2nstVwspk. It is best to attach the relevant excerpt from the **log** right away (Chapter 12) and write down what you expected and what happened instead. That saves a lot of back and forth.

ASTO-BOT — AI Streamer Twitch Orchestrator

Version 1.1.15 | Developed by CaptainApolloo | 2026

Join Apolloo's Discord Server for support and updates